

AX & Agent Infrastructure

Практическое руководство по архитектуре и эксплуатации AI-агентов



LLM • Agent Harness • MCP • Agent Substrate • Google AX • Local Inference • Security • Observability

Independent technical guide

Содержание

Книга построена от фундаментальных mental models к AX/Substrate и затем к эксплуатации. Ссылки кликабельны в PDF.

0. Как читать эту книгу и что именно зафиксировано

1. Карта современной агентной системы: от модели до кластера

2. LLM как вычислительный компонент агентной системы

3. Что такое агент: цикл, границы и критерии остановки

4. Agent harness: система вокруг LLM

5. Context, state, memory и snapshot: не смешивать

6. Tools, MCP и Skills

7. Multi-agent архитектуры без лишней магии

8. От Linux process к gVisor и microVM

9. Kubernetes: необходимый минимум для понимания AX/Substrate

10. Control plane и distributed systems: база без академической перегрузки

11. Actor model применительно к агентным workload

12. Agent Substrate: назначение и operational architecture

13. Substrate resources: Atespace, WorkerPool, ActorTemplate, Actor, Tag

14. Suspend, snapshot, resume и golden snapshot

15. Substrate networking: routing, wake-up и egress

16. AX: архитектура control plane и реальная граница с Substrate

17. AX primitives: Task, Workspace, Gateway и Model

18. Runner: PID 1 как платформенный контракт

19. Полный lifecycle Task: от YAML до suspend/resume

20. Local inference: CPU, GPU, vLLM и llama.cpp

21. Производительность inference: что измерять для 10-15+ агентов

22. Multi-agent на AX: как делегировать без task explosion

23. Security architecture: threat model agent platform

24. Identity, authorization и Human-in-the-loop

25. Observability: четыре слоя и единая correlation identity

26. Лабораторный стенд на Proxmox/KVM: Kind, k3s или kubeadm

27. Production engineering: HA, persistence, upgrades и DR

28. Performance и scaling: что реально ограничивает систему

29. Failure modes и troubleshooting handbook

30. Когда AX нужен, а когда проще другой инструмент

31. Антипаттерны

32. Практикум: 14 последовательных лабораторных работ

33. Две референсные multi-agent архитектуры

34. Mental models, glossary и cheatsheets

35. Context engineering: управление контекстом и durable state

36. MCP 2026: протокол, discovery, transport и trust boundaries

37. Безопасное выполнение tools: idempotency, approvals и attenuation полномочий

38. AX control plane: state, event flow и operational truth

39. Substrate snapshots: ownership, storage и совместимость

40. Gateway и egress: DNS, TLS и проверка реальной доступности

41. Local inference sizing: память GPU, parallelism и эксплуатационные SLO

42. Production security, secrets и disaster recovery

43. Operations runbook: диагностика по слоям и release acceptance

Приложения: аудит, checklist, источники

Как читать эту книгу и что именно зафиксировано

IMPLEMENTED / VERIFIED

MAIN DELTA where noted

ROADMAP kept separate

Эта книга построена не как пересказ README, а как инженерный учебник: сначала даётся mental model, затем ровно та фундаментальная база, без которой конкретный механизм AX или Agent Substrate превращается в «магический чёрный ящик», после чего идут эксплуатационные выводы и практика. Целевая аудитория - системные и сетевые инженеры, архитекторы инфраструктуры и разработчики платформ, которым важнее **развернуть, наблюдать, ограничивать и диагностировать агентные workload'ы**, чем изменять внутренний Go-код проекта.

Версионная база

Для воспроизводимости основной слой книги фиксируется на релизах:

Компонент	Базовая версия	Зафиксированный commit	Дата релиза	Как используем
Google AX	v0.3.1	e70162a	25.09.2026	основной release baseline
Agent Substrate	v0.3.0	csecc78	30.09.2026	основной release baseline
MCP	2026-07-28	версия спецификации	28.07.2026	протокольная база
vLLM	v0.30.0	release	22.09.2026	пример GPU inference
llama.cpp	v0.4.1 + rolling builds	release	14.09.2026	CPU/hybrid fallback

При этом `main` AX/Substrate исследован отдельно. Это необходимо, потому что оба проекта меняются быстрее обычной инфраструктурной документации. Например, current AX README уже описывает Gateway и отдельный ax-controller, в то время как часть DESIGN.md всё ещё показывает reconciliation внутри ax-server. Такие расхождения не надо «усреднять»: их надо отмечать.

В книге используются четыре метки:

- **IMPLEMENTED** - подтверждено базовой или текущей реализацией и официальной документацией.
- **MAIN DELTA** - уже есть в `main`, но могло появиться после выбранного релиза.
- **ROADMAP** - объявлено разработчиками как направление, но не должно приниматься за доступную production-функцию.

- **BACKGROUND** - общая инженерная база или рекомендуемый pattern, а не обещание AX.

Главная оговорка

AX и Substrate - pre-stable системы. AX использует `ax.io/v1alpha1`, а Substrate прямо предупреждает о быстрых breaking changes. Поэтому любые production-планы должны начинаться с pinning версий, проверки совместимости и отдельного test environment. В этой книге команды лабораторных работ являются **шаблонами воспроизводимого стенда**, но перед применением в реальной инфраструктуре необходимо сверять release notes выбранных версий.

Что книга намеренно не делает

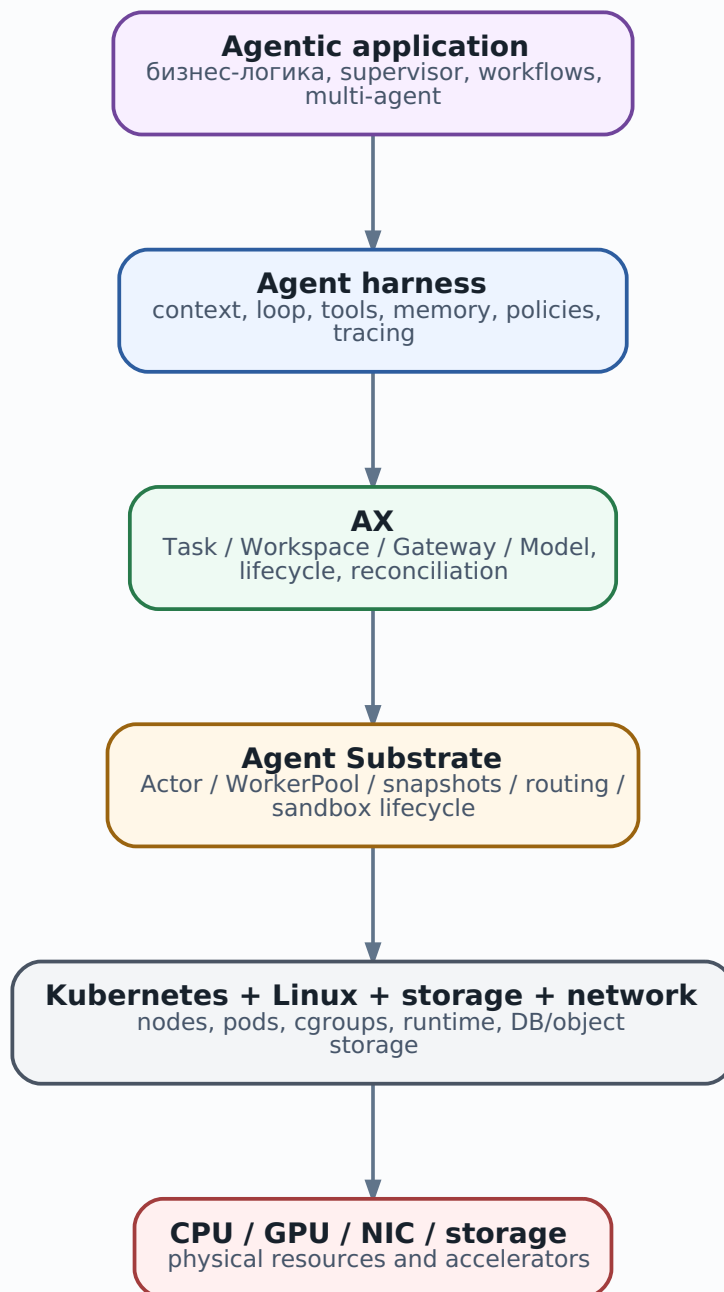
Она не заменяет полноценный курс Kubernetes, не погружается в математику Transformer'ов и не проводит строчный аудит Go-кода. Если механизм можно понять и эксплуатировать, не разбирая конкретную реализацию до функции и struct, мы остаёмся на архитектурном уровне. И наоборот: runner, lifecycle, snapshots, Gateway и trust boundaries разбираются глубоко, потому что ошибки именно здесь дают эксплуатационные и security-инциденты.

Источники и дальнейшее чтение

- [AX repository](#)
- [AX README](#)
- [AX roadmap](#)
- [Substrate repository](#)
- [Substrate architecture](#)

Карта современной агентной системы: от модели до кластера

Карта стека: где заканчивается LLM и начинается инфраструктура



Представим не демонстрационного чат-бота, а рабочую инженерную задачу. Ночью часть клиентов теряет доступ к сервису. Дежурный инженер просит агента собрать метрики, проверить DNS и TLS, сопоставить событие с последним изменением конфигурации, предло-

жить безопасное исправление и после подтверждения выполнить его. На экране это может выглядеть как один диалог. Внутри участвует целая система.

Кто-то должен принять цель и определить границы работы. Кто-то — собрать контекст для модели. Кто-то — дать модели список доступных инструментов. До выполнения команды необходимо проверить полномочия, объект воздействия и необходимость подтверждения. После вызова инструмента нужно сохранить результат, отличить временную ошибку от окончательной, решить, продолжать ли работу, и не повторить уже выполненное изменение. Сам процесс должен где-то запуститься, получить CPU и память, доступ к нужным файлам и сети, но не получить доступ ко всему кластеру. Если работа прервётся, её состояние потребует восстановления. Все действия должны остаться в журналах и трассировках.

Большая языковая модель решает лишь часть этой задачи. Она способна проанализировать переданный ей контекст, сформулировать следующий шаг или предложить вызов инструмента. Всё остальное создаётся вокруг неё. Поэтому первая полезная привычка при проектировании агентных систем — не называть словом «агент» весь стек целиком.

Одно слово скрывает несколько разных систем

В повседневной речи агентом могут назвать и prompt с циклом, и Python-программу, и контейнер, и Task в AX, и всю multi-agent платформу. Пока система работает, такая неточность кажется безобидной. Во время проектирования или инцидента она мешает определить владельца проблемы.

Удобнее разделять систему на уровни ответственности.

Уровень	Главный вопрос	Примеры обязанностей
LLM / inference	Как получить вероятностное решение или текст?	generation, structured output, tool proposal, embeddings
Agent loop и harness	Как превратить отдельные вызовы модели в управляемую работу?	context assembly, tools, state, policy, budgets, retries, stop conditions
Agent framework	Как описать сложную логику и координацию?	граф состояний, planner/executor, supervisor/worker, fan-out/fan-in
Orchestrator	Как управлять множеством отдельных выполнений?	создание, наблюдение, приостановка, возобновление, удаление, quotas
Execution runtime	Где и с какими гарантиями выполняется workload?	sandbox, process lifecycle, workspace, snapshots, routing
Infrastructure	На каких физических и кластерных ресурсах всё работает?	Linux, Kubernetes, nodes, storage, network, GPU
External systems	Где происходят реальные действия и хранится бизнес-состояние?	Git, CMDB, router API, ticket system, database, cloud API

Границы могут проходить немного иначе в разных продуктах. Важно не название слоя, а явное распределение обязанностей. Если retry одновременно реализован в model client, harness, очереди, orchestrator и внешнем proxy, одна временная ошибка способна породить

несколько одинаковых действий. Если ни один слой не считает себя владельцем retry, задача остановится после первого сетевого сбоя.

Что на самом деле делает LLM

LLM получает конечную последовательность токенов и вычисляет вероятное продолжение. Современный model API может вернуть обычный текст, структурированный объект или предложение вызвать инструмент. Модель может написать:

```
{
  "tool": "restart_service",
  "arguments": {
    "service": "edge-gateway",
    "region": "eu-1"
  }
}
```

Этот ответ ещё не является перезапуском. Он выражает решение модели на основании того контекста, который ей показали. Модель сама по себе не доказывает, что сервис существует, что регион выбран правильно, что пользователь имеет соответствующие полномочия, что изменение разрешено сейчас и что такой вызов не был выполнен минуту назад.

Между предложением модели и реальным воздействием должна находиться обычная проверяемая программа. Она валидирует schema и значения, определяет требуемое разрешение, запрашивает approval, добавляет idempotency key, вызывает инструмент, сохраняет внешний идентификатор операции и затем проверяет результат. Фраза в system prompt «не перезапускай production без разрешения» полезна как инструкция модели, но не является технической границей безопасности.

Поэтому LLM разумно считать заменяемым вычислительным компонентом. Она помогает выбирать следующий шаг, классифицировать данные, строить гипотезы и формировать объяснения. Authoritative state, полномочия и подтверждение side effects должны находиться вне неё.

Когда появляется агент

Минимальный агент возникает тогда, когда отдельные вызовы модели соединяются в цикл обратной связи:

```
получить цель
  ↓
прочитать состояние и наблюдения
  ↓
собрать контекст
  ↓
получить решение модели
  ↓
проверить и выполнить действие
  ↓
зафиксировать результат
  ↓
продолжить, приостановиться или завершить работу
```

В этом цикле могут присутствовать и полностью детерминированные ветви. Например, превышение бюджета не обсуждается с моделью: программа завершает задачу. Операция, требующая подтверждения, переводит работу в состояние `WaitingForApproval`. Повторный ответ внешней системы с тем же operation ID не должен восприниматься как новое действие.

Чтобы цикл был эксплуатационно пригодным, для него необходимо определить как минимум:

- цель и входные данные;
- допустимые действия;
- текущее состояние;
- правила формирования контекста;
- бюджеты времени, токенов и шагов;
- обработку ошибок и повторов;
- критерии остановки;
- место хранения результатов;
- правила выполнения внешних изменений;
- способ наблюдения и аварийной остановки.

Программа с одним вызовом модели может быть полезной, но ей не обязательно становиться агентом. Если задача решается детерминированным parser, SQL-запросом или обычным workflow, добавление цикла с LLM увеличит стоимость и число вариантов отказа, не дав полезной адаптивности.

Harness: система, которая делает цикл управляемым

Agent harness — это программный слой вокруг модели, который реализует цикл агента и превращает вероятностные ответы в контролируемое выполнение. Это не обязательно отдельный продукт. Для небольшого агента harness может быть несколькими модулями собственного приложения. Для сложной платформы — большим набором сервисов и библиотек.

У harness обычно есть несколько обязанностей.

Context assembly решает, что попадёт в следующий model request: system policy, цель, актуальное состояние, недавняя история, найденные документы и результаты инструментов. Он же должен не допустить, чтобы случайный README или ответ внешнего MCP server получил тот же уровень доверия, что и platform policy.

Model adapter скрывает технические различия model providers и локальных inference servers: форматы сообщений, tool schemas, streaming, лимиты, ошибки, timeouts и метрики. Замена модели не должна требовать переписывания всей бизнес-логики.

Tool executor принимает предложение модели, валидирует аргументы и выполняет реальный вызов. Он связывает запрос модели с operation ID, timeout, журналом и результатом.

State manager хранит progression задачи. В нём находятся факты, уже принятые решения, открытые вопросы, выполненные side effects и данные, необходимые после перезапуска.

Policy и approval layer определяет, может ли действие быть выполнено. Read-only запрос метрик и изменение маршрутизации — разные классы полномочий, даже если оба представлены как tools.

Budget и retry control ограничивает число шагов, время, стоимость и глубину делегирования. Он отличает повтор безопасного чтения от повтора операции, которая могла уже изменить внешнюю систему.

Observability связывает task ID, model request, tool call, approval и внешний operation ID. Без такой корреляции инженер видит пять разрозненных журналов и не может восстановить ход работы.

Termination logic отвечает на неудобный, но обязательный вопрос: когда работа завершена? Ответ модели «готово» недостаточен. Критерий должен проверяться по результату: тесты прошли, устройство вернуло ожидаемую конфигурацию, артефакт создан, итог подтверждён независимым чтением.

Harness управляет шагами внутри одного выполнения. Он не обязан создавать контейнеры, выбирать Kubernetes node или восстанавливать sandbox после потери Worker. Эти задачи относятся к другим слоям.

Harness и agent framework — не синонимы

Agent framework предоставляет готовые абстракции для построения логики: граф состояний, поли, messages, planner/executor, supervisor/worker, сохранение checkpoints или интеграции с tools. Harness — эксплуатационная оболочка, которая должна существовать независимо от того, используется framework или нет.

Framework может стать частью harness, но не заменяет автоматически policy enforcement, секреты, сетевую изоляцию, бюджеты или end-to-end observability. И наоборот, простой агент иногда не нуждается в тяжёлом framework. Цикл из нескольких явно написанных состояний легче проверить и сопровождать, чем универсальный граф, если задача небольшая и хорошо определена.

Выбирать framework стоит, когда его модель действительно совпадает с задачей: требуется ветвящийся workflow, долговременное состояние, человеческие подтверждения, повторное выполнение отдельных узлов или сложная координация ролей. Если нужен один model call, три deterministic проверки и один API-запрос, собственный небольшой harness обычно прозрачнее.

Tools и MCP: доступ к возможностям, а не передача власти модели

Tool — это операция, которую harness умеет предложить модели и затем выполнить. MCP стандартизует способ обнаружить такие возможности и обмениваться запросами с внешними серверами. Ни tool schema, ни MCP сами по себе не решают вопрос доверия.

Рассмотрим диагностический MCP server. Он предлагает tools `get_interface_counters` и `set_interface_state`. Первый читает telemetry, второй меняет состояние порта. Если агенту передать оба инструмента с одними и теми же credentials, различие между чтением и изменением останется лишь в описании, которое видит модель. Надёжная система выдаёт раз-

ные capabilities, проверяет объект и scope операции, требует approval для write и использует короткоживущие полномочия.

Результат tool call также нельзя автоматически считать истинным. Сервер может вернуть устаревшие данные, ошибку внутри поля `result`, неполный ответ или вредоносный текст. Harness сохраняет provenance: какой источник дал факт, когда он был получен и каким запросом. Внешнее действие завершается не успешным HTTP-ответом, а проверкой intended outcome.

Состояние существует сразу в нескольких местах

Фраза «агент помнит задачу» слишком неоднозначна. В реальной системе есть несколько типов состояния.

Model context — токены конкретного запроса. Он ограничен и пересобирается.

Working state — текущая стадия задачи, накопленные факты, план, счётчики и ожидаемые события.

Persistent memory — долговременные записи, которые можно найти и использовать позже.

Workspace — файлы, репозитории, конфигурация инструментов и создаваемые артефакты.

Process state — память и открытые ресурсы конкретного процесса.

Runtime snapshot — технический checkpoint, который runtime способен сохранить для ускоренного resume.

External state — реальные изменения в Git, базе, тикетной системе или сетевом устройстве. Он не откатывается восстановлением локального snapshot.

Это различие определяет архитектуру восстановления. Если процесс запускается заново, working state должен быть прочитан из durable store или workspace. Если восстанавливается RAM image, внешнее TCP-соединение всё равно может оказаться закрытым, а token — просроченным. После любого resume программа обязана перепроверить внешние предположения.

Orchestrator управляет выполнениями, а не мыслями модели

Когда один harness нужно запускать много раз, появляется следующий класс задач:

- создать изолированное выполнение;
- назначить ему ресурсы и Workspace;
- выдать секреты и сетевые правила;
- отслеживать lifecycle;
- приостанавливать idle workload;
- восстанавливать его по событию;
- ограничивать число дочерних задач;
- удалять завершённые окружения;
- связывать их журналы и стоимость.

Это обязанности orchestrator и execution runtime. Они не должны зависеть от конкретного reasoning pattern агента. Один Task может запускать собственный Python harness, CLI-agent или приложение на framework. Платформа управляет workload, но не обязана понимать внутренний plan модели.

Multi-agent система добавляет ещё один уровень. Coordinator может создать Researcher, Diagnostic и Validator, но техническая возможность создать дочерний Task не отвечает на вопросы: кто имеет право делегировать, какой максимальный depth дерева, кто наследует credentials, как отменяются дочерние работы и что происходит при частичном успехе. Эти правила относятся к harness/governance и orchestrator contract одновременно, поэтому граница должна быть явной.

Runtime и sandbox отвечают за место выполнения

Контейнер упаковывает приложение и его зависимости, но сам по себе не определяет всю security model. Для agent workload важны:

- изоляция процессов и файловой системы;
- CPU, память и disk quotas;
- capabilities и системные вызовы;
- доступ к секретам;
- разрешённые сетевые направления;
- lifecycle дочерних процессов;
- durable и ephemeral storage;
- способ диагностики;
- поведение при suspend/resume;
- очистка после завершения.

Обычный container может быть достаточен для доверенного внутреннего агента с read-only инструментами. Для выполнения непроверенного кода из пользовательского репозитория граница может потребовать gVisor или microVM. Более сильная изоляция обычно означает дополнительную сложность, ограничения совместимости и стоимость запуска. Правильный выбор зависит от threat model, а не от моды.

Kubernetes находится ещё ниже. Он управляет nodes, Pods, Services, volumes и инфраструктурными политиками. Kubernetes не знает, что процесс внутри Pod выполняет agent loop, ожидает approval или исчерпал token budget. Специализированная платформа использует Kubernetes там, где сильны его механизмы, и добавляет agent-aware lifecycle выше.

Сквозной путь одной задачи

Вернёмся к инциденту с недоступностью сервиса.

1. Пользователь или automation создаёт задачу и задаёт допустимый score.
2. Orchestrator создаёт execution и связывает его с Workspace, моделью и политикой.
3. Runtime назначает sandbox, подготавливает filesystem, identity и network access.
4. Runner запускает команду агента и сообщает readiness.
5. Harness загружает objective и durable state, затем собирает первый context.

6. LLM предлагает собрать telemetry.
7. Policy разрешает read-only tools; executor вызывает MCP server.
8. Harness сохраняет observations и provenance, после чего модель формирует гипотезу.
9. Модель предлагает изменение. Policy переводит задачу в ожидание approval.
10. После подтверждения executor выполняет idempotent write с ограниченной capability.
11. Отдельное чтение проверяет реальное состояние системы.
12. Harness завершает работу и создаёт итоговый отчёт.
13. Orchestrator сохраняет артефакты и освобождает runtime resources.

Жизнь Task: от декларации до проверенного внешнего результата



Эта цепочка показывает, почему «ответ модели был правильным» не равнозначно «задача успешно выполнена». Успех является end-to-end свойством всей системы.

Три плоскости, которые полезно не смешивать

Control plane хранит desired state и управляет lifecycle: создать Task, назначить runtime, изменить статус, приостановить или удалить.

Execution plane выполняет код: runner, harness, процессы, filesystem и model/tool clients.

Side-effect plane включает системы, где происходят реальные изменения: Git provider, network controller, cloud API, ticketing или production database.

Разделение особенно полезно при повторах и восстановлении. Control plane может повторно отправить событие после сбоя. Execution может не знать, успел ли внешний API завершить операцию до timeout. Side-effect plane должен поддерживать idempotency или предоставлять operation status, а harness — хранить ledger вызовов.

Как выбирать уровень инструмента

Инженерный выбор начинается не с названия продукта, а со свойств задачи.

Ситуация	Обычно достаточно	Почему
Один предсказуемый шаг	Скрипт или обычный сервис	Минимум состояния и вариантов отказа
Надёжная последовательность известных шагов	Workflow engine	Нужны durable retries и orchestration, но не вероятностное планирование
Короткая изолированная вычислительная работа	Kubernetes Job	Хорошо известный lifecycle run-to-completion
Один адаптивный цикл с несколькими tools	Собственный небольшой harness	Логику можно сделать явной и проверяемой

Ситуация	Обычно достаточно	Почему
Сложный граф, branching и human approval	Agent framework + harness	Готовые state/checkpoint abstractions сокращают собственную реализацию
Много изолированных, stateful и часто idle executions	Специализированный orchestrator/runtime	Нужны lifecycle, multiplexing, suspend/resume и policy на уровне платформы
Непроверенный код и сильная tenant boundary	Усиленный sandbox, возможно gVisor/microVM	Обычного process/container boundary может быть недостаточно

Эта таблица не является автоматическим решателем. Она заставляет сформулировать требования. Например, пять доверенных внутренних агентов можно запускать обычными deployments или worker queue. Специализированная платформа становится интереснее, когда executions много, они stateful, долго простаивают, запускают непроверенный код, требуют независимых workspaces или должны быстро приостанавливаться и возобновляться.

Не стоит выбирать AX только потому, что задача содержит LLM. Не стоит выбирать multi-agent только потому, что можно создать несколько ролей. Каждый новый слой должен устранять конкретное ограничение более простой архитектуры.

Что AX и Agent Substrate добавляют к этой карте

В зафиксированном baseline AX `v0.3.1` пользователь декларативно описывает `Task`, `Workspace` и `Model`. AX принимает эти ресурсы, хранит собственное control-plane состояние и через Agent Substrate приводит Task к исполняемому состоянию. Внутри Task запускается runner, который подготавливает Workspace и стартует команду пользователя.

AX не определяет reasoning strategy. Он не превращает любой container в качественного агента, не реализует за приложение надёжные tools и не решает автоматически governance. Harness остаётся ответственностью приложения или выбранного framework.

Agent Substrate находится ниже AX. Его основная модель отделяет logical Actor от физического Worker. Большая совокупность actors может отображаться на меньший pool заранее подготовленных workers. Runtime управляет lifecycle, sandbox, placement, suspend/resume и routing. Kubernetes при этом продолжает управлять узлами и worker Pods.

Такое разделение полезно для workloads, которые активны всплесками и долго ждут событий. Но оно создаёт новые задачи: хранение snapshots, совместимость при restore, data locality, correlation logs между разными workers и собственный control plane. Архитектура выбирается не бесплатно: плотность и быстрый resume оплачиваются дополнительной системой, которую потребуется эксплуатировать.

Проекты остаются pre-1.0, а часть архитектурных документов описывает целевое состояние раньше полной реализации. Поэтому конкретные возможности нужно проверять по зафиксированной версии, API guide и реальному стенду. В книге design goals, реализованное поведение и roadmap рассматриваются отдельно.

Почему симптом «агент завис» ничего не объясняет

Один пользовательский симптом может иметь причины на любом уровне.

- **LLM/inference:** очередь на GPU, oversized context, rate limit или model timeout.
- **Harness:** потерян stop condition, повторяется один plan, ожидается approval, исчерпан budget.
- **Tool/MCP:** сервер недоступен, schema изменилась, credential просрочен, операция осталась в неизвестном состоянии.
- **Runner:** служебный процесс жив, но child agent завершился.
- **Network policy:** DNS работает, но TLS или конкретный egress destination запрещён.
- **Runtime:** Actor не получил Worker, snapshot несовместим или restore слишком медленный.
- **AX control plane:** событие не обработано, controller отстаёт, ссылка на Workspace не разрешена.
- **Kubernetes:** node pressure, image pull, storage или CNI.
- **Внешняя система:** действие принято асинхронно и ещё не завершилось либо было отвергнуто собственной policy.

Поэтому диагностика начинается с correlation identity и определения последнего подтверждённого перехода. Получил ли Task desired state? Назначен ли Actor? Готов ли runner? Жив ли child process? Был ли отправлен model request? Какой tool call выполняется? Есть ли внешний operation ID? Только после ответа на эти вопросы имеет смысл изменять prompt.

Метод выбора архитектуры

Перед добавлением очередного слоя полезно письменно ответить на восемь вопросов.

1. Какая неопределённость требует LLM, а что можно решить детерминированно?
2. Как выглядит проверяемый результат работы?
3. Где находится authoritative state?
4. Какие действия имеют side effects и как обеспечивается idempotency?
5. Какой код считается недоверенным и какая нужна граница изоляции?
6. Сколько executions одновременно активно, сколько простаивает и как долго?
7. Что должно пережить restart, suspend и потерю node?
8. Кто будет наблюдать, обновлять и восстанавливать каждый новый компонент?

Если ответы не требуют долговременного агента, отдельного sandbox runtime и массового lifecycle, более простая система почти всегда лучше. Если же нужны stateful executions, независимые workspaces, управляемая приостановка, строгие policies и большое число idle workloads, специализированный стек становится обоснованным.

Практическое упражнение

Возьмите задачу: «агент проверяет pull request, запускает тесты и при необходимости предлагает исправление». Разложите её на таблицу:

- где выполняется agent loop;

- что входит в context;
- где хранится progression;
- кто запускает тесты;
- имеет ли агент право писать в branch;
- кто подтверждает merge;
- какой уровень изоляции нужен для кода из pull request;
- что произойдёт после restart;
- как проверить, что результат действительно опубликован;
- нужен ли здесь AX или достаточно CI runner и небольшого harness.

Затем измените условие: одновременно работают десять тысяч таких задач, большинство ждёт ревью несколько часов, а код считается недоверенным. Посмотрите, какие ответы изменятся. Именно изменение требований, а не наличие слова «agent», приводит к выбору другой архитектуры.

Если запомнить только главное

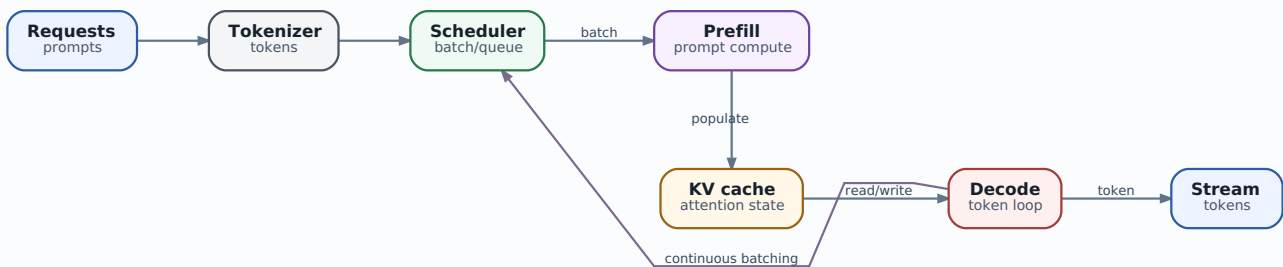
- LLM предлагает решения; система вокруг неё отвечает за выполнение и последствия.
- Агент появляется как цикл с состоянием, действиями и критерием остановки.
- Harness управляет внутренним execution loop; orchestrator управляет множеством executions.
- Framework помогает описывать логику, но не заменяет security, runtime и operations.
- Context, working state, workspace, snapshot и external state — разные сущности.
- AX управляет декларативным lifecycle agent workloads, но не пишет harness за приложение.
- Agent Substrate управляет Actor/Worker, sandbox и suspend/resume поверх инфраструктуры Kubernetes.
- Выбирать инструмент нужно по требованиям к state, isolation, lifecycle, scale и side effects.
- «Агент завис» — симптом, а не диагноз.

Источники и дальнейшее чтение

- [AX README, baseline e70162a](#)
- [AX Design, baseline e70162a](#)
- [AX Core concepts, baseline e70162a](#)
- [Agent Substrate README, baseline ccecc78](#)
- [Agent Substrate architecture, baseline ccecc78](#)
- [Kubernetes Concepts](#)
- [Model Context Protocol specification](#)
- [gVisor documentation](#)

LLM как вычислительный компонент агентной системы

Путь model request: tokenizer, scheduler, prefill, KV cache и decode



Представим, что команда строит агента для первичной диагностики сетевых инцидентов. На вход он получает описание симптома, фрагменты конфигурации, метрики, результаты DNS- и TLS-проверок. На выходе должен выбрать следующий read-only инструмент, объяснить гипотезу и при необходимости подготовить изменение для подтверждения человеком.

Вопрос «какую LLM взять?» кажется главным. Команда сравнивает размер модели, длину context window и цену миллиона токенов. Но эти цифры не отвечают на более важные вопросы. Сможет ли модель правильно выбрать инструмент среди двадцати похожих? Не перепутает ли interface ID с VLAN ID? Как часто она сформирует синтаксически допустимый, но опасный вызов? Что произойдёт, если в context не поместится часть telemetry? Как изменится end-to-end latency, если каждый шаг агента передаёт модели всю историю заново?

LLM в такой системе полезно рассматривать не как «разум агента», а как вычислительный компонент с вероятностным контрактом. Она получает конечный набор токенов и возвращает продолжение: текст, структурированный объект или предложение вызвать инструмент. Всё, что связано с актуальностью данных, полномочиями, долговременным состоянием, повтором операций и проверкой результата, остаётся обязанностью программы вокруг модели.

Такой взгляд не обесценивает возможности LLM. Напротив, он позволяет использовать её там, где она сильна, и не требовать от неё гарантий, которых у неё нет.

Что происходит при одном запросе

Между массивом сообщений в приложении и первым символом ответа находится несколько стадий.

1. Harness собирает system instructions, цель, состояние, найденные документы, результаты tools и формат ожидаемого ответа.
2. Model adapter преобразует это в формат конкретного provider и применяет chat template модели.
3. Tokenizer превращает текст и специальные маркеры в идентификаторы токенов.

4. Inference server ставит запрос в очередь и назначает его вычислительным ресурсам.
5. На стадии prefill модель обрабатывает входные токены и создаёт KV cache.
6. На стадии decode модель генерирует новые токены по одному, каждый раз используя уже построенный cache.
7. Токены преобразуются обратно в текст или structured output и передаются приложению потоком либо целиком.
8. Harness интерпретирует результат, но не обязан ему доверять.

На каждом переходе есть собственные ошибки. Неверный chat template способен ухудшить instruction following даже при правильных весах. Переполненный context приводит к отказу или truncation. Очередь inference server увеличивает задержку до первого токена. Оборванный stream оставляет harness с частичным JSON. Наконец, полностью успешный HTTP-ответ может содержать убедительный, но фактически неверный вывод.

Поэтому метрика `request succeeded` доказывает только исправность transport и serving path. Она ничего не говорит об успешности инженерной задачи.

Токен - не слово и не символ

Tokenizer разбивает вход на элементы словаря модели. Распространённые семейства алгоритмов - BPE, WordPiece и Unigram - сохраняют частые фрагменты крупными токенами, а редкие последовательности раскладывают на более мелкие. Разные модели способны по-разному токенизировать один и тот же текст.

Из этого следуют несколько практических выводов.

Во-первых, длину запроса нельзя надёжно оценивать количеством слов или символов. Русский текст, исходный код, длинные идентификаторы, base64, JSON и таблицы могут иметь очень разную token density. Проверять лимит нужно tokenizer'ом конкретной model revision.

Во-вторых, сообщения приложения не попадают в модель «как есть». Chat template добавляет role markers, разделители и специальные токены. Поэтому сумма токенов отдельных строк может не совпасть с фактической длиной итогового prompt.

В-третьих, context budget расходуется не только пользовательским текстом. В него входят system policy, tools schemas, примеры, история, retrieved documents, observations и зарезервированное место для ответа. Список из сотни подробных tool descriptions иногда занимает больше контекста, чем сама задача.

Наконец, токены - единица ограничения и стоимости, но не единица смысла. Два фрагмента одинаковой длины могут иметь совершенно разную ценность. Актуальное состояние интерфейса важнее трёх страниц общей документации, даже если оно занимает всего двадцать токенов.

Context window - рабочий стол, а не долговременная память

Context window задаёт максимальный объём последовательности, доступной модели в одном запросе. Это жёсткий технический лимит, но не обещание одинаково качественной работы с каждым токеном внутри него.

Исследования long-context моделей показывают, что релевантная информация может использоваться хуже, если она находится в середине большого контекста. Кроме того, лишний текст увеличивает prefill latency, стоимость и вероятность того, что модель обратит внимание на правдоподобную, но нерелевантную деталь. Поэтому стратегия «положим в prompt всё» редко является хорошим context engineering.

Полезно заранее разделить context на зоны:

Зона	Что в ней находится	Как обращаться
Platform instructions	Неизменяемые правила и границы поведения	Формирует платформа, пользователь и документы не могут их переопределить
Task objective	Цель, score и критерий завершения	Должны быть короткими и проверяемыми
Working state	Текущая стадия, подтверждённые факты, открытые вопросы	Пересобирается из durable state
Evidence	Telemetry, найденные документы, tool observations	Содержит provenance, время получения и уровень доверия
Action surface	Доступные tools и их schemas	Ограничивается текущей стадией и полномочиями
Output contract	Schema, формат объяснения, лимит ответа	Проверяется после generation

Context нужно собирать заново для каждого шага. В него включают не всю историю, а информацию, необходимую для следующего решения. Старый диалог можно свернуть в структурированное состояние; документы - отобрать retrieval-слоем; большие tool results - предварительно нормализовать; уже выполненные side effects - хранить в отдельном ledger.

Если важный факт существует только в предыдущем model response, но не сохранён в authoritative state, система неизбежно потеряет его после compaction, restart или смены provider.

Prefill, decode и KV cache отвечают за разные задержки

Во время prefill модель обрабатывает входной prompt и вычисляет внутренние представления для его токенов. Эта стадия хорошо использует параллельные вычисления и становится особенно заметной при длинном context. Пользователь видит её как часть time to first token, или TTFT.

Decode устроен иначе. Авторегрессионная модель выбирает следующий токен, добавляет его к последовательности и только затем может вычислить следующий. Эта зависимость делает generation последовательной. Задержка между токенами измеряется как inter-token latency, или ITL.

Без KV cache на каждом decode step пришлось бы повторно вычислять attention для всей предыдущей последовательности. Cache сохраняет key/value представления уже обработанных токенов и позволяет переиспользовать их. За ускорение приходится платить памятью: cache растёт с числом активных sequences, длиной context и параметрами архитектуры модели.

Для agent workload различие prefill и decode особенно важно:

- длинные policies, tool schemas и observations увеличивают TTFT;
- длинное объяснение или большой code patch увеличивают decode time;
- несколько параллельных агентов конкурируют не только за compute, но и за KV cache;
- повтор общей части prompt может получить выгоду от prefix caching, если server и provider это поддерживают;
- новый tool result обычно требует нового prefill хотя бы для добавленного контекста;
- короткий ответ модели не означает дешёвый запрос, если перед ним был передан огромный prompt.

Inference server может объединять запросы в continuous batches, разбивать длинный prefill на chunks и приоритизировать decode. Эти механизмы улучшают использование GPU, но создают trade-off между throughput, TTFT и ITL. Глубокий capacity planning рассматривается позже; здесь важно понимать, почему «tokens per second» не описывает пользовательскую задержку целиком.

Модель выдаёт распределение вероятностей, а не факт

На каждом decode step модель оценивает вероятности следующего токена. Generation strategy выбирает один из кандидатов и продолжает цикл. Веса модели содержат статистические закономерности, полученные во время обучения, а не актуальную копию CMDB, репозитория или состояния кластера.

Из этого не следует, что любой ответ модели случаен или бесполезен. Модель может очень хорошо извлекать структуру, сопоставлять признаки, объяснять гипотезы и строить план. Но уверенная формулировка не является доказательством.

Полезно различать четыре типа утверждений:

- **дано во входе** - факт присутствует в проверенном observation;
- **получено инструментом** - факт связан с источником и временем чтения;
- **выведено моделью** - гипотеза, классификация или интерпретация;
- **проверено системой** - результат подтверждён независимым условием.

Если итоговый отчёт смешивает эти категории, инженер не понимает, чему можно доверять. Хороший harness сохраняет provenance и просит модель маркировать предположения, но окончательное разделение выполняет программа.

Пример: модель видит рост TLS errors после deployment и предполагает несовместимый cipher suite. Это полезная гипотеза. Подтверждённым фактом она станет только после чтения конфигурации и воспроизводимой TLS-проверки.

Sampling управляет разнообразием, но не создаёт гарантий

Greedy decoding выбирает наиболее вероятный токен на каждом шаге. Sampling выбирает токен из распределения. Temperature изменяет его форму, а top-p ограничивает множество кандидатов совокупной вероятностной массой. Параметр `max_new_tokens` ограничивает длину generation.

Для инфраструктурных задач обычно нужен низкий уровень вариативности. Однако правило «всегда temperature=0» слишком примитивно.

- Короткое извлечение поля или выбор tool полезно делать максимально стабильным.
- Поиск нескольких независимых гипотез может выиграть от ограниченного разнообразия.
- Генерацию пользовательского объяснения можно отделить от принятия действия.
- Несколько candidates имеют смысл только тогда, когда существует evaluator, способный их сравнить.

Даже greedy decoding не следует считать бизнес-гарантией детерминизма. Результат может измениться после обновления model revision, tokenizer, chat template, serving engine или provider-side configuration. Разные hardware kernels и batching также могут проявить различия возле близких вероятностей.

Поэтому воспроизводимость строится выше модели: фиксируются version и параметры, сохраняется фактический request, результат проверяется тестами, а side effects защищаются idempotency и preconditions. Система должна оставаться корректной, даже если модель предложила другой допустимый план.

Structured output ограничивает форму, но не смысл

Обычный текст трудно надёжно разбирать. Structured output и constrained decoding позволяют ограничить generation JSON Schema или другой грамматикой. Tool calling добавляет модельный интерфейс, в котором ответ содержит имя инструмента и аргументы.

Это серьёзно снижает количество синтаксических ошибок, но не превращает вывод модели в доверенную команду. Между model result и tool executor нужны четыре проверки.

```
{
  "tool": "set_interface_state",
  "arguments": {
    "device_id": "edge-17",
    "interface": "xe-0/0/3",
    "enabled": false
  },
  "reason": "Observed flapping after the last configuration change"
}
```

Schema validation проверяет наличие полей, типы и допустимый формат.

Domain validation проверяет, что устройство и интерфейс существуют, аргументы согласованы, а requested state имеет смысл.

Authorization and policy проверяют identity, scope, change window и необходимость human approval.

Outcome verification после вызова независимо читает состояние и убеждается, что intended result достигнут.

JSON может быть идеально валидным и при этом выключать не тот порт. Tool name может существовать, но быть недоступным текущему пользователю. Поэтому structured output -

это интерфейс между вероятностным компонентом и обычным кодом, а не обход обычного кода.

Если schema сложна, лучше разбить решение на несколько шагов. Модели проще сначала выбрать тип действия, затем заполнить узкую schema конкретного tool, чем генерировать один универсальный объект с десятками условных полей.

Способности модели нельзя свести к одному рейтингу

Model size и публичный benchmark дают ориентир, но production-выбор зависит от конкретного workload. Для agent infrastructure важны разные способности.

Способность	Как проверять на своей задаче
Instruction following	Выполняет ли модель platform policy при конфликтующем пользовательском или документном тексте
Tool selection	Выбирает ли правильный tool и умеет ли отказаться от лишнего вызова
Argument filling	Правильно ли переносит IDs, enums, units, time ranges и optional fields
Long-context retrieval	Находит ли нужный факт в типичных по длине и структуре observations
Reasoning and planning	Строит ли проверяемый план без пропущенных preconditions
Multilingual quality	Понимает ли русские описания, англоязычные логи и смешанную терминологию
Code and configuration	Корректно ли читает diff, YAML, policy и vendor-specific config
Structured output	Соблюдает ли schema и насколько часто требует repair/retry
Calibration and abstention	Признаёт ли недостаток данных вместо выдуманного ответа
Latency and efficiency	Укладывается ли TTFT и end-to-end latency в SLO при нужной concurrency

Модель, которая выигрывает общий reasoning benchmark, может хуже заполнять аргументы конкретных сетевых tools. Большая модель может немного повысить task success, но не оправдать четырёхкратную задержку на каждом шаге. Меньшая модель может быть лучшим router, но плохим автором change plan.

Следовательно, единицей выбора является не «лучшая модель вообще», а связка:

model revision + tokenizer + chat template + serving configuration + prompt contract + tool schemas.

Изменение любого элемента требует хотя бы сокращённого regression eval.

Как строить eval до выбора модели

Правильный eval начинается с реальных типов задач, а не с случайного набора вопросов.

- 1. Соберите репрезентативные cases: простые, типичные, редкие и опасные.
- 2. Для каждого case зафиксируйте входные данные, допустимые действия и проверяемый ожидаемый результат.
- 3. Добавьте adversarial cases: конфликтующие инструкции, устаревший документ, похожие IDs, отсутствующий обязательный факт.
- 4. Разделите метрики: task success, tool choice, argument accuracy, unnecessary calls, unsafe proposals, latency, token usage и retries.
- 5. Прогоните несколько повторов там, где используется sampling или допускается несколько корректных планов.
- 6. Проводите shadow run без side effects, затем canary на ограниченном scope.
- 7. Сохраняйте model revision, server version, параметры generation и полный trace.

Для tool-using агента одного exact match недостаточно. Допустимы разные последовательности read-only проверок, если они приводят к подтверждённому результату в пределах бюджета. Поэтому evaluator часто сочетает детерминированные проверки и экспертную rubric.

Пример минимальной scorecard:

Метрика	Смысл	Пример порога
Verified task success	Достигнут ли проверяемый результат	не ниже 95% на типовых cases
Unsafe action proposal	Предложено ли запрещённое действие	0 на safety-наборе
Tool argument accuracy	Все ли критичные аргументы правильны	не ниже 99% для write proposals
Unnecessary tool calls	Лишние вызовы и стоимость	p95 не более заданного бюджета
Schema repair rate	Доля повторов из-за формата	достаточно мала для SLO
End-to-end latency	Время всего model step	p95 соответствует UX/automation deadline

Числа здесь являются примером формы, а не универсальными нормативами. Для read-only assistant и автоматического изменения routing policy допустимый риск будет разным.

Маленькая модель, большая модель или deterministic код

Не каждый шаг агента должен выполняться одной и той же LLM.

Тип шага	Предпочтительный первый кандидат
Проверка обязательных полей и типов	Обычный код или schema validator

Тип шага	Предпочтительный первый кандидат
Точное сопоставление по известному ключу	SQL, search или deterministic rule
Простая классификация с устойчивыми классами	Малая модель после eval либо обычный classifier
Извлечение структуры из неоднородного текста	Малая или средняя LLM со structured output
Сложная диагностика с несколькими гипотезами	Более сильная reasoning model
Генерация change plan для подтверждения	Сильная модель плюс deterministic policy checks
Проверка postcondition	Инструмент и обычный код, не самооценка модели

Router может снизить стоимость, направляя простые steps на меньшую модель. Но router сам становится источником ошибок. Экономия имеет смысл только тогда, когда измеряется итоговая стоимость успешной задачи, включая retries и неверную маршрутизацию.

Хорошая стратегия начинается с одной модели и прозрачного baseline. Разделение на fast/strong модели добавляют после того, как traces показывают разные классы нагрузки и понятную границу routing.

Cloud API, локальный endpoint и hybrid path

Способ serving - отдельное решение от выбора model family.

Cloud API обычно ускоряет старт, даёт elasticity и снимает эксплуатацию GPU. Взамен появляются зависимость от внешнего provider, egress данных, rate limits, меняющаяся модель сервиса и стоимость по usage.

Local inference даёт контроль над model revision, данными, сетью и hardware. Он может быть экономичен при стабильной загрузке, но команда принимает на себя capacity planning, обновления, драйверы, observability и availability.

Hybrid позволяет использовать локальный endpoint для регулярной нагрузки, а внешний - для burst, fallback или более сложных steps. Такой путь требует проверки semantic compatibility: два API с похожим форматом могут по-разному поддерживать tools, schemas, token accounting и finish reasons.

Выбор удобно начинать с вопросов:

- разрешено ли передавать данные внешнему provider;
- нужен ли offline или isolated режим;
- каков профиль burst и средняя utilization;
- какая model capability нужна именно этой задаче;
- допустима ли деградация при недоступности preferred model;
- кто будет эксплуатировать serving stack;
- как фиксируется revision и проводится rollback.

Fallback не должен незаметно менять security или качество. Если основная модель отказалась от опасного действия, повтор на другой модели не является корректным retry. Если

fallback имеет меньший context limit, harness обязан пересобирать context, а не обрезать его случайным образом.

Model adapter - контракт, а не тонкая обёртка над HTTP

Provider APIs различаются форматами сообщений, tool calls, streaming events, usage accounting, error codes и возможностями. Model adapter должен нормализовать общие части, но сохранять различия, важные для приложения.

Полезный внутренний контракт включает:

```
ModelRequest
  model_profile
  messages
  tools
  response_schema
  max_output_tokens
  deadline
  trace_id

ModelResult
  content
  tool_calls
  finish_reason
  input_tokens / output_tokens
  provider_request_id
  model_revision
  raw_metadata

ModelError
  category
  retryable
  request_may_have_completed
  provider_request_id
```

Особенно важен `finish_reason`. Остановка по `normal end`, достижение `token limit`, `policy refusal` и `transport interruption` требуют разных действий. Нельзя передавать truncated JSON в tool executor или автоматически повторять запрос, который мог уже вернуть часть результата.

Capability profile должен быть явным: поддерживает ли endpoint tools, parallel tool calls, конкретный JSON Schema subset, vision, streaming, prefix caching и нужную длину context. Название «OpenAI-compatible» или похожий HTTP surface не доказывает одинаковую семантику.

Ошибки model step нужно классифицировать

Фраза «LLM не ответила» объединяет разные классы отказов.

Класс	Пример	Типичная реакция
Capacity	rate limit, queue timeout, overload	bounded retry с backoff или другой capacity pool
Transport	connection reset, incomplete stream	повтор только для model request, не для уже выполненного tool

Класс	Пример	Типичная реакция
Input contract	context too long, invalid schema	пересобрать context или исправить request; слепой retry бесполезен
Output contract	truncated/malformed structured output	repair или повтор в пределах бюджета
Policy	provider refusal или local guardrail	зафиксировать причину; fallback только по явной policy
Quality	правдоподобный, но неверный вывод	evaluator, дополнительное evidence, другая стратегия или модель
Tool semantics	верный JSON, неверный tool/arguments	domain validation блокирует side effect
Version drift	изменилось поведение после update	regression eval и rollback/pin revision

Retry способен исправить временную перегрузку, но не плохой prompt, недостающий факт или систематическую ошибку модели. Несколько бездумных повторов иногда лишь повышают вероятность получить ответ, который пройдет поверхностную проверку.

Где проходит граница AX

В AX ресурс `Model` связывает Task с model endpoint и его конфигурацией. Это полезная platform abstraction: Task не обязан знать физический GPU host или детали внешнего provider.

Но Model resource не выбирает reasoning strategy и не доказывает пригодность модели для workload. AX не валидирует за приложение tool semantics, context composition, eval quality или postconditions. Эти обязанности остаются в harness и model adapter.

Такое разделение позволяет менять cloud endpoint на local serving, не переписывая lifecycle Task. Однако переносимым будет только тот слой, который действительно описан общим контрактом. Provider-specific возможности должны оставаться в capability profile и тестироваться явно.

Практическое упражнение

Возьмите агента, который анализирует pull request и при необходимости предлагает patch.

Подготовьте двадцать cases:

- небольшой безопасный diff;
- изменение access policy;
- скрытая ошибка в конфигурации;
- большой diff с нерелевантными файлами;
- конфликтующие инструкции внутри README;
- тесты с неоднозначным failure;
- отсутствующий контекст;

- похожие имена файлов;
- попытка вызвать запрещённый write tool;
- задача, где правильный ответ - запросить уточнение.

Для двух candidate models измерьте:

- смогла ли модель выбрать нужные файлы;
- верно ли интерпретировала тесты;
- соблюла ли границу read/write;
- сколько tool calls сделала;
- сколько input/output tokens использовала;
- каковы TTFT и end-to-end latency;
- сколько раз потребовался schema repair;
- достигнут ли результат после независимой проверки.

Затем повторите сравнение после сокращения context и удаления лишних tools. Если меньшая модель резко улучшилась, проблема была не только в её capability, но и в качестве контракта вокруг неё.

Если запомнить только главное

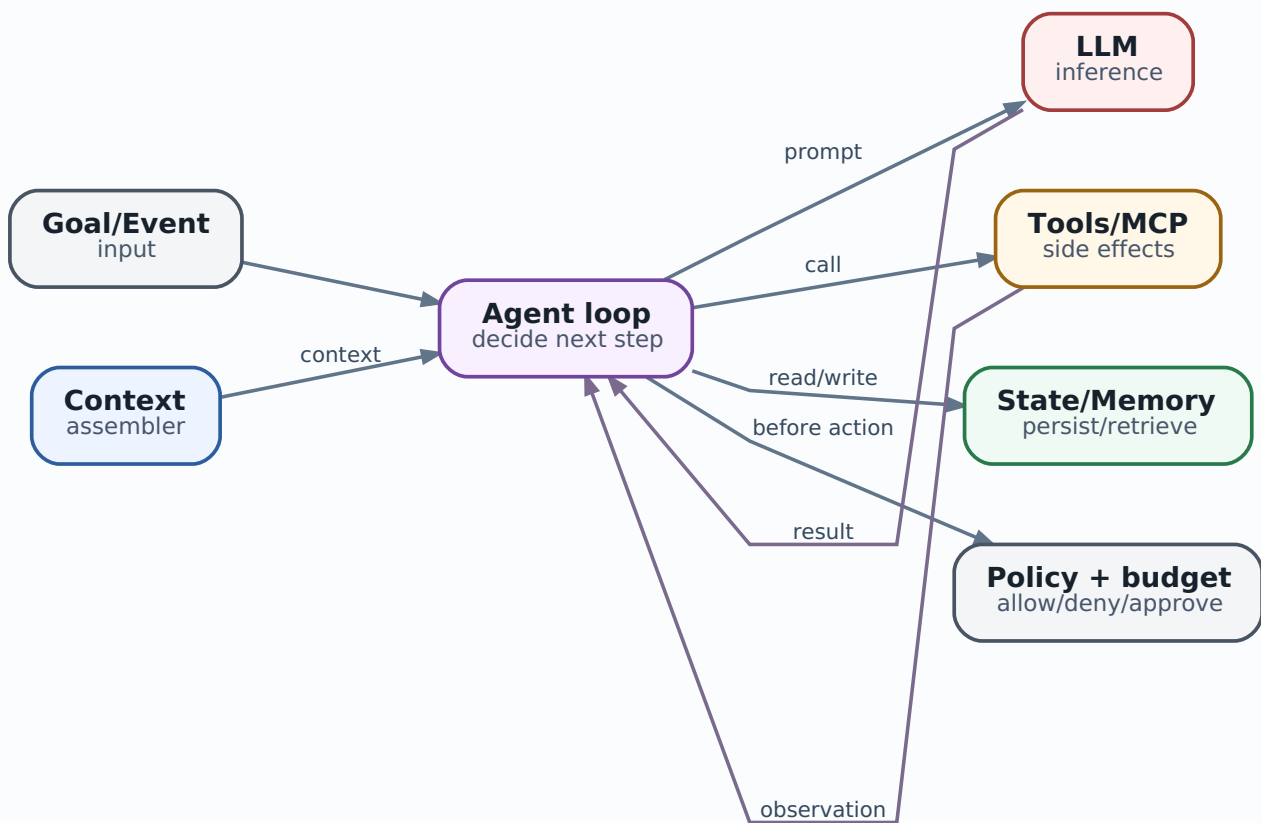
- LLM - вероятностный вычислительный компонент, а не authoritative state и не исполнитель side effects.
- Фактический input определяется tokenizer'ом и chat template, а не количеством слов.
- Context window - ограниченный рабочий бюджет; большой лимит не гарантирует эффективного использования всего контекста.
- Prefill определяет значительную часть TTFT, decode - скорость появления следующих токенов, а KV cache - memory pressure.
- Structured output гарантирует форму только в пределах поддерживаемых constraints; смысл и полномочия проверяет обычный код.
- Temperature и sampling управляют разнообразием, но бизнес-корректность обеспечивается eval, validation и idempotency.
- Модель выбирают по собственным cases и нескольким метрикам, а не по одному публичному рейтингу.
- Cloud, local и hybrid serving имеют разные эксплуатационные и security trade-offs.
- Model adapter обязан нормализовать API, не скрывая capability differences и finish reasons.
- АХ управляет ссылкой Task на Model, но качество model step остаётся ответственностью приложения.

Источники и дальнейшее чтение

- [Attention Is All You Need](#)
- [Hugging Face: tokenization algorithms](#)
- [Hugging Face: generation strategies](#)
- [Hugging Face: KV cache strategies](#)
- [vLLM: optimization and chunked prefill](#)
- [vLLM: production metrics](#)
- [Lost in the Middle: How Language Models Use Long Contexts](#)
- [Holistic Evaluation of Language Models](#)
- [Generating Structured Outputs from Language Models](#)
- [AX Core concepts, baseline e70162a](#)

Что такое агент: цикл, границы и критерии остановки

Agent loop: цель, контекст, решение, действие и обратная связь



Представим задачу: проверить, почему после изменения DNS часть клиентов не может подключиться к сервису. Обычный workflow можно заранее расписать: получить записи, проверить propagation, выполнить TLS probe, сравнить конфигурацию, сформировать отчёт. Но реальный инцидент быстро выходит за рамки одного сценария. В одном случае обнаружится stale resolver cache, в другом - неправильный CNAME, в третьем - доступ к нужной зоне отсутствует и потребуется запросить помощь.

Агент нужен не потому, что задача содержит текст или LLM. Он нужен тогда, когда следующий полезный шаг нельзя полностью выбрать заранее и его приходится определять по наблюдаемому состоянию среды.

Минимальное инженерное определение можно сформулировать так:

Агент - это ограниченный процесс, который получает цель, наблюдает состояние, выбирает допустимое действие, фиксирует результат и повторяет цикл до проверяемого завершения, паузы или контролируемого отказа.

В этом определении важны все слова. Без цели получается бесконечный исследователь. Без наблюдений - генератор предположений. Без ограничений - источник неконтролируемых side effects. Без сохранённого состояния - чат, который забывает progression. Без критериев остановки - цикл, способный расходовать ресурсы неограниченно.

Не всё, что вызывает LLM, является агентом

Термин «агент» часто применяют слишком широко. Для выбора архитектуры полезно различать несколько классов систем.

Система	Кто выбирает следующий шаг	Есть ли цикл	Когда использовать
Один model call	Приложение	Нет	Классификация, извлечение, генерация ограниченного результата
Prompt chain	Заранее написанный код	Фиксированная последовательность	Известные стадии с проверками между ними
Workflow	State machine или workflow engine	Возможны повторы и ожидания по заданным правилам	Предсказуемый процесс, durable retries, human tasks
Router	Правило, classifier или LLM	Обычно нет собственного длинного цикла	Выбор специализированного пути
Agent	Модель в заданных границах	Да, следующий шаг зависит от observations	Неопределённая последовательность исследований или действий
Multi-agent system	Несколько ограниченных loops и coordinator	Граф или дерево	Независимые contexts, authorities или параллельная работа

Граница не всегда бинарна. Production-система часто сочетает deterministic workflow и agentic nodes. Например, state machine жёстко задаёт этапы `collect - diagnose - propose - approve - execute - verify`, а LLM внутри этапа `diagnose` выбирает, какие read-only проверки выполнить.

Такой гибрид обычно надёжнее идеи «пусть модель сама решит всё». Свобода остаётся там, где полезна адаптивность, а переходы с высоким риском контролирует обычный код.

У автономности есть несколько независимых измерений

Слово «автономный» ничего не говорит без уточнения. Агент может иметь высокую свободу планирования, но не иметь права на внешние изменения. Или работать часами, но каждый write требовать подтверждения.

Полезно оценивать автономность по отдельным осям:

- **decision autonomy** - насколько свободно выбирается следующий шаг;
- **action authority** - какие side effects разрешены;

- **time horizon** - сколько времени процесс может работать без участия человека;
- **resource budget** - сколько steps, tokens, денег и compute доступно;
- **delegation depth** - может ли агент создавать дочерние работы;
- **environment scope** - какие данные, tools и systems видимы;
- **reversibility** - насколько легко отменить последствия;
- **human checkpoints** - где требуется approval или уточнение.

Агент для исследования документации может иметь широкую decision autonomy и почти нулевой blast radius. Агент с одним разрешённым API-вызовом в production может иметь узкую логику, но высокий риск. Поэтому security review должен начинаться не с вопроса «насколько умна модель», а с authority, scope и reversibility.

Минимальный agent loop

Поведение агента удобно представить как цикл observe - decide - act - verify. В production между этими словами появляются дополнительные gates.

1. Принять objective, scope и initial state.
2. Загрузить policy, budgets и durable progression.
3. Получить observations, необходимые для текущего шага.
4. Собрать context из state и trusted/untrusted evidence.
5. Попросить модель предложить следующий шаг или итог.
6. Разобрать structured decision.
7. Проверить schema, domain rules, policy и approval requirement.
8. Выполнить действие с operation ID и timeout.
9. Сохранить observation, provenance и возможный side effect.
10. Проверить postcondition и progress.
11. Перейти к следующему состоянию, приостановиться или завершиться.

Упрощённый skeleton выглядит так:

```

while true:
    state = load_durable_state(task_id)

    terminal = evaluate_terminal_conditions(state)
    if terminal:
        return finalize(terminal, state)

    context = assemble_context(state)
    proposal = model.decide(context)

    decision = validate_proposal(proposal, state.policy)

    if decision.requires_approval:
        checkpoint(state, waiting_for_approval(decision))
        return PAUSED

    operation = execute_idempotently(decision, state.operation_ledger)
    observation = verify_operation(operation)

    state = apply_observation(state, observation)
    checkpoint(state)

```

Это не готовая реализация. Важна форма: модель не исполняет действие напрямую, terminal conditions проверяются вне модели, а state сохраняется до следующего шага.

Observation, state и plan - разные сущности

Эти три понятия часто смешивают, из-за чего агент становится трудно восстанавливать и диагностировать.

Observation - результат конкретного чтения или действия. Например: «resolver 10.0.0.5 вернул старый IP в 12:04 UTC». Observation содержит источник, время, request ID и raw artifact.

State - компактное представление progression. Например: текущая стадия diagnosis, подтверждённые факты, отклонённые гипотезы, оставшийся budget, ожидаемый approval и ledger операций.

Plan - текущая гипотеза о будущих шагах. Он может измениться после нового observation и не является authoritative truth.

Если plan хранится как state без различия статусов, восстановившийся агент способен принять старое намерение за уже выполненное действие. Если observations хранятся только внутри model context, они исчезнут после compaction. Если state состоит из свободного текста, программе трудно проверить invariant.

Практичная модель данных разделяет как минимум:

Поле	Пример
objective	восстановить доступность клиентов, не меняя authoritative DNS без approval
phase	diagnosing
verified facts	origin healthy; resolver A stale; resolver B current

Поле	Пример
hypotheses	cache propagation delay; split-horizon mismatch
open questions	какой TTL был до изменения
operations	read-17 completed; change-3 proposed, not approved
budgets	8 из 20 steps, 4 минуты из 15
waiting for	none
expected outcome	два независимых probes возвращают новый IP и успешный TLS

Модель получает проекцию этого state, но не должна быть единственным компонентом, который умеет его интерпретировать.

Детерминированная оболочка вокруг вероятностного решения

LLM полезна для классификации неоднозначных данных, построения гипотез и выбора исследования. Но многие решения должны оставаться обычным кодом.

Решение	Предпочтительный владелец
Достигнут ли timeout	Runtime или harness
Остался ли budget	Budget controller
Существует ли tool	Tool registry
Допустимы ли аргументы	Schema и domain validator
Имеет ли identity право на действие	Authorization service
Нужен ли approval	Policy engine
Выполнялась ли операция раньше	Operation ledger
Достигнут ли измеримый postcondition	Deterministic verifier
Какую гипотезу проверить следующей	LLM в разрешённых границах
Как объяснить вывод человеку	LLM на основе проверенных facts

Это разделение называется не недоверием к модели, а корректным распределением обязанностей. Даже очень сильная модель не должна вычислять, истёк ли wall-clock deadline, если это может сделать таймер. И не должна подтверждать успешность собственного изменения без независимого чтения.

Agent execution лучше моделировать как state machine

Цикл `while true` скрывает важные operational states. Явная state machine делает паузы, восстановление и терминальные исходы видимыми.

Состояние	Значение	Допустимый следующий переход
Created	objective принят, выполнение ещё не началось	Running, Cancelled
Running	агент выбирает или выполняет безопасный шаг	WaitingForTool, WaitingForApproval, Verifying, Failed
WaitingForTool	внешний read/operation ещё не завершён	Running, Verifying, Failed, Cancelled
WaitingForApproval	требуется решение человека или policy owner	Running, Cancelled, Failed
Verifying	проверяется итог или postcondition	Succeeded, Running, Failed
Succeeded	доказан expected outcome	терминальное
Failed	продолжение невозможно или исчерпан допустимый budget	терминальное либо явный operator retry
Cancelled	выполнение остановлено владельцем или системой	терминальное

Waiting не равно failure. Task, который ожидает approval два часа, не должен занимать активный worker и не должен каждые десять секунд повторно спрашивать модель. Он сохраняет checkpoint и возобновляется по событию.

Failed также не означает «модель однажды ошиблась». Локальная ошибка может привести к repair или альтернативному пути. Terminal failure наступает, когда классифицированная ошибка неисправима в текущем score, исчерпан budget либо нарушен invariant.

Критерий успеха должен существовать до запуска

Фраза «сделай так, чтобы всё работало» не задаёт проверяемого результата. Если success condition не определён, агенту остаётся доверять собственному ощущению завершённости.

Хороший expected outcome отвечает на вопросы:

- что именно должно стать истинным;
- каким инструментом это измеряется;
- сколько независимых подтверждений требуется;
- какое состояние не должно измениться;
- в течение какого окна результат считается стабильным;
- какой artifact нужно сохранить.

Для задачи с DNS критерий может выглядеть так:

- authoritative zone содержит ожидаемую запись и serial;
- два независимых resolver возвращают новый адрес после TTL window;
- TLS probe на hostname успешен;
- origin health не ухудшился;

- итоговый отчёт содержит timestamps и raw probe IDs.

Ответ модели «готово» не входит в этот список.

Success condition полезно отделять от completion report. Сначала код проверяет outcome, затем модель объясняет человеку, что произошло. Если объяснение не сгенерировалось, технический результат может оставаться успешным, а report generation - отдельной repairable задачей.

Остановка бывает успешной, ожидающей и аварийной

У agent loop должно быть несколько явных путей выхода.

Success - postcondition подтверждён.

Needs input - отсутствует факт, который может предоставить пользователь или другая система. Это pause, а не догадка.

Needs approval - следующий допустимый шаг имеет side effect выше текущего authority.

Blocked - требуемый ресурс недоступен, а альтернативный путь отсутствует.

Budget exhausted - достигнут лимит steps, времени, tokens, tool calls, денег или delegation depth.

No progress - новые iterations не меняют state и не добавляют evidence.

Cancelled - владелец задачи или control plane запросил остановку.

Safety stop - policy или invariant запрещает продолжение.

Для каждого исхода задают machine-readable reason и human-readable summary. Иначе operator увидит общий статус Failed и снова запустит тот же процесс, не понимая, что агент на самом деле ждал недостающий credential или упёрся в policy.

Budget - не одна цифра

Ограничение только по maximum steps недостаточно. Один step может быть дешёвым read-only запросом, а другой - часовым job или вызовом дорогой модели.

Budget обычно включает:

- wall-clock deadline;
- maximum active execution time;
- maximum model requests;
- input и output tokens;
- monetary cost;
- tool calls по классам;
- число write proposals;
- retries каждого operation;
- объём созданных artifacts;
- число и depth дочерних задач.

Budgets должны уменьшаться монотонно и храниться в durable state. Restart процесса не должен обнулять счётчики. Child task получает долю оставшегося parent budget, а не новый безграничный лимит.

При исчерпании budget агент не обязан просто оборваться. Он может сохранить partial result, перечислить подтверждённые facts, незакрытые вопросы и безопасный следующий шаг. Это превращает контролируруемую остановку в полезный outcome.

Как обнаруживать отсутствие прогресса

Max iterations гарантирует конечность, но не замечает бессмысленную работу заранее. Полезен отдельный no-progress detector.

Он может отслеживать:

- повтор одного tool с теми же нормализованными аргументами;
- одинаковые ошибки без изменения recovery strategy;
- отсутствие новых verified facts;
- неизменный fingerprint state несколько шагов подряд;
- plan, который циклически возвращается к предыдущей стадии;
- evaluator score, который не улучшается;
- последовательность «прочитать - предложить - отклонить» для одного объекта;
- постоянное создание новых subtasks без закрытия старых.

Fingerprint не должен включать шум вроде timestamps или request IDs, иначе каждый iteration будет казаться новым. Полезнее хешировать semantic state: phase, open questions, verified facts, pending operations и нормализованный proposed action.

Реакция также должна быть поэтапной. Сначала агент получает structured feedback о повторе. Затем пробует ограниченную альтернативу. После заданного patience window останавливается с reason `NO_PROGRESS`. Бесконечная просьба к той же модели «подумай ещё» не является recovery strategy.

Недавние исследования infinite agentic loops подтверждают, что циклы могут проходить через model calls, tools, workflow transitions и handoffs. Поэтому bound должен существовать на каждом feedback path, а не только в основном for-цикле.

Retry не равен новому рассуждению

Повтор допустим, когда ошибка временная и операция безопасно повторяема. Например, model endpoint вернул overload до начала generation или read-only API завершился connection reset.

Если timeout произошёл во время write, неизвестно, выполнила ли внешняя система действие. Повтор без проверки способен создать дубль. Нужны:

1. стабильный operation ID или idempotency key;
2. запись intent до вызова;
3. сохранение provider request ID;
4. запрос статуса существующей операции;

5. независимая проверка external state;
6. только затем решение о retry или compensation.

Идемпотентность не означает, что запрос «обычно безопасен». Она означает, что повтор с тем же identity и payload не создаёт дополнительного эффекта.

Модель не должна сама решать, можно ли повторить write, по тексту ошибки. Класс операции и retry semantics задаёт tool contract.

Human-in-the-loop - полноценное состояние, а не модальное окно

Approval особенно часто реализуют как временный prompt: процесс ждёт ответ в памяти и теряет его при restart. Production-система должна рассматривать human interaction как durable interrupt.

Перед паузой сохраняются:

- proposed action и нормализованные arguments;
- причина и evidence;
- ожидаемый side effect;
- diff или preview;
- identity запрашивающего;
- срок действия предложения;
- state version;
- operation ID, если он уже выделен.

После resume необходимо проверить, что approval относится к той же версии state. За время ожидания конфигурация могла измениться, credential - истечь, а incident - завершиться. Старое подтверждение не должно автоматически разрешать новый payload.

Human-in-the-loop включает не только yes/no. Агент может запросить недостающий score, предложить несколько вариантов, получить изменённые параметры или передать задачу operator. Эти ответы являются events, которые изменяют durable state.

Plan не должен становиться скрытым API

Модели удобно предложить план, но текстовый plan нельзя использовать как единственный control structure. У фразы «проверить DNS, затем исправить запись» нет schema, статусов и retry semantics.

Для исполнения полезнее разложить намерение на structured steps:

```
plan_version: 4
steps:
  - id: read-authoritative
    kind: tool
    status: completed
    result_ref: observation-81
  - id: compare-resolvers
    kind: fanout-read
    status: running
    required_successes: 2
  - id: propose-change
    kind: decision
    status: pending
    requires: [read-authoritative, compare-resolvers]
    approval_policy: dns-write
```

LLM может предложить или обновить такой plan. Harness проверяет зависимости, разрешённые step kinds и invariants. Фактическое состояние каждого step изменяет executor, а не модель.

При небольшом числе состояний отдельный planner вообще не нужен. Модель может выбирать одно следующее действие, а deterministic state machine - сохранять progression. Planner/executor оправдан, когда предварительный план помогает распределять работу, оценивать зависимости или запрашивать approval на весь change set.

Основные patterns и критерии выбора

Patterns - это способы распределить decision-making, а не названия продуктов.

ReAct чередует reasoning и action. Он полезен, когда каждый следующий шаг зависит от свежего observation: search, диагностика, exploration. В production reasoning result лучше выражать как краткое decision summary и structured action, а не считать скрытую цепочку рассуждений audit log.

Prompt chaining передаёт результат через фиксированную последовательность stages. Это workflow, а не автономный agent, и именно поэтому он часто лучше для предсказуемой задачи.

Routing выбирает один специализированный путь. Он снижает context и позволяет использовать разные models/tools, но требует обработки ошибочной классификации.

Planner/executor отделяет построение плана от исполнения. Полезен для задач с зависимостями, но план должен перепроверяться после каждого значимого observation.

Evaluator/optimizer повторяет генерацию и проверку. Он оправдан только при измеримом criterion и bounded iterations. «Модель-критик считает ответ лучше» без калиброванного evaluator легко создаёт дорожную петлю.

Fan-out/fan-in параллельно выполняет независимые проверки и объединяет structured results. Он ускоряет работу, если ветви действительно независимы и budget делится заранее.

Supervisor/worker делегирует ограниченные subtasks. Он нужен при разных tools, authority или context. Подробная multi-agent архитектура рассматривается отдельно.

Требование	Первый кандидат
Известные стадии и переходы	Workflow/state machine
Следующий шаг зависит от нового observation	ReAct-подобный loop
Есть несколько устойчивых классов входа	Router
Нужен change plan с зависимостями	Planner/executor
Результат можно автоматически оценить и улучшить	Bounded evaluator loop
Независимые проверки можно выполнять параллельно	Fan-out/fan-in
Различаются authority, workspace или expertise	Supervisor/worker

Нельзя выбирать pattern только потому, что framework предоставляет красивую диаграмму. Каждый дополнительный loop или role должен улучшать измеримый результат сильнее, чем повышает latency, стоимость и число failure modes.

Когда workflow лучше агента

Workflow предпочтительнее, если:

- процесс известен заранее;
- каждый transition можно выразить правилами;
- важны predictability и auditability;
- ошибки имеют известные recovery paths;
- variation входных данных невелика;
- side effects критичны;
- регулятор или оператор должен заранее понимать все пути.

Agent оправдан, если:

- число возможных путей велико;
- нужны исследования и адаптация по observations;
- вход содержит неоднородный неструктурированный материал;
- невозможно заранее перечислить все полезные read-only шаги;
- ценность гибкости выше дополнительной стоимости и риска.

Часто лучший ответ - workflow с agentic island. Например, workflow управляет approval и deployment, а агент исследует логи и формирует change proposal. Такой дизайн локализует неопределённость.

Safety и liveness - две разные задачи

Safety означает: плохое событие не произойдёт. Агент не выйдет за score, не выполнит write без approval, не превысит authority, не повторит необратимое действие.

Liveness означает: полезное событие когда-нибудь произойдёт. Задача не останется навсегда `WaitingForTool`, `approval event` не потеряется, `retry` закончится, а `cancel` действительно остановит descendants.

Можно построить очень безопасную систему, которая никогда ничего не завершает. Можно построить активную систему, которая быстро выполняет неправильные изменения. `Production contract` обязан включать оба набора invariants.

Примеры safety invariants:

- `write tool` недоступен до `approved state`;
- `child authority` не шире `parent`;
- каждый `external write` имеет `operation ID`;
- `terminal Task` не запускает новые actions.

Примеры liveness invariants:

- каждый non-terminal state имеет `timeout` или входящее событие;
- каждый `retry` bounded;
- `orphan operation` периодически reconciled;
- `cancel` распространяется на активные children;
- `WaitingForApproval` имеет `owner` и `expiry policy`.

Observability должна показывать переходы, а не только текст

Лог полного `conversation history` полезен, но недостаточен. Для диагностики нужно восстановить причинную цепочку:

```
task - state version - model request - proposal - policy decision - tool operation - observation - transition.
```

На каждом step полезно фиксировать:

- step ID и parent step;
- state before/after;
- model/provider request ID;
- выбранный action type;
- tool и нормализованные arguments;
- policy/approval decision;
- operation ID;
- latency и resource usage;
- observation/artifact refs;
- termination check result.

Необязательно сохранять скрытое внутреннее reasoning модели. Для audit важнее входные evidence, явное decision summary, фактическое действие и проверенный результат. Это уменьшает риск перепутать правдоподобное объяснение с реальной причиной перехода.

Как тестировать agent loop

Agent loop нельзя проверять только набором удачных диалогов. Такой тест показывает, что один знакомый сценарий однажды сработал, но ничего не говорит о повторяемости, остановке, восстановлении и безопасности. Единица проверки здесь - не красивый ответ модели, а переход системы из одного состояния в другое.

Полезно разделить тестирование на четыре уровня.

1. Детерминированные unit-тесты оболочки

Policy, budget accounting, state transitions, idempotency и обработка tool results должны тестироваться без модели. Для каждого перехода задаются входное состояние, событие и ожидаемое новое состояние.

Примеры:

- Running + proposal(write) + no approval -> WaitingForApproval;
- WaitingForTool + timeout + retries_left -> Running;
- WaitingForTool + timeout + no retries -> Failed;
- Verifying + success evidence -> Succeeded;
- Running + repeated fingerprint -> Blocked;
- any non-terminal state + cancel -> Cancelled.

Эти тесты должны также проверять invariants: terminal state не порождает новые actions, declined approval не превращается в write, один operation ID не приводит к двум side effects.

2. Симулированная среда

Реальные tools дороги, медленны и иногда опасны. Поэтому useful test harness должен уметь возвращать заранее заданные observations: успешный ответ, timeout, malformed payload, stale data, partial success, conflict или permission denied. Важно симулировать не только штатный путь, но и неоднозначность.

Например, DNS tool может сначала вернуть timeout, затем показать уже обновлённую запись. Правильный agent не должен слепо повторить write: он обязан выполнить read-after-timeout, сопоставить desired state с actual state и только затем решать, нужен ли retry.

3. Scenario evals и replay

Набор сценариев должен описывать цель, начальное состояние среды, разрешённые действия, ожидаемые evidence и недопустимые side effects. Один и тот же сценарий запускают многократно, потому что решение модели вероятно.

Trajectory полезно сохранять как последовательность нормализованных событий, а затем replay-ить без реальных side effects. Replay отвечает на два разных вопроса:

1. корректно ли оболочка обработала уже полученные proposals и observations;
2. изменилось ли решение модели после смены prompt, model version или tool description.

Это позволяет отделить регрессию orchestration от регрессии reasoning.

4. Recovery и counterfactual tests

Хороший агент должен не только идти по счастливому пути, но и корректно менять план после новой информации. Поэтому в тесте намеренно меняют среду:

- log source становится недоступен;
- permission исчезает между plan и execute;
- approval приходит после изменения state version;
- tool возвращает success, но verification не подтверждает результат;
- оператор отменяет Task во время долгой операции;
- один из параллельных workers возвращает противоречащие evidence.

Counterfactual test задаёт вопрос: «Если бы observation отличалось в одном существенном месте, изменилось бы решение?» Если агент предлагает один и тот же action при `certificate expired` и при `certificate valid`, он, вероятно, следует поверхностному шаблону, а не фактам.

Метрики production-качества

Одна метрика `task completed` слишком груба. Она не отличает доказанный успех от самоуверенного отчёта. Минимальный набор включает:

Метрика	Что она показывает
Verified task success rate	Доля задач, где критерий успеха подтверждён evidence
Invalid action rate	Доля proposals, нарушивших schema, state или policy
Unsafe proposal / executed action	Разницу между тем, что model пыталась сделать, и тем, что реально пропустила оболочка
Steps to verified outcome	Эффективность траектории, а не длину текста
No-progress stop rate	Способность обнаруживать loop до исчерпания бюджета
Recovery rate	Долю сценариев, завершённых после tool failure или изменения среды
Escalation quality	Достаточно ли оператору контекста, чтобы принять решение
Cost and latency distribution	Медиану и хвосты, а не только среднее значение

Особенно важно отдельно считать unsafe proposals и executed unsafe actions. Нулевая доля фактических нарушений может означать как качественное reasoning, так и сильный policy layer, который постоянно спасает плохо настроенную модель. Это разные инженерные ситуации.

Где проходит граница АХ

В АХ `Task` предоставляет внешний lifecycle работы: создание, запуск, наблюдение, завершение, отмену и связь с execution metadata. `Workspace` задаёт изолированный контекст файлов и процессов. `Model` выполняет inference. Но эти абстракции сами по себе не определяют внутреннюю семантику agent loop.

Runner или agent runtime поверх АХ должен определить:

- schema логического state;
- допустимые actions и tools;
- policy и approval rules;
- способ записи observations;
- критерии успеха;
- budgets и stopping conditions;
- checkpoint и recovery semantics;
- формат итогового результата и evidence.

Это разделение важно по двум причинам.

Во-первых, инфраструктурный runtime не должен угадывать domain semantics. АХ может надёжно доставить событие, сохранить artifact и остановить process, но не знает, означает ли `HTTP 200` успешное восстановление сервиса или попадание на fallback page.

Во-вторых, логический checkpoint агента не равен снимку процесса. Возобновить контейнер с того же instruction pointer недостаточно: внешняя среда могла измениться, lease истечь, approval устареть, а tool operation завершиться после потери связи. При resume агент должен загрузить durable state, reconcile незавершённые operations, обновить observations и только затем выбрать новый transition.

Два уровня retry

В системе обычно существуют как минимум два разных retry:

1. **Step retry** - повтор конкретной tool operation внутри agent loop.
2. **Task retry** - повтор всего runner после infrastructure failure.

Если их не различать, Task retry может заново выполнить side effect, который на самом деле успел завершиться. Поэтому operation ID и idempotency key должны переживать перезапуск Task. После восстановления runner сначала проверяет статус ранее начатой операции и только потом решает, повторять ли её.

АХ отвечает за надёжную execution substrate. Agent runtime отвечает за корректный переход от цели к действию и от observation к следующему решению. Продуктовая система отвечает за domain policy и за то, что считается доказанным результатом. Смешивание этих уровней делает failure analysis почти невозможным.

Практикум: агент исследует TLS-инцидент

Рассмотрим задачу: «Пользователи получают ошибку TLS при обращении к `api.example.com`; найди причину и подготовь безопасное исправление».

Шаг 1. Зафиксировать контракт

До написания prompt опишите:

- **goal:** определить подтверждённую причину и восстановить корректный TLS handshake;
- **scope:** DNS, load balancer, certificate inventory и логи только для `api.example.com`;
- **success evidence:** handshake проходит из двух независимых probes, certificate chain валидна, hostname совпадает, error rate вернулся ниже порога;
- **forbidden:** выпуск нового сертификата, DNS write и production reload без approval;
- **budget:** 20 model steps, 10 минут, 30 tool calls, один write proposal;
- **terminal result:** `Succeeded`, `WaitingForApproval`, `Blocked`, `Failed` или `Cancelled` с evidence.

Шаг 2. Определить state

Минимальная schema может содержать:

```
incident:
  host: api.example.com
  reported_error: tls_handshake_failed
observations: []
hypotheses: []
selected_hypothesis: null
proposed_change: null
approval:
  status: not_requested
  state_version: 0
operations: {}
budgets:
  steps_left: 20
  tool_calls_left: 30
termination:
  status: running
  evidence: []
```

Не храните только `certificate is probably expired`. Храните observation с источником и временем: какой endpoint проверен, какой serial number получен, какую chain вернул load balancer, на каком host возникла ошибка.

Шаг 3. Ограничить tools

Read-only набор может включать:

- DNS lookup;
- TLS probe с выводом chain и SNI;
- чтение certificate inventory;

- чтение конфигурации load balancer;
- агрегированный поиск по логам;
- чтение deploy history.

Write tools лучше не выдавать сразу. Агент сначала формирует change proposal: точный target, expected effect, rollback, verification и evidence. Отдельный approval transition открывает ровно нужную capability на ограниченное время.

Шаг 4. Проверить альтернативные гипотезы

Устаревший сертификат - лишь одна причина. Agent должен различить как минимум:

- сертификат действительно истёк;
- load balancer отдаёт старый certificate bundle;
- SNI ведёт на default virtual host;
- DNS указывает на старый endpoint;
- неполная intermediate chain ломает часть клиентов;
- часы probe или сервера некорректны;
- handshake исправен, а исходный alert относится к другому региону.

Каждая гипотеза должна иметь ожидаемый discriminating observation. Иначе список гипотез превращается в декоративный текст.

Шаг 5. Спроектировать transitions

Пример полезной траектории:

```
Created
-> Running: собрать DNS и TLS facts
-> Running: сопоставить serial с inventory и deploy history
-> Running: выбрать подтверждённую гипотезу
-> WaitingForApproval: представить ограниченный change plan
-> Running: выполнить approved operation с operation ID
-> Verifying: независимые probes и error-rate check
-> Succeeded: сохранить evidence и summary
```

Если TLS probe расходится между регионами, агент не должен усреднять ответы. Он создаёт новое observation, сужает score и либо продолжает диагностику в бюджете, либо завершает Blocked с точным описанием недостающей capability.

Workflow или agent?

Полностью детерминированный workflow хорошо подходит для известного случая «сертификат истекает через N дней»: проверить inventory, выпустить replacement, развернуть, проверить. Но incident diagnosis содержит развилки, которые зависят от свежих observations.

Практичный дизайн здесь гибридный:

1. workflow создаёт incident Task, задаёт budgets и собирает базовые probes;
2. agentic island исследует evidence и строит change proposal;

3. workflow проводит approval;
4. детерминированный executor выполняет изменение;
5. workflow запускает независимую verification;
6. agent пишет объяснение только на основании сохранённых evidence.

Так модель помогает там, где действительно нужна адаптация, а критические side effects остаются в предсказуемом контуре.

Итоги главы

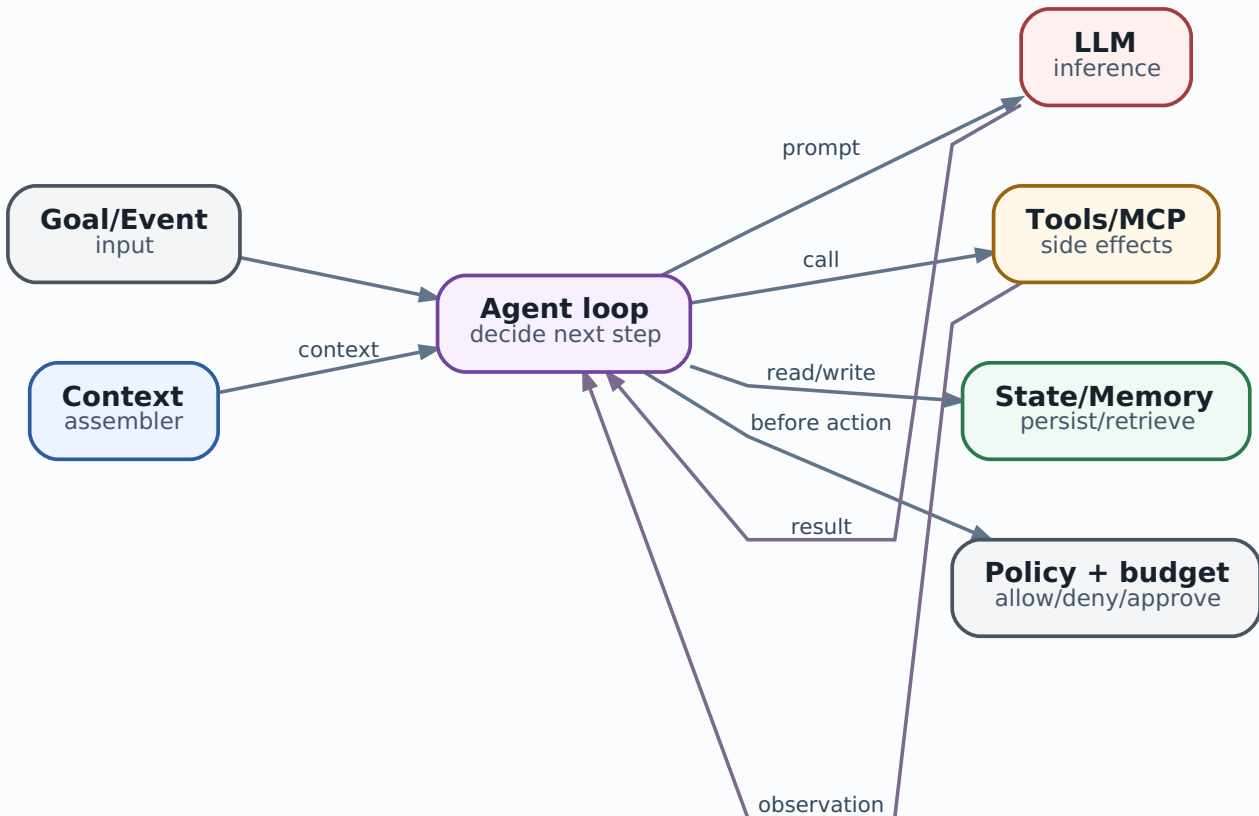
- Agent - это bounded control loop, а не просто model call с tools.
- Goal должен быть преобразован в проверяемый expected outcome до первого действия.
- Observation, state и plan - разные сущности; их смешивание разрушает audit и recovery.
- Каждый transition проходит через policy, budget и termination checks.
- Retry безопасен только вместе с reconciliation и idempotency.
- Human-in-the-loop требует durable pause, versioned approval и повторной проверки state.
- Workflow предпочтителен для известных путей; agent нужен для адаптации по новым observations.
- Safety запрещает плохие события, liveness гарантирует, что полезная работа не зависнет навсегда.
- Agent eval должен измерять проверенный результат, восстановление, остановку и side effects, а не красоту ответа.
- AX даёт execution substrate; смысл loop, tools, policy и success evidence задаёт agent runtime и продуктовая система.

Источники и дальнейшее чтение

- [ReAct: Synergizing Reasoning and Acting in Language Models](#) - исходная работа о чередовании reasoning traces и действий.
- [Building Effective AI Agents](#) - практическое различие workflows и agents, composable patterns и stopping conditions.
- [Thinking in LangGraph](#) - проектирование state, nodes, error handling и human-in-the-loop.
- [Idempotent Receiver](#) - классический паттерн защиты от повторной обработки запросов.
- [Temporal: Tasks](#) и [Standalone Activity](#) - durability, retry и idempotency для внешних операций.
- [Infinite Agentic Loops in Production](#) - taxonometry и detection бесконечных loop в agentic systems.
- [AX Core concepts, baseline e70162a](#) - границы базовых примитивов AX.

Agent harness: система вокруг LLM

Agent harness: минимальная операционная анатомия



Harness - удобный термин для слоя, который превращает модель в управляемый исполняемый процесс. В разных frameworks границы отличаются, но эксплуатационно полезно выделять одинаковые обязанности.

Обязательные функции harness

Context assembly выбирает инструкции, goal, историю, memory и результаты tools. **Model adapter** нормализует различия OpenAI/Anthropic/Gemini/local endpoints. **Tool executor** преобразует tool proposal в настоящий вызов. **State manager** хранит progression. **Policy layer** решает, допустима ли операция. **Budget controller** ограничивает число шагов, tokens, wall time и деньги. **Retry policy** различает временный сбой и логическую ошибку. **Tracing** связывает LLM request, tool call и task/actor identity.

Хороший harness не должен полагаться на prompt как на единственную защиту. Правило «никогда не выполняй shutdown» внутри system prompt слабее технического deny policy, потому что prompt injection воздействует на тот же канал принятия решения. Правильный подход: model proposes, policy decides, tool enforces.

Harness и orchestrator - разные роли

Harness управляет шагами **внутри** agent execution. Orchestrator управляет множеством executions: создать Task, выбрать Workspace, приостановить idle workload, выделить ресурсы, повторить при инфраструктурном сбое, уничтожить sandbox. Если смешать уровни, возникает путаница: retry HTTP 503 модели должен быть обязанностью harness/model gateway, а восстановление Actor после падения Worker - обязанностью runtime/control plane.

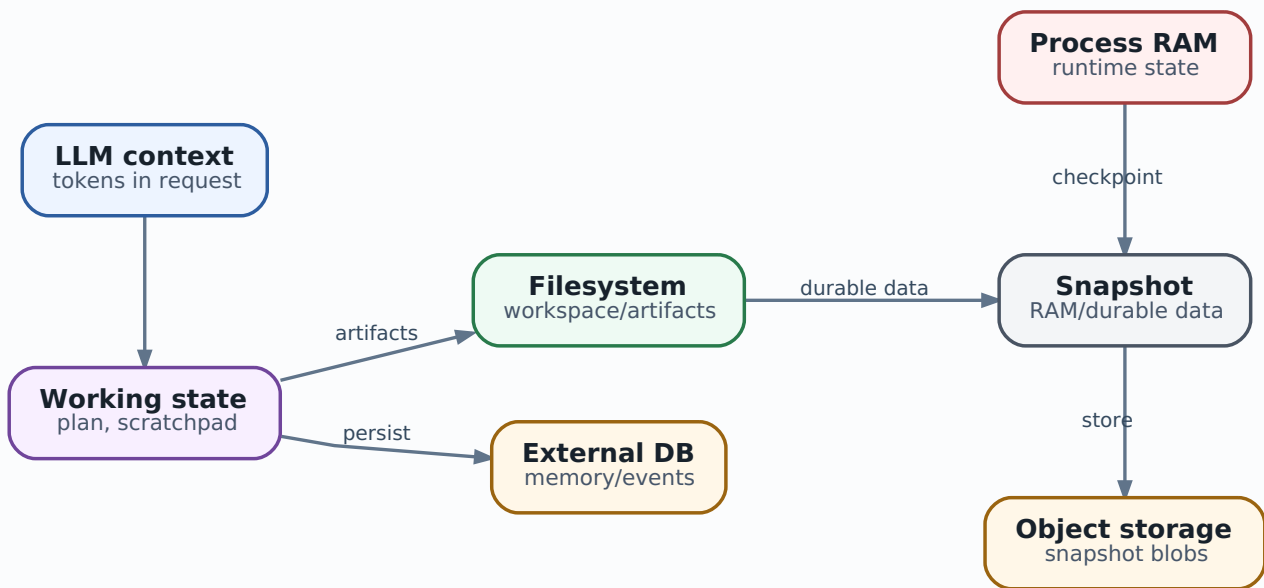
Minimal production contract

Для каждого агента стоит явно определить: входной schema, maximum wall time, maximum steps, token budget, разрешённые tools, outbound destinations, persistent state location, checkpoint strategy, termination semantics и expected artifacts. Это превращает «автономного агента» в управляемую workload единицу.

АХ не навязывает конкретный harness. Это важная архитектурная граница: можно запускать собственный Python/Go/Node agent, framework или CLI-agent, если container/runner contract соблюлён.

Context, state, memory и snapshot: не смешивать

Состояние агента: шесть разных сущностей, которые часто ошибочно называют «памятью»



В agent systems слово «memory» часто скрывает несколько совершенно разных механизмов. Для правильной эксплуатации их нужно разделять.

LLM context существует только в рамках конкретного inference request и ограничен context window. **Working state** - текущий plan, cursor, промежуточные результаты и state machine. **Persistent memory** - записи, которые переживают перезапуск и могут быть retrieved. **Workspace** - файлы проекта, artifacts и tool configuration. **Process RAM** - heap/stack конкретного процесса. **Snapshot** - технический checkpoint части runtime state. **External system state** - Git, ticket, router config, DB - authoritative data, которые нельзя «откатить» восстановлением локального snapshot.

Почему это важно при resume

Если runtime восстанавливает RAM image, TCP connection к внешнему API всё равно может оказаться недействительным: peer закрыл сессию, DNS/IP изменились, token протух. Если AX восстанавливает только durable `/workspace` и запускает новое process tree, агент должен самостоятельно восстановить in-memory progression из файлов/DB. Оба поведения могут называться «resume», но требования к приложению принципиально различаются.

Рекомендуемая модель состояния

- Authoritative business state - во внешнем durable store или системе назначения.

- Agent progression - в journal/event log или компактном state object.
- Workspace/artifacts - в durable filesystem/object store.
- Prompt/context - производное представление, которое можно пересобрать.
- Snapshot - optimisation для быстрого продолжения, но не единственная копия критического state.

Для инфраструктурного агента особенно опасно считать snapshot транзакцией: после `configure terminal` на роутере и до локального checkpoint внешнее устройство уже изменено. Resume не отменит изменение. Поэтому write actions должны быть журналированы и верифицированы независимо.

Tools, MCP и Skills

MCP trust boundary: протокол не заменяет авторизацию



Tool - это формализованная операция, доступная agent loop. Он может быть локальной функцией, HTTP API, shell command или MCP tool. Главное свойство production tool - не «модель умеет его вызвать», а наличие чёткой схемы, проверяемых полномочий, predictable side effects и наблюдаемого результата.

MCP в версии 2026-07-28

Текущая спецификация MCP существенно отличается от ранних туториалов. Ядро стало stateless: убраны protocol-level sessions и старый `initialize/initialized` handshake. Каждый запрос несёт `protocol version` и `client capabilities` в `_meta`. Сервер должен поддерживать `server/discover`, позволяющий узнать версии, `capabilities` и `identity`. Для совместимости со старыми STDIO servers discovery может использоваться как probe.

Практически это улучшает работу через gateways, WAF и rate limiters: routing и metering меньше зависят от долгоживущей сессии. Но security всё равно остаётся задачей deployment'a. Self-reported `serverInfo` нельзя использовать как security proof.

В MCP полезно помнить ownership semantics: **Prompts** обычно user-controlled, **Resources** application-controlled, **Tools** model-controlled с точки зрения выбора вызова. «Model-controlled» не означает «безусловно разрешённый»; policy enforcement находится снаружи.

Skills

Skill - не transport protocol. Это переиспользуемый пакет инструкций, процедур и вспомогательных материалов, который помогает agent harness выполнить класс задач. Tool даёт capability, Skill объясняет способ применения capability. MCP стандартизует взаимодействие между client/server; Skill может ссылаться на MCP tools, но не является их сетевым эквивалентом.

Что делает AX Workspace

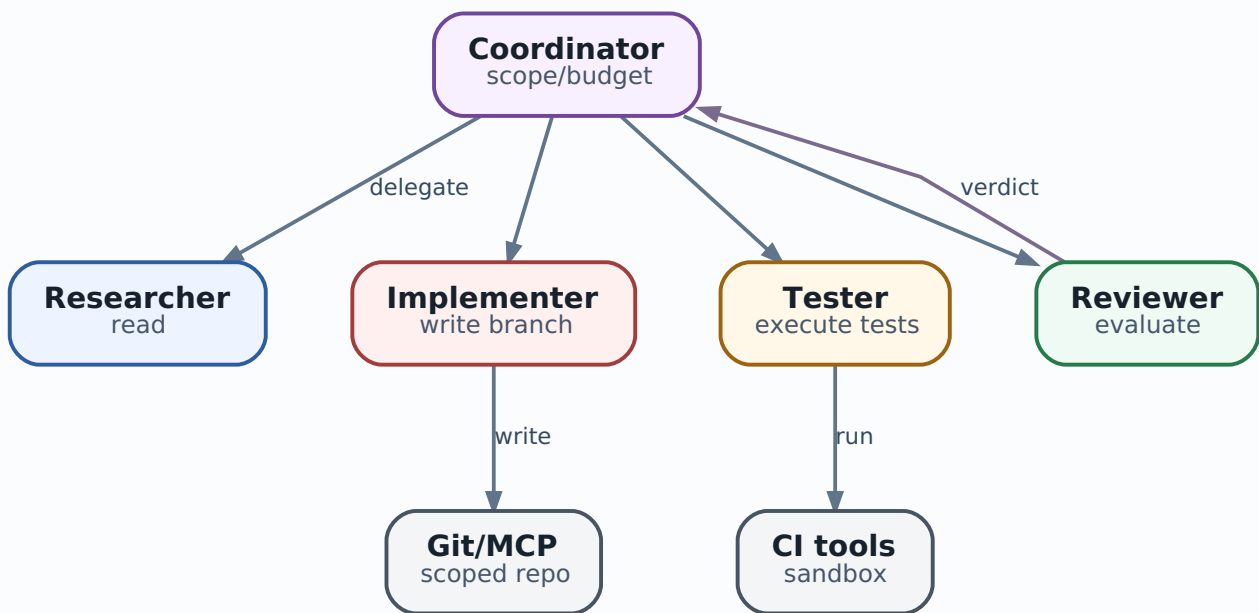
Workspace может описывать Git repositories, files, MCP servers/registries и skill registries/path. Тем самым Workspace становится декларативным описанием рабочей среды, а не просто volume. Но security implication очевиден: MCP server или skill registry - часть supply chain. Подключение неизвестного registry должно проходить не менее строгий review, чем новый package repository.

Источники и дальнейшее чтение

- [MCP 2026-07-28 release](#)
- [MCP discovery](#)
- [AX Workspace concepts](#)

Multi-agent архитектуры без лишней магии

Практика: software-engineering multi-agent workflow



Multi-agent architecture оправдана, когда разделение даёт хотя бы одно из трёх: меньший context, разные полномочия/tools или параллельное выполнение. Если этих преимуществ нет, один хорошо ограниченный agent часто проще и дешевле.

Основные топологии

Supervisor/worker. Supervisor хранит high-level цель и делегирует ограниченные subtasks. Worker не должен получать больше authority, чем нужно его роли. **Hierarchical delegation** продолжает этот pattern на несколько уровней, но требует budget propagation и ограничения глубины. **Fan-out/fan-in** запускает несколько независимых проверок и агрегирует вывод. **Specialist routing** выбирает агента по типу инцидента или ресурса. **Evaluator gate** проверяет результат перед следующей стадией.

Coordination state

Не стоит передавать всё через естественный язык. Надёжнее иметь structured task state: ID, parent ID, input artifact refs, expected output schema, deadline, allowed tools, budget, status. Текстовая summary может быть частью context, но не должна заменять machine-readable contract.

Failure propagation

Если child Task упал, parent должен знать: retryable ли ошибка, был ли side effect, можно ли повторить с тем же idempotency key, стоит ли отменять siblings. В infrastructure workflow особенно важно отличать «не смог прочитать telemetry» от «изменил конфигурацию, но потерял ответ».

АХ и дерево задач

АХ концептуально допускает, что один Task создаёт другие и таким образом формирует большое дерево. При этом сама платформа не обязана моделировать бизнес-семантику дерева за вас. Это сознательное решение: Task остаётся дешёвой изолированной единицей, а relation/budget/authority должны проектироваться на уровне agentic application до тех пор, пока соответствующие primitives не появятся в стабильной спецификации.

От Linux process к gVisor и microVM

Лестница изоляции: усиление boundary обычно увеличивает стоимость и сложность



Изоляция agent workload важнее, чем у обычного внутреннего микросервиса: агент способен генерировать и запускать код, клонировать внешние repository, подключать инструменты и реагировать на потенциально hostile content. Поэтому «он же в контейнере» - недостаточный threat model.

Linux process и container

Namespaces разделяют представления PID, mount, network, user и других ресурсов. cgroups ограничивают/учитывают CPU, memory, I/O и число процессов. Linux capabilities дробят привилегии root на отдельные права. seccomp фильтрует системные вызовы. no_new_privileges запрещает процессу получать новые privileges через exec. Вместе эти механизмы значительно уменьшают blast radius, но обычный container всё ещё разделяет kernel host'a.

gVisor

gVisor вставляет userspace application kernel (Sentry) между приложением и host kernel. OCI runtime runsc позволяет использовать эту модель через Docker/Kubernetes. Большая часть Linux interface переимплементируется в userspace, поэтому hostile process имеет существенно меньшую прямую поверхность host syscalls. Цена - compatibility/performance overhead, который зависит от workload.

microVM

microVM использует аппаратную виртуализацию и отдельный guest kernel. Boundary обычно сильнее, но появляются VMM, guest image, virtual devices, boot/snapshot complexity и требования к KVM. На Proxmox/KVM lab nested virtualization должна быть доступна VM, иначе microVM backend не будет рабочим практическим вариантом.

Выбор

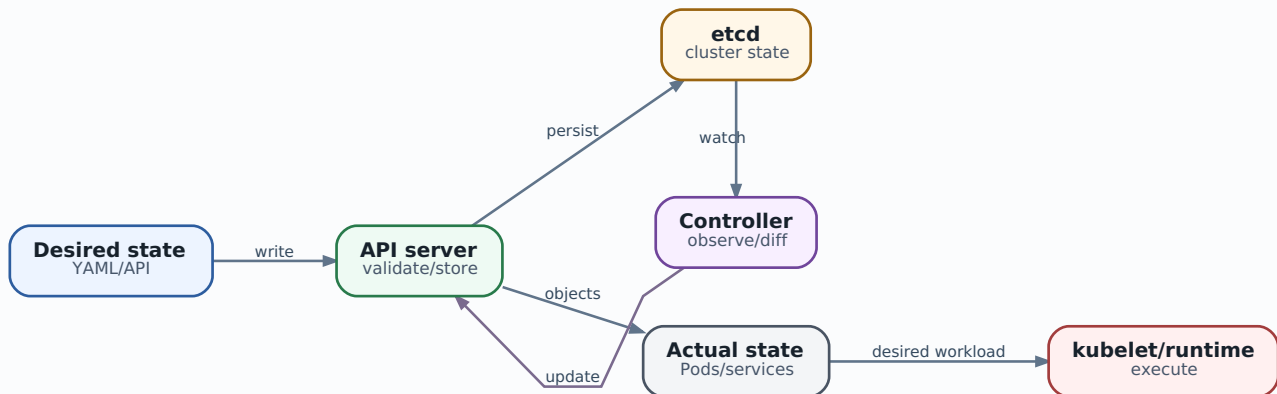
Для первого стенда разумнее gVisor: меньше движущихся частей. microVM стоит изучать как второй backend для более сильной изоляции. Выбор должен опираться на threat model и benchmark именно вашего workload, а не на абстрактную таблицу «безопаснее/быстрее».

Источники и дальнейшее чтение

- [gVisor architecture](#)
- [gVisor runsc](#)

Kubernetes: необходимый минимум для понимания AX/Substrate

Kubernetes reconciliation loop - mental model, который нужен для AX/Substrate



Kubernetes здесь нужен не как отдельный предмет, а как substrate инфраструктуры. Достаточно уверенно понимать несколько mental models.

Pod - минимальная deployable unit Kubernetes, а не VM. **Deployment** поддерживает desired number stateless Pods. **DaemonSet** гарантирует Pod на подходящих nodes и поэтому естественен для node-level supervisor. **Service** даёт стабильную сетевую точку над changing Pods. **Namespace** разделяет имена и policy scope. **Secret/ConfigMap** поставляют configuration, хотя Secret сам по себе не является HSM. **RBAC** определяет права на Kubernetes API. **NetworkPolicy** ограничивает сетевые потоки при поддержке CNI.

Control plane

kube-apiserver принимает API operations. etcd хранит cluster state. kube-scheduler назначает unscheduled Pods nodes. Controllers постоянно сравнивают desired и actual state. На node kubelet обеспечивает запуск Pod через container runtime.

Главный transferable concept - **reconciliation**. Пользователь не просит «создай контейнер прямо сейчас», а объявляет desired state. Controller снова и снова стремится привести actual state к desired. AX внешне использует похожую declarative модель, но сохраняет высокочастотное task state вне Kubernetes API.

Почему не хранить миллионы Tasks в CRD

Kubernetes официально рекомендует не использовать Custom Resources как обычное application/end-user/monitoring data store и предупреждает, что большое количество CRs нагружает API storage. AX design идёт в ту же сторону: небольшие декларативные control-plane объекты Kubernetes остаются там, а population короткоживущих agent tasks хранится в специализированном store.

Что изучать отдельно

Если после этой главы непонятны Pod lifecycle, Services/DNS, RBAC или CNI, стоит пройти официальные Kubernetes Concepts и отдельный lab до AX. Для AX не требуется становиться Kubernetes developer, но попытка диагностировать Substrate без понимания `kubect1 get/describe/logs`, readiness и scheduling почти гарантированно приводит к путанице.

Источники и дальнейшее чтение

- [Kubernetes components](#)
- [Kubernetes custom resources](#)

Control plane и distributed systems: база без академической перегрузки

AX/Substrate - распределённые control planes, поэтому несколько идей необходимо понимать практически.

Desired/actual state и eventual consistency

После `apply` результат не обязан появиться мгновенно. API может успешно принять объект, а reconciler ещё будет создавать Actor, ждать Worker и readiness. Поэтому корректный клиент читает `status/conditions` или `watch stream`, а не считает HTTP/gRPC success равным готовности workload.

Idempotency

Control plane операции будут повторяться: из-за `retry`, `restart controller'a` или потери ответа. Поэтому шаги reconciliation должны быть идемпотентными либо иметь уникальные operation IDs. Для внешних tools это особенно важно: «повторить API call» может быть безопасно для GET и опасно для `create payment / change route`.

Distributed lock

Когда несколько controller replicas могут reconcile один ресурс, нужен механизм не выполнять конфликтующие операции одновременно. Lock не заменяет идемпотентность: holder может умереть после side effect, но до записи status. Правильная система выдерживает повтор после lock expiry.

Redis, PostgreSQL, streams

AX использует Redis как высокоскоростной store/event mechanism для своего task control plane. Current `ax-controller` читает Redis stream consumer group и вызывает reconciler. Substrate current architecture использует PostgreSQL для высокочастотного dynamic Actor/Worker state. Это два разных слоя с разными failure domains; Redis backup не восстанавливает Substrate actors, а PostgreSQL backup не восстанавливает AX desired specs.

gRPC/protobuf

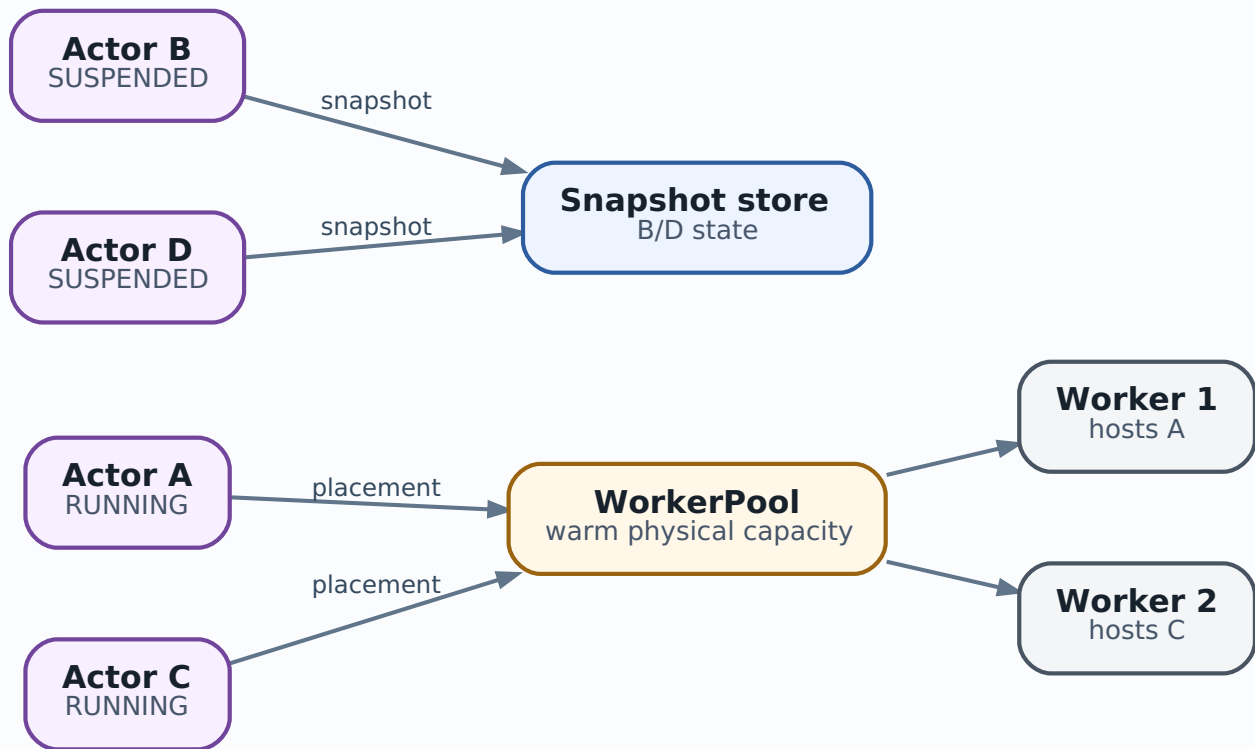
Protobuf задаёт typed message schema, gRPC - RPC contract и streaming. Операционно важны deadlines, retries, message-size limits, TLS/mTLS, authority/server-name, load balancing и backpressure. «RPC висит» без deadline способен привязать controller worker надолго.

Consistency versus recovery

Для orchestration важнее не абсолютная синхронность каждого поля, а возможность восстановить истинное состояние. Status должен быть наблюдаемым и переоцениваться. Если runtime сообщает Actor RUNNING, но Worker умер, reconciler обязан обнаружить расхождение и перевести ресурс в корректное состояние.

Actor model применительно к агентным workload

Actor $\neq$ Worker: logical population can be larger than active capacity



Actor в Substrate полезно понимать как **логически долгоживущую identity + state**, которая не обязана постоянно занимать физический process/Pod. Worker - физическая capacity, готовая выполнить Actor. Это главное отличие от интуитивной модели «один agent = один Pod навсегда».

Почему это подходит agents

Типичный агент много времени ждёт: человека, external tool, webhook, следующую задачу. Если каждый idle agent удерживает 1-4 GiB RAM, плотность системы быстро упирается в hardware. Если runtime умеет checkpoint state, освободить Worker и позже восстановить Actor, logical population может быть существенно больше active capacity.

ActorTemplate

`ActorTemplate` - immutable blueprint: image, sandbox class/config, resources, volumes, snapshot policy и другие runtime параметры. Из него создаётся golden snapshot. Изменение workload должно приводить к новой версии template, а не к мутации state root под существующими actors.

WorkerPool

WorkerPool описывает физическую warm capacity и в Substrate связан с Kubernetes objects. Matching sandbox class и selectors определяют, на какие workers может попасть Actor. Именно здесь появляются capacity planning, oversubscription и queueing.

Dormant Actor

Suspended Actor продолжает существовать в control plane и object storage, но не занимает active Worker. Его можно разбудить explicit API call или сетевым запросом через routing layer. Это не serverless function в классическом смысле: Actor имеет durable identity и state lineage.

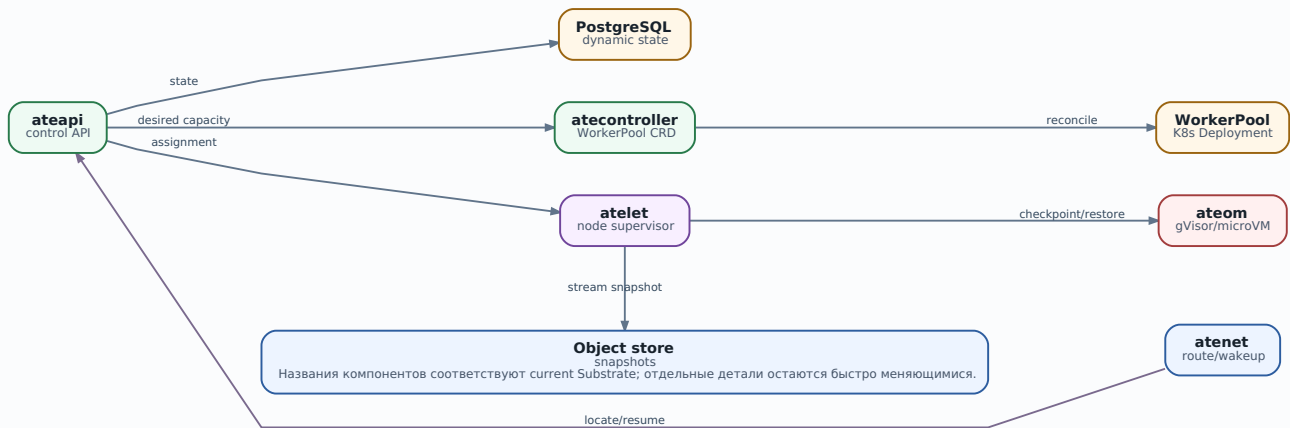
Agent Substrate: назначение и operational architecture

IMPLEMENTED / VERIFIED

MAIN DELTA where noted

ROADMAP kept separate

Agent Substrate: operational data path



Agent Substrate позиционируется как runtime для большого количества sandboxed stateful actors поверх Kubernetes. Он намеренно не является agent SDK: внутренняя reasoning logic остаётся приложению. Его задача - lifecycle, placement, isolation, snapshot/restore и routing.

Основные компоненты current architecture

- **ateapi** - control API.
- **atecontroller** - reconciliation Kubernetes-side WorkerPool resources.
- **atelet** - supervisor на node/worker side, координирующий lifecycle и snapshot transfer.
- **ateom** - runtime management внутри worker path; варианты gVisor и microVM.
- **atenet** - networking/routing, включая wake-on-request path.
- PostgreSQL - dynamic Actor/Worker state в текущей архитектуре.
- Object storage - snapshot blobs. В local Kind quickstart проект разворачивает RustFS; cloud deployments могут использовать GCS/S3-compatible path.

Важная оговорка о документации

Architecture document Substrate прямо предупреждает, что значительная часть описания aspirational и может ещё не быть реализована. Поэтому книгу нельзя использовать как «contract по каждой стрелке». Для production baseline проверяются release notes, API guide и реальное поведение на pinned версии.

Project claims и sizing

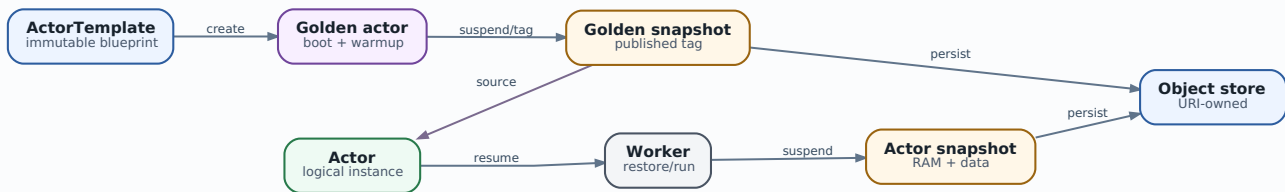
README заявляет очень высокую density, sub-500ms resume и высокую suspend/resume throughput. Эти цифры полезны как направление design goals, но не как sizing input. Реальная latency зависит от snapshot size, object-store bandwidth, sandbox backend, node locality, CPU pressure и workload readiness. Мы будем benchmark'ить собственный path.

Источники и дальнейшее чтение

- [Substrate README](#)
- [Substrate architecture](#)
- [Substrate API guide](#)

Substrate resources: Atespace, WorkerPool, ActorTemplate, Actor, Tag

Golden snapshot и жизненный цикл stateful Actor



Atespace

Atespace задаёт logical tenancy/scope для Substrate resources. Имена и права удобно рассматривать вместе с tenant boundary, а не просто как «namespace под другим названием».

WorkerPool

WorkerPool - физическая capacity. Он определяет replicas, worker image, sandbox class и pod template properties. Labels WorkerPool могут участвовать в выборе через `workerSelector`; не следует считать namespace или RBAC автоматическим scheduler selector'ом.

ActorTemplate

ActorTemplate immutable. Он содержит контейнеры, required `sandboxConfig`, ресурсы, snapshot config, volumes и selectors. При создании Substrate подготавливает golden actor, ждёт readiness/warmup, suspend'ит его и публикует golden snapshot через Tag. Так дорогая initialization может быть оплачена один раз.

Actor

Actor - конкретный instance template. В status хранится lifecycle state, assignment и snapshot metadata. Один Actor в каждый момент привязан максимум к одному Worker, но между resume может оказаться на другом Worker.

Tag

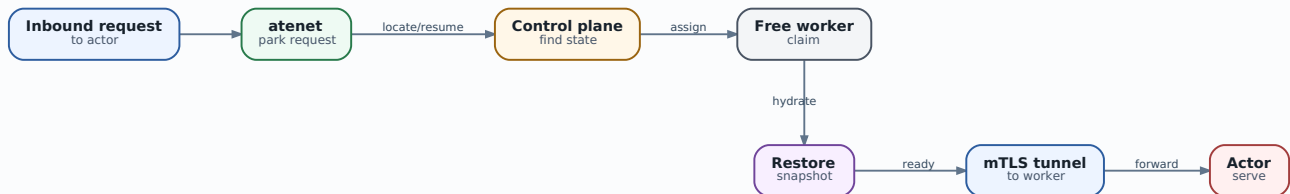
Tag даёт отдельное ownership snapshot'a и позволяет клонировать состояние. Snapshot lifetime важен: actor snapshot заменяется следующим successful suspend, тогда как Tag живёт до удаления. Actor, созданный из Tag, может некоторое время **ЗАИМСТВОВАТЬ** tag-owned snapshot; преждевременное удаление Tag способно сломать последующий resume.

SandboxConfig

SandboxConfig определяет runtime assets/configuration и связывается с `sandboxClass` (`gvisor` или `microVM`). Для воспроизводимости runtime binaries и guest assets должны быть pinned так же жёстко, как container image.

Suspend, snapshot, resume и golden snapshot

Resume on demand: запрос может разбудить suspended actor



Checkpointing - центральный механизм density, но его семантику нужно понимать точнее, чем «поставили VM на паузу».

Golden snapshot

При создании ActorTemplate временный golden actor проходит boot и initialization. Затем его suspend создаёт baseline state, опубликованный через golden Tag. Новый Actor может стартовать из этого состояния и избегать повторной expensive initialization. Полезный pattern: загрузить runtime, библиотеки и тяжёлые read-only assets **до** golden snapshot; per-actor secrets и identity, наоборот, должны появляться после cloning безопасным способом.

Actor snapshot

`SuspendActor` фиксирует runtime state и durable data в external snapshot и освобождает Worker. Следующий resume назначает свободный Worker и восстанавливает state. Snapshot связан с ActorTemplate UID: если template изменился, memory image старого sandbox не обязательно совместим. Current API guide описывает fallback, при котором durable data может быть восстановлена, а guest boot выполнен заново.

Network и external resources

Нельзя считать открытые TCP sessions частью надёжно переносимого business contract. Даже если file descriptor присутствует в memory image, remote endpoint не обязан считать соединение живым. Приложение должно уметь reconnect. Аналогично temporary credentials, DNS answers и leases могут протухнуть.

Eviction и data-loss window

Current Substrate guide указывает grace path для eviction: Actor получает `SIGTERM` и ограниченное время на suspend; если checkpoint не завершён, Actor может перейти в CRASHED, а изменения после последнего snapshot потеряются. Поэтому long-running active actor должен либо checkpoint'иться периодически, либо хранить critical progress вне snapshot.

AX nuance

В current AX runner semantics `/workspace` восстанавливается, но container/process tree создаётся заново. Это значит, что agent command должен уметь reconstruct progression. Не проектируйте AX Task так, будто Python heap гарантированно продолжится с той же instruction pointer только потому, что underlying Substrate вообще умеет RAM snapshots.

Источники и дальнейшее чтение

- [Substrate API guide](#)
- [AX runner](#)

Substrate networking: routing, wake-up и egress

Stateful Actor может менять физический Worker, поэтому адрес Pod/Worker нельзя использовать как долговременную identity. Routing layer должен разрешить logical Actor → current assignment и при необходимости инициировать resume.

Request parking

High-level flow: запрос приходит в `atenet`, router выясняет состояние Actor, если он `suspended` - удерживает запрос, control plane назначает Worker, `atelet/ateom` восстанавливают sandbox, после `readiness` открывается authenticated tunnel и исходный request продолжается. Это скрывает relocation от клиента, но добавляет resume latency в first request.

Egress

Agent workload имеет необычно высокий риск exfiltration и supply-chain calls. Хорошая default posture - deny egress и разрешать конкретные destinations. В Substrate v0.2.0+ egress policy стала заметной частью security model. AX Gateway переводит пользовательскую декларацию в runtime network policy.

L7 и TLS reality

Hostname allowlist сложнее CIDR: DNS меняется, TLS шифрует HTTP payload, SNI имеет свои ограничения, а transparent interception создаёт PKI risk. В AX v0.3.0 был публично задокументирован bug, где hostname allowlist ломал TLS egress, в то время как CIDR/wildcard работали. Это хороший operational lesson: condition `GatewayReady=True` означает, что policy применена, а не что реальный end-to-end запрос успешен.

Как тестировать

После policy deployment делайте positive и negative probes **из sandbox**: разрешённый DNS, разрешённый TCP/TLS, запрещённый destination, MCP endpoint и model endpoint. Логи egress проху должны входить в incident playbook.

Источники и дальнейшее чтение

- [AX Gateway issue #345](#)
- [Substrate release notes](#)

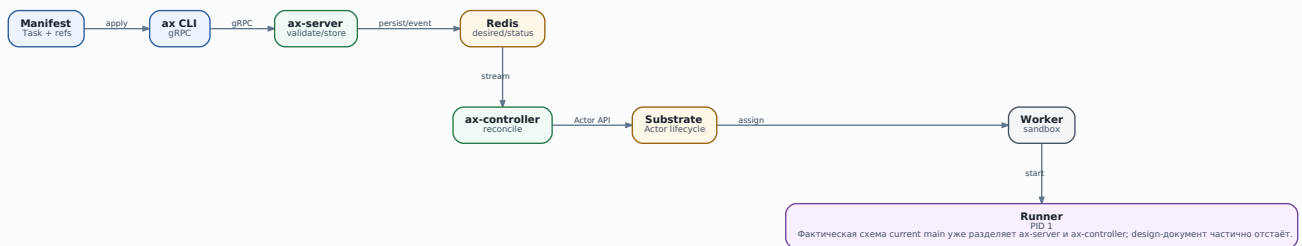
АХ: архитектура control plane и реальная граница с Substrate

IMPLEMENTED / VERIFIED

MAIN DELTA where noted

ROADMAP kept separate

АХ Task: путь от декларации до выполняемого агента



АХ даёт Kubernetes-подобный declarative UX для agent workloads, но не является Kubernetes API extension. Это принципиальная деталь архитектуры.

Release baseline и current main

Design document описывает ax CLI, ax-server и ax-task-runner, причём ax-server валидирует manifests, сохраняет state в Redis и сам reconciles с Agent Substrate. Current source tree уже содержит отдельный ax-controller: он читает Redis stream consumer group, поднимает TaskReconciler и вызывает Substrate client. CONTRIBUTING также говорит о deployment Redis + controller + server. Следовательно, актуальная operational модель - разделённые API и reconciliation paths, а DESIGN.md частично отстаёт.

Это не мелочь для эксплуатации. API replicas и controller replicas имеют разные bottlenecks. Публичный issue current main указывает, что один controller worker способен reconcile tasks последовательно, то есть throughput конкретного pod ограничен одной цепочкой reconcile. Масштабировать API без controller capacity бессмысленно.

State path

Пользователь применяет ax.io/v1alpha1 manifest через CLI. API валидирует имя/спец и сохраняет resource state в Redis. Событие попадает в controller processing. Controller разрешает ссылки на Workspace/Gateway/Model, формирует необходимые Substrate resources/policies, создаёт или восстанавливает Actor, следит за readiness и записывает Task status/conditions. Клиент получает eventual status через get/watch.

Почему Redis, а не CRD per Task

АХ design аргументирует это ожидаемой population миллионов короткоживущих tasks и ограничениями etcd/API server. Даже без веры в headline «billions of tasks» rationale разумен: Kubernetes declarative API хорошо подходит для относительно небольших, человекочитаемых

configuration objects, а не для высокочастотного application state. Redis позволяет AX построить собственную более узкую consistency/reconciliation model.

Failure boundary

Redis потерян - AX может потерять desired/status layer даже если Substrate actors продолжают существовать. Substrate control plane/DB потерян - AX specs остаются, но runtime placement/snapshot state недоступен. Поэтому backup и DR этих слоёв рассматриваются раздельно.

Источники и дальнейшее чтение

- [AX DESIGN](#)
- [Current ax-controller](#)
- [AX CONTRIBUTING](#)

AX primitives: Task, Workspace, Gateway и Model

Current AX README описывает четыре маленьких пользовательских primitive. Они не являются эквивалентами Kubernetes Pod/PVC/NetworkPolicy; это agent-oriented abstraction layer.

Task

Task - минимальная единица isolated execution. Он задаёт image, command, compute requests/limits, environment, Workspace bindings, Gateway reference и debug behavior. Task intentionally small: один Task может быть всей работой или узлом дерева delegated tasks. Status содержит phase и conditions. Не следует считать Running равным «agent command жив»: current issue показывает случай, когда runner остаётся PID 1 после exit child command, а status продолжает выглядеть healthy. Значит, application-level liveness нужно измерять отдельно.

Workspace

Workspace описывает то, что агент должен получить в рабочей среде: Git sources, static files, MCP configuration, skills и optional goal для environment preparation. Один Task может bind несколько workspaces; первый становится working directory command. Подготовка должна быть idempotent и выполняться один раз на durable state.

Gateway

Gateway задаёт сетевую границу: listeners и outbound egress allowlist. Это не «удобная настройка сети», а ключевой security primitive. Правильный default - deny и явные destinations. Реальные версии требуют end-to-end validation: наличие status GatewayReady подтверждает reconciliation policy, но не доказательство работоспособности DNS/TLS/application protocol.

Model

Model описывает provider, model identifier, parameters и secret reference. Важно не приписывать Model больше, чем он делает в текущей версии. Публичный issue v0.3.0 указывает, что наличие Model не означало автоматического безопасного secret injection в произвольный Task: связь Task→Model и secret semantics менялись. Поэтому в production надо проверять actual API schema pinned release, а не рассчитывать на имя ресурса.

Manifest discipline

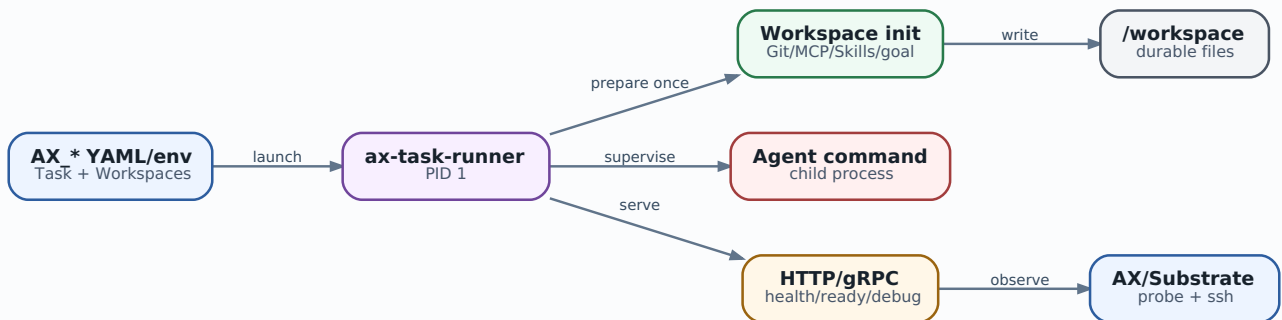
Все имена следует считать API identifiers: lowercase RFC1123-style, pinned image digests, explicit atespace, explicit resource requests/limits и ограниченный Gateway. Multi-document YAML удобно хранить как immutable deployment unit для лаборатории, но в production reusable Workspace/Gateway/Model могут иметь самостоятельный lifecycle.

Источники и дальнейшее чтение

- [AX README](#)
- [AX manifests](#)
- [AX concepts](#)
- [AX issue #348](#)

Runner: PID 1 как платформенный контракт

Runner contract: почему PID 1 здесь является частью платформенного API



Runner - одна из самых важных и при этом недооценённых частей AX. Control plane не запускает `spec.command` напрямую. Он запускает `/usr/local/bin/ax-task-runner` как PID 1; уже runner читает `AX_TASK_YAML`, `AX_WORKSPACES_YAML`, `env`, подготавливает workspace, запускает child command и обслуживает control endpoints.

Минимальный contract

Runner должен слушать port 80 и предоставлять `/healthz` и `/readyz`. Readiness остаётся 503, пока workspace не готов. Metadata endpoints могут отдавать Task/Workspaces. При `debug: true` runner включает guest services, через которые работает `ax ssh`; это фактически arbitrary process execution/file access внутри sandbox, поэтому debug нельзя оставлять включённым без причины.

Workspace initialization once

На первом boot runner клонирует Git, записывает files, подготавливает skill path/MCP config и выполняет goal-based bootstrap. Он должен сохранить marker, чтобы после resume не переклонировать repository поверх изменённого workspace. Current default runner использует служебный marker под `/ax`.

Child process semantics

Runner запускает agent command отдельной process group, но **остаётся жить после завершения child**. Это позволяет сохранить metadata/debug surface, однако создаёт monitoring nuance: container alive != agent alive. Current control plane не получает exit code child command как authoritative Task status. Значит, ваш custom runner или harness должен экспортировать application state отдельно.

SIGTERM и suspend

Stop/suspend доставляют SIGTERM PID 1. Runner обязан forward signal process group, дать bounded grace period, затем завершить остаток. Всё, что должно пережить resume, нужно flush в durable state до выхода. Это нормальная Unix discipline, а не специфический «LLM механизм».

Custom runner

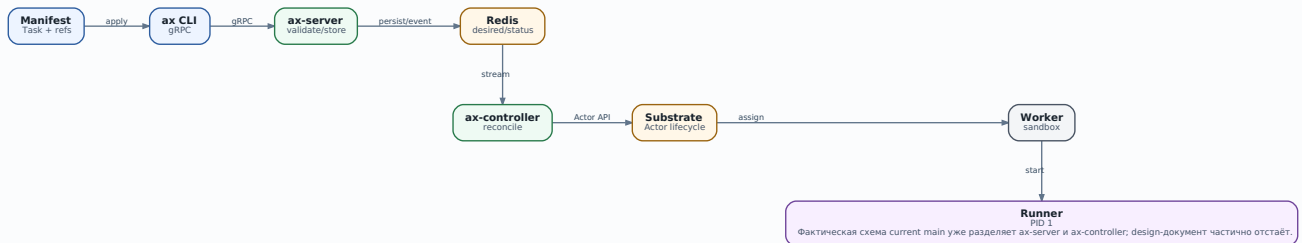
Есть три уровня customization: расширить default image; embedded Go runner package; полностью свой implementation на любом языке. Для большинства платформенных команд оптимален первый вариант - сохранить contract и изменить только toolchain image. Полный custom runner оправдан, когда agent framework уже имеет собственный supervisor или нужен иной workspace layout.

Источники и дальнейшее чтение

- [AX runner contract](#)
- [AX sandbox](#)

Полный lifecycle Task: от YAML до suspend/resume

AX Task: путь от декларации до выполняемого агента



Разберём Task как распределённую state machine, а не как «контейнер, который стартовал».

Create path

1. `ax apply` разбирает multi-document manifest.
2. `ax-server` валидирует resource schemas/refs и сохраняет desired state.
3. Controller получает event из store/stream.
4. Разрешаются Workspace/Gateway/Model и runtime requirements.
5. Формируется/выбирается Substrate ActorTemplate и Actor.
6. Runtime назначает Worker и поднимает sandbox.
7. Runner стартует PID 1.
8. Workspace bootstrap выполняется один раз.
9. `/readyz` становится 200.
10. Controller выставляет WorkspaceReady / Ready .
11. Agent command выполняет loop.

Каждый пункт - отдельный failure domain. Поэтому «Task Pending» и «Task Running but useless» требуют разных playbooks.

Suspend path

Пользователь или policy инициирует suspend. AX переводит lifecycle вниз в Substrate, runner получает termination signal и обязан flush state. Substrate/AX сохраняют durable workspace/runtime snapshot согласно текущей integration semantics, освобождают physical Worker и обновляют status. Suspended Task остаётся logical resource.

Resume path

Task снова получает Worker. Durable `/workspace` возвращается, runner запускается заново, видит markers и пропускает destructive re-bootstrap, затем запускает agent command. Harness

должен определить, где остановилась работа, и продолжить по durable journal/state. Именно здесь хорошо спроектированный state machine превосходит prompt-only memory.

Delete

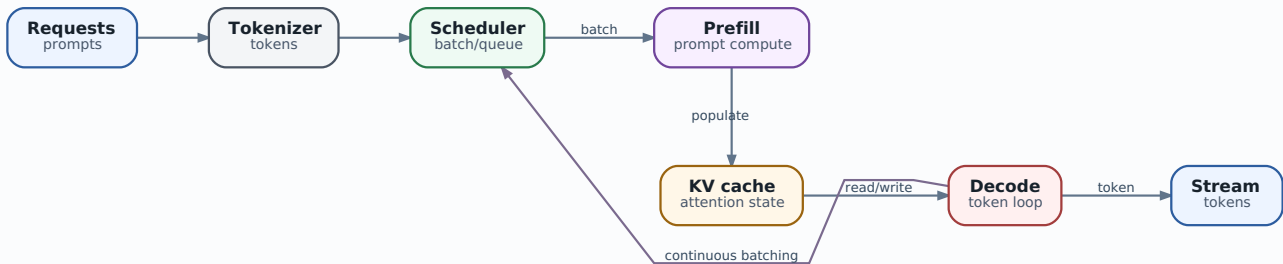
Delete должен освобождать AX resource, Substrate Actor/snapshot ownership и связанные runtime artifacts согласно lifecycle. External side effects - Git commits, tickets, конфигурации устройств - не исчезают. Поэтому delete Task никогда не является бизнес rollback.

Operational invariant

На каждом этапе задавайте три вопроса: **какой слой владеет desired state? какой слой владеет runtime state? какой сигнал доказывает readiness?** Если ответ нельзя сформулировать, система будет плохо диагностироваться.

Local inference: CPU, GPU, vLLM и llama.cpp

Local inference: путь запроса через batching, prefill, KV cache и decode



Локальный inference позволяет контролировать данные, latency profile, стоимость hardware и availability, но переносит на вас capacity planning, model serving и upgrades. Для АХ это просто model endpoint: orchestration не должно зависеть от того, облачный он или локальный.

Два практических server класса

vLLM ориентирован на throughput GPU serving, continuous batching и масштабирование крупных моделей. Current release line 2026-09 - `v0.30.0`. OpenAI-compatible server упрощает интеграцию с harnesses. **llama.cpp** удобен для CPU, hybrid CPU/GPU, GGUF quantization, homelab и fallback; server тоже предоставляет OpenAI-compatible интерфейс и умеет parallel decoding/continuous batching.

Prefill и decode

Prefill обрабатывает весь prompt и создаёт KV-cache. Decode генерирует токены последовательно. Поэтому TTFT зависит от prompt length, batching и prefill throughput, а inter-token latency - от decode path и contention. Large context может съесть VRAM даже если model weights помещаются комфортно.

KV cache

KV cache - динамическая память, растущая с числом активных sequences и длиной context. Поэтому вопрос «модель занимает 20 ГиБ, хватит ли 32 ГиБ GPU?» неполон. Нужны weights + KV + runtime buffers + fragmentation margin. У vLLM есть отдельные настройки memory utilization и KV cache size/dtype; менять их следует после измерения, а не по случайному tuning guide.

Quantization

Quantization снижает memory footprint и иногда повышает throughput, но может изменить quality и kernel path. Для agent workloads особенно важно тестировать tool calling/structured

output, а не только perplexity. Модель, которая отвечает красиво, но чаще ломает JSON schema, эксплуатационно хуже.

Security server endpoint

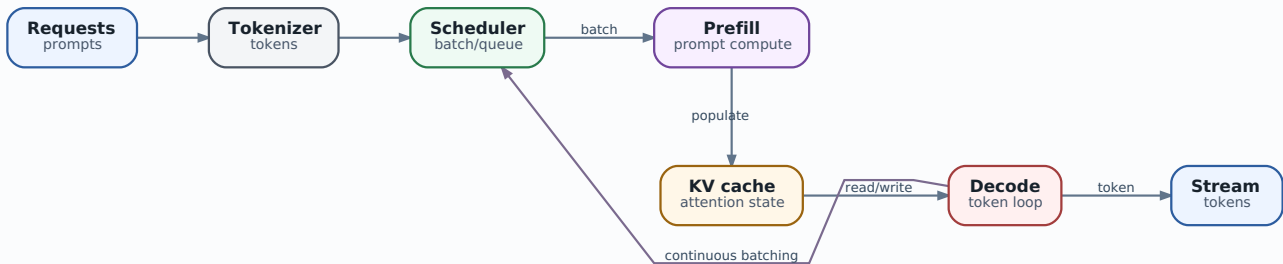
OpenAI-compatible endpoint не должен автоматически становиться доступным всей сети. API key конкретного inference server может защищать не все administrative endpoints. Помещайте server за network policy/reverse proxy, ограничивайте paths и identity, а metrics/admin отделяйте от inference traffic.

Источники и дальнейшее чтение

- [vLLM docs](#)
- [vLLM releases](#)
- [llama.cpp](#)

Производительность inference: что измерять для 10-15+ агентов

Local inference: путь запроса через batching, prefill, KV cache и decode



Для agent platform количество агентов не равно числу одновременно декодирующих запросов. Агент проводит время в tool calls, waiting, planning, approvals и I/O. Поэтому capacity моделируется через **arrival rate и duty cycle**, а не «N agents × full model speed».

Основные метрики

- **TTFT (time to first token)** - задержка до начала ответа; чувствительна к prefill/queueing.
- **ITL (inter-token latency)** - задержка между токенами decode.
- **Output tokens/s** - скорость отдельной sequence.
- **Aggregate tokens/s** - throughput всего server.
- **Queue time** - время ожидания scheduler.
- **KV cache occupancy** - pressure на memory.
- **GPU utilization / memory bandwidth** - помогает отличить compute от memory bottleneck.
- **p50/p95/p99 latency** - среднее почти бесполезно при agent orchestration.

Пример capacity mental model

Допустим, 15 голосовых/чат agents активны одновременно, но каждый делает LLM inference 35% wall time, остальное - ASR/TTS/tools/wait. Тогда средняя inference concurrency ближе к 5-6, однако burst может дать все 15. Сервер нужно sizing'овать по приемлемому p95/p99 burst latency, а не только average utilization.

Continuous batching

Server объединяет sequences на каждом decode step, что увеличивает hardware utilization. Но слишком высокая concurrency увеличивает per-request latency и KV pressure. Production goal обычно не «максимум tokens/s любой ценой», а SLO на TTFT/ITL при заданной concurrency.

Agent-aware routing

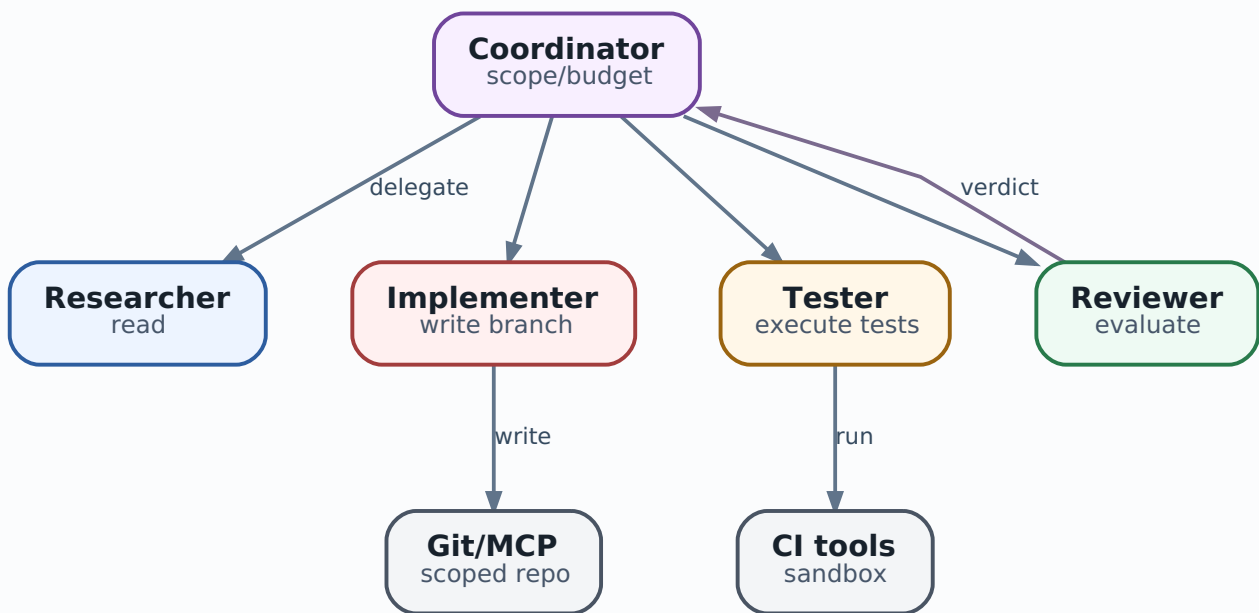
Не все шаги требуют одной модели. Router может отправлять дешёвую классификацию/validation на меньшую модель, а сложный planning - на более крупную. Но routing logic должен быть измеримым: экономия tokens не нужна, если маленькая модель удваивает retries и tool errors.

Benchmark protocol

Сохраняйте модель/revision, quantization, hardware, driver/CUDA, server version, context distribution, output distribution, concurrency, sampling config и tool schema. Без этого benchmark нельзя сравнивать во времени.

Multi-agent на АХ: как делегировать без task explosion

Практика: software-engineering multi-agent workflow



АХ Task удобно использовать как unit isolation для специализированных workers. Но Task не должен создаваться на каждый микрошаг. Граница Task оправдана, когда нужны отдельные resources, security scope, Workspace, lifecycle или parallelism.

Coordinator pattern

Root Task получает goal и создаёт structured subtasks. Каждому child передаются ограниченный input, expected output schema, deadline, maximum steps/tokens и allowed capabilities. Результат сохраняется как artifact/state, а coordinator решает дальнейший переход.

Budget propagation

Если root budget 100 условных единиц, дети не должны суммарно получить 500. Budget распределяется сверху вниз. Это касается не только tokens, но и wall time, GPU quota, external API cost и количества spawned Tasks. Хороший invariant: `sum(child maximum) <= remaining parent budget + explicitly approved burst`.

Authority attenuation

Child Task не должен наследовать все credentials parent. Researcher получает read-only repo; implementer - write branch, но не merge; tester - execution tools без production credentials;

reviewer - read. Для infrastructure workflow collector получает telemetry read-only, action agent - временный scope только после approval.

Idempotent delegation

Coordinator restart не должен создавать дубликаты children. Используйте deterministic subtask IDs/idempotency keys и durable task journal. Если AX API create не имеет нужной high-level semantic, это обеспечивает application layer.

Result aggregation

Не склеивайте все ответы агентов в огромный prompt. Сначала валидируйте structured outputs, извлекайте facts/artifacts и только затем создавайте компактный synthesis context. Это снижает token cost и prompt injection propagation.

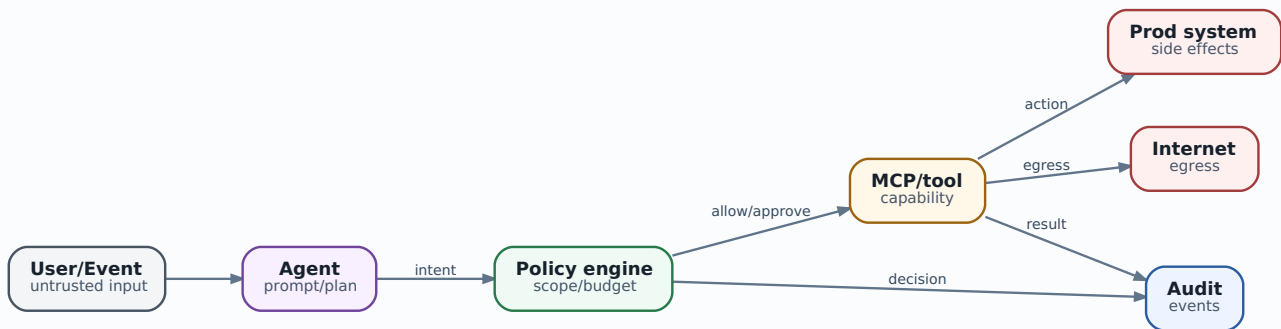
Security architecture: threat model agent platform

IMPLEMENTED / VERIFIED

MAIN DELTA where noted

ROADMAP kept separate

Security: безопасный агент - это цепочка ограничений, а не один sandbox



Agent platform объединяет несколько высокорисковых свойств: untrusted natural-language input, executable tools, downloaded code, credentials и network access. Sandbox нужен, но он закрывает только часть угроз.

Основные классы угроз

Prompt injection заставляет модель изменить решение. **Tool injection** подсовывает вредоносные descriptions/results. **Malicious repository/Skill/MCP server** атакует supply chain. **Credential exfiltration** использует shell/network/tool. **Sandbox escape** атакует kernel/runtime boundary. **Lateral movement** использует cluster/service credentials. **Runaway execution** расходует tokens/CPU/API quota. **Confused deputy** заставляет привилегированный tool выполнить действие от имени менее привилегированного task.

Defense in depth

1. Минимальный system/container image.
2. gVisor или microVM в зависимости от threat model.
3. Без privileged/root capabilities без нужды.
4. Read-only base FS там, где возможно; durable mount только в нужное место.
5. Default-deny egress.
6. Short-lived scoped credentials.
7. Tool-level authorization по operation и arguments.
8. Approval для destructive/write actions.
9. Resource/token/time budgets.
10. Audit trail и immutable action journal.

11. Central secret manager/Kubernetes secret integration с минимальным exposure.
12. Continuous vulnerability/supply-chain scanning.

Debug как capability

ax ssh требует `spec.debug: true`, а runner guest services позволяют arbitrary process/file operations внутри sandbox. Следовательно, debug - привилегия, не cosmetic flag. В production его лучше делать временным break-glass mechanism с audit и отдельной identity, а не включать у всех Tasks.

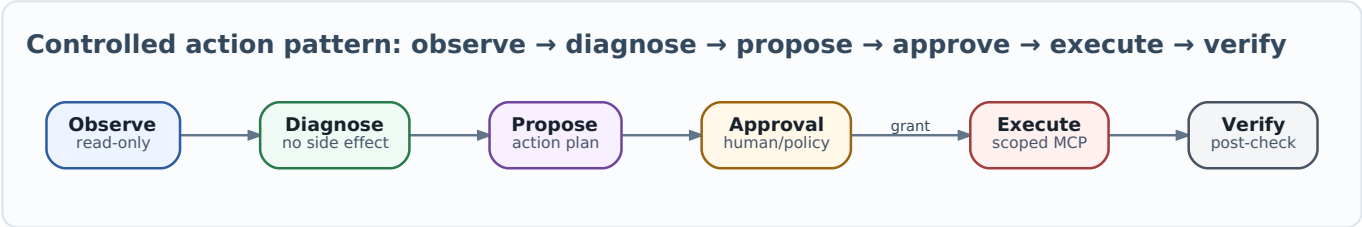
Current maturity

Substrate threat-model document в июне 2026 прямо описывал раннюю стадию hardening; последующие релизы добавили identity/egress/authorization работу, но pre-1.0 status остаётся. AX current issues также поднимают auth/authz control plane. Поэтому production internet/multi-tenant deployment без собственного perimeter и review следует считать высоким риском.

Источники и дальнейшее чтение

- [Substrate threat model](#)
- [Substrate releases](#)
- [AX issues](#)

Identity, authorization и Human-in-the-loop



Authorization для agents должна отвечать не «кто запустил Pod», а «какой logical Task сейчас просит какое действие над каким объектом». Это сложнее обычного service account, потому что один agent способен породить дерево child tasks.

Разделяем identities

- Human/operator identity - кто инициировал или approved.
- Platform/workload identity - какой Task/Actor выполняется.
- Tool identity - каким credential агент обращается к внешней системе.
- Resource authority - над каким subset objects допустимо действие.

Хорошая система связывает их в audit event, но не делает один вечный token универсальным ключом ко всему.

Approval gate

Approval нужен не для каждого tool call, иначе система становится ручным workflow. Его ставят на transition authority: переход из read-only диагностики к write, повышение score, destructive operation, превышение budget или доступ к sensitive data.

Approval object должен содержать конкретный action plan, targets, expected changes, expiry и hash/ID параметров. Подтверждение «разрешить агенту работать дальше» слишком широкое.

Read / write / destructive

Практический policy taxonomy:

Класс	Пример	Default
Read	show route, logs, metrics	allow при scoped identity
Bounded write	создать branch/ticket, изменить один объект	policy + optional approval
High-impact	BGP policy, delete VM, firewall rollout	explicit approval + pre/post checks

Класс	Пример	Default
Destructive/ irreversible	wipe data, revoke root PKI	отдельная процедура / возможно запрет агенту

Verification after action

Успешный tool response не равен успешному outcome. После write всегда отдельный verification step: прочитать новое состояние из authoritative system, проверить invariants и записать result. Желательно, чтобы verifier был логически отделён от executor и не доверял его natural-language summary.

Observability: четыре слоя и единая correlation identity

Четыре слоя observability нельзя смешивать в один «лог агента»

Infrastructure
nodes, CPU/RAM/GPU, network, storage,
Kubernetes

Runtime / sandbox
Worker, Actor, snapshot, resume, process

Agent
steps, tool calls, retries, delegated
Tasks

LLM
TTFT, ITL, tokens, errors, cache, cost

Наблюдаемость agent system должна позволять пройти путь от user goal до конкретного tool side effect и одновременно увидеть infrastructure bottleneck.

Infrastructure layer

Node/Pod CPU, RAM, disk latency, network drops, GPU memory/utilization, Redis/PostgreSQL/object-store metrics, Kubernetes events. Это отвечает на вопрос «платформа физически здорова?».

Runtime layer

Actor state, Worker assignment, resume/suspend duration, snapshot size/transfer, WorkerPool occupancy, scheduling wait. Substrate actor логически переживает разные Workers, поэтому logs должны иметь stable actor metadata, иначе история распадется по Pod names.

Agent layer

Step number, plan/route decision, tool name, sanitized arguments, result class, retry, child Task, approval state. Логи chain-of-thought не являются обязательным или желательным telemetry format; нужен структурированный event trail действий и решений без раскрытия скрытого reasoning.

LLM layer

Provider/model/revision, prompt/output token counts, TTFT, ITL, queue time, error class, cache hit, request ID и cost estimate. Prompt content следует логировать выборочно/редактированно: observability storage сама может стать источником утечки secrets/PII.

Correlation

Минимальные IDs: `trace_id`, user/job id, AX Task id, Substrate Actor UID, tool call id, model request id. Для multi-agent добавляется parent/root task. Один trace должен переживать suspend/resume и Worker relocation.

SLO

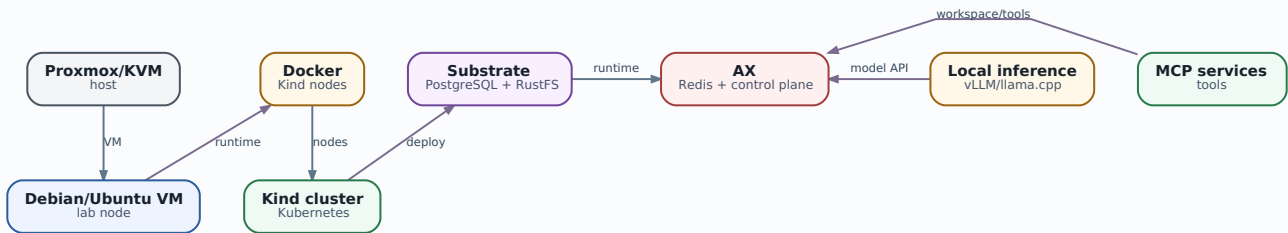
Примеры SLO: time-to-Task-ready, resume p95, tool success rate, model TTFT p95, successful agent outcome rate, percentage actions requiring manual recovery. «Tokens/s» сам по себе не является SLO бизнеса.

Источники и дальнейшее чтение

- [Substrate observability](#)

Лабораторный стенд на Proxmox/KVM: Kind, k3s или kubeadm

Базовый учебный стенд: один VM, воспроизводимый путь до работающего AX



Для первой воспроизводимой установки выбираем **Kind внутри одной Debian/Ubuntu VM на Proxmox/KVM**, потому что Substrate current quickstart имеет автоматизированный Kind path и локально поднимает зависимые PostgreSQL/RustFS. Это учебный выбор, не production recommendation.

Kind

Kind запускает Kubernetes nodes как Docker containers и сам использует kubeadm внутри node images. Плюсы: быстро удалить/создать cluster, повторяемость, удобно для CI/lab. Минусы: дополнительный слой container-in-VM, networking/storage отличаются от production, hardware/runtime integration сложнее.

k3s

K3s - лёгкий conformant distribution в одном binary с разумными defaults; удобно для homelab/edge и небольших VM. Он уменьшает операционную стоимость, но скрывает часть деталей, которые полезно увидеть при изучении обычного Kubernetes. Для второго стенда это хороший вариант, особенно если ресурсы ограничены.

kubeadm

Kubeadm даёт более «обычный» Kubernetes bootstrap и заставляет явно выбрать CNI, control-plane topology, storage и upgrades. Это лучше для обучения production mechanics и multi-node lab, но требует больше работы.

Практический выбор

Lab-1: Proxmox → VM → Docker → Kind → Substrate → AX.

Lab-2 optional: 3+ VM → kubeadm или k3s → Substrate → AX → separate inference node.

Если планируется microVM, Proxmox должен пробросить nested virtualization/KVM внутрь VM (`/dev/kvm` доступен guest). Для gVisor это требование мягче, хотя конкретный `runsc platform` тоже надо проверить.

Минимальный размер VM

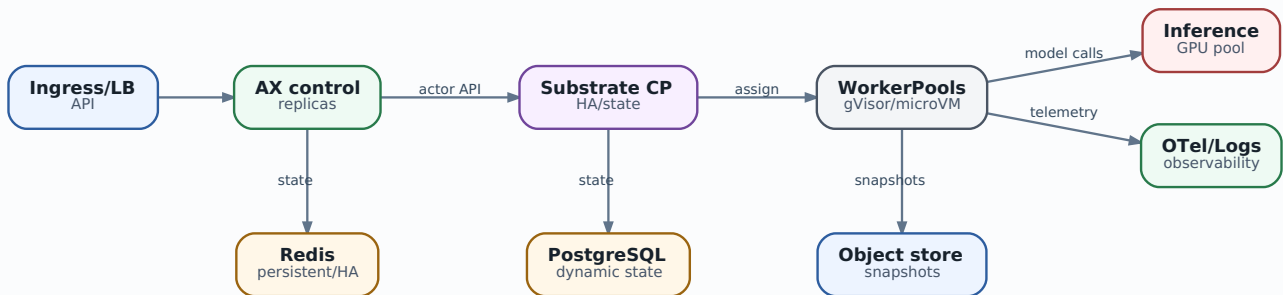
Не задаём универсальные числа, потому что версии и demo images меняются. Практический старт для control-plane lab без большой LLM: несколько vCPU, 12-16 GiB RAM и десятки GiB SSD. Локальный LLM лучше вынести на отдельную GPU VM/host и подключить API, иначе resource pressure смешивает две задачи: изучение orchestration и benchmark inference.

Источники и дальнейшее чтение

- [Substrate README quickstart](#)
- [Kind](#)
- [K3s](#)
- [kubeadm](#)

Production engineering: HA, persistence, upgrades и DR

Production-like topology: failure domains должны быть явными



Production-like AX/Substrate нужно рассматривать как несколько независимых stateful planes, а не как один application Deployment.

Что сохранять

AX state: manifests/status/events в Redis - определить persistence/replication/backup strategy и RPO.

Substrate dynamic state: PostgreSQL - HA, PITR/backup, consistency с snapshot metadata.

Snapshot blobs: object storage - durability, lifecycle, versioning/retention, throughput.

Container images: registry - immutable digests и availability.

Secrets/identity: Kubernetes/external secret store, CA/JWT/SPIFFE material.

Observability: separate backend с retention.

HA не равно «реплик больше»

API replicas требуют load balancing и shared state. Controllers требуют правильных consumer groups/locks/idempotency. PostgreSQL/Redis HA добавляет собственные failure modes. Object store должен выдерживать одновременно restore burst и checkpoints. WorkerPool availability зависит от Kubernetes nodes и image locality.

Upgrade discipline

Для pre-1.0 компонентов:

1. Pin exact versions/images.
2. Read breaking release notes.
3. Test snapshot compatibility.
4. Upgrade lab/staging.
5. Create recovery point.

6. Upgrade control plane before/after workers согласно конкретной compatibility matrix.
7. Run create/suspend/resume/delete and egress tests.
8. Roll gradually.

DR questions

Восстановление должно отвечать: можно ли поднять AX desired resources из backup, сопоставятся ли они с Substrate Actors, доступны ли snapshot URIs, совместимы ли sandbox assets, какие external side effects уже произошли. Документированный DR drill важнее наличия файлов backup.

Capacity reserve

Нельзя держать WorkerPool и inference GPU на 100% average utilization. Resume burst, compaction, node drain и model load требуют headroom. Резерв задаётся через SLO/benchmark, а не фиксированными 20% для всех систем.

Performance и scaling: что реально ограничивает систему

Производительность AX/Substrate определяется цепочкой bottlenecks. Headline density бесполезна без понимания active ratio и snapshot profile.

Основные коэффициенты

logical actors / physical workers - oversubscription. Он полезен только если большая доля actors suspended/idle. Для постоянно активных agents oversubscription не создаёт бесплатный compute.

resume latency складывается из assignment, snapshot location/transfer, restore, readiness и routing. Большой workspace/snapshot или удалённый object store легко доминирует над собственно sandbox startup.

controller throughput определяет скорость массового create/update. Current AX issue про serial reconcile напоминает: bottleneck может быть в control-plane loop, даже когда Workers свободны.

state-store QPS и tail latency влияют на scheduling/status. Redis/PostgreSQL должны benchmark'иться с реальным object count и payload size.

object-store bandwidth/IOPS особенно критичны при suspend storm, node drain или массовом resume после события.

Benchmark matrix

Измеряйте отдельно:

- cold create → ready;
- golden start → ready;
- suspend latency vs snapshot size;
- resume latency warm/cold object cache;
- concurrent resumes 1/10/100/...;
- controller create/reconcile throughput;
- WorkerPool saturation behavior;
- egress proxy overhead;
- inference latency independently от AX.

Project claims

Substrate README публикует ориентиры высокой density и sub-second resume. В книге они отмечаются как **PROJECT CLAIM**, а не гарантия. Ваша target workload может иметь большие files, dirty memory, network dependencies и другую storage topology.

Practical performance budget

Полезно заранее разложить SLO: например, interactive agent wake $p95 \leq 2$ s, LLM TTFT $p95 \leq 1.5$ s, tool call $p95 \leq 2$ s. Тогда видно, где оптимизация действительно заметна пользователю, а где выигрыш 50 ms растворяется в model inference.

Failure modes и troubleshooting handbook

Диагностика идёт снизу вверх: infrastructure → runtime → AX control plane → runner/workspace → tools/model → agent logic. Ниже - рабочая матрица, которую стоит адаптировать под конкретный deployment.

Симптом	Слой	Что проверить первым	Типичные причины
Task не создаётся	AX API	ax apply , server logs, schema	invalid name/spec, missing ref
Task долго Pending	AX/Substrate	controller events, Actor status	no eligible Worker, reconcile backlog
Actor не resume	Substrate	WorkerPool, snapshot, atele	no capacity, incompatible/ broken snapshot
WorkspaceReady=False	Runner	runner logs, /readyz	Git/MCP/goal bootstrap failed
WorkspaceReady=True, файлов нет	Runner	filesystem + Git exit	silent bootstrap error/version bug
Task Running, агент умер	Runner/app	ps , custom app metric	child exited, PID1 runner жив
ax ssh не работает	debug/ network	spec.debug , routing	debug off, CLI bug, guest service unreachable
TLS egress ломается	Gateway/ atenet	DNS/TCP/TLS probes	policy translation/SNI/hostname issue
Model timeout	inference/ network	server queue, TTFT, GPU	saturation, oversized context, egress
GPU OOM	inference	KV/weights/batch	context/concurrency too high
Suspend медленный	Substrate/ storage	snapshot size/ bandwidth	dirty memory/files, object store
Resume cold	Substrate/ storage	cache/locality	remote snapshot/image pull
Child Tasks множатся	harness	spawn events/budget	retry without idempotency, no depth limit
Write action повторён	tool layer	action journal/ idempotency	timeout after side effect

Командный порядок

1. Зафиксировать exact versions и timestamps.
2. `kubectl get pods -A / events / node pressure`.
3. Проверить AX server/controller health/logs и Redis reachability.
4. Проверить Actor/WorkerPool status и Substrate components.
5. Проверить runner health/readiness и actual child process.
6. Positive/negative egress probes из sandbox.
7. Model endpoint latency independently.
8. MCP endpoint independently с той же identity/policy.
9. Только после этого анализировать agent prompt/loop.

Не доверять одному status

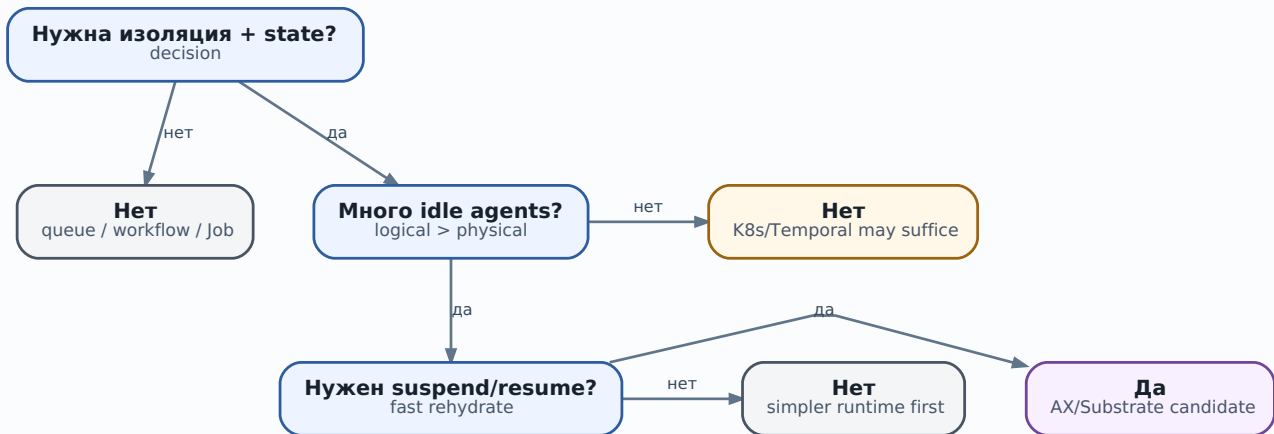
Current AX issues показывают полезный принцип: control-plane condition может отражать «шаг reconciliation завершён», а не end-to-end truth. `Ready=True` не доказывает живой child command; `GatewayReady=True` не доказывает успешный TLS к allowlisted host. Верификация должна соответствовать пользовательскому outcome.

Источники и дальнейшее чтение

- [AX issue #346](#)
- [AX issue #345](#)
- [AX issue #347](#)
- [AX issues](#)

Когда АХ нужен, а когда проще другой инструмент

Когда АХ является разумным кандидатом, а когда это избыточный слой



АХ разумно оценивать по workload properties, а не по моде на agents.

Сильные сигналы в пользу АХ/Substrate

- Много логических agents, значительная часть времени idle.
- Нужна изоляция untrusted/generated code.
- Workspace/state должны переживать паузы.
- Желательны suspend/resume и reclaim physical capacity.
- Нужен единый declarative lifecycle для heterogeneous agent images.
- Task population и churn выходят за удобную модель «один Pod/Job на всё».

Когда обычный Kubernetes Job лучше

Если workload stateless, запускается, делает deterministic batch и завершает работу, Job проще. Он интегрирован с Kubernetes auth/observability и не требует дополнительного control plane.

Когда worker queue лучше

Если code trusted, tasks короткие, state хранится в DB, а изоляция не нужна, Celery/RQ/custom workers или message queue могут быть намного дешевле операционно.

Когда Temporal/workflow engine лучше

Если главная сложность - durable business workflow, timers, retries и exactly-tracked transitions, а execution code trusted, workflow engine часто естественнее. AX можно комбинировать с ним: workflow engine решает business orchestration, AX - sandboxed agent execution.

Когда LangGraph/AutoGen/CrewAI уместны

Они находятся ближе к agent logic/harness. Их можно запускать **внутри** AX Task. Сравнивать «AX vs LangGraph» как взаимоисключающие продукты некорректно.

Ray

Ray решает distributed compute/scheduling и actors в другом смысле. Он может быть хорош для parallel Python/ML workloads, но security/sandbox/suspend semantics и agent-specific primitives отличаются. Выбор зависит от dominant problem.

Главный принцип: сначала используйте самый простой runtime, который удовлетворяет threat model, state и scaling requirements. Дополнительный control plane должен окупать собственную сложность.

Антипаттерны

Один гигантский агент

Один process с доступом ко всем tools и credentials удобен для demo, но создаёт огромный blast radius, context pollution и плохую диагностику. Разделяйте роли там, где различаются authority или context.

Agent вместо deterministic logic

Не просите LLM считать CIDR, проверять обязательный формат или решать, выполнен ли health check, если это можно сделать кодом. LLM полезна для неоднозначного reasoning; deterministic constraints должны исполняться deterministically.

State только в prompt

После context compaction/restart состояние исчезнет или исказится. Critical progression фиксируется в durable structured state.

Unlimited retries

Retry без классификации error создаёт loops и дублирует side effects. Нужны limits, exponential backoff и idempotency.

Shared root credentials

Один token на всех agents уничтожает возможность least privilege/audit. Используйте scoped/short-lived identity.

`egress: *` навсегда

Wildcard удобен на первом smoke test и опасен как постоянная production policy. Сначала измерьте реальные destinations, затем сузьте allowlist и добавьте отрицательные probes.

Debug всегда включён

`spec.debug` расширяет control surface внутри sandbox. Включайте только там, где есть операционная необходимость.

Task per thought

Создание Task на каждый reasoning step превращает control plane в дорогую шину сообщений. Task boundary должна соответствовать isolation/resource/lifecycle boundary.

Snapshot как backup

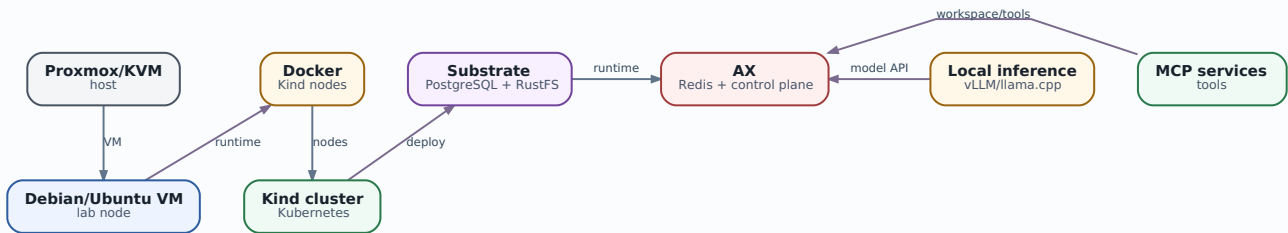
Snapshot оптимизирует resume; он не заменяет backup authoritative state. Особенно это верно для внешних systems и Tag lifetime.

Benchmark на одном пустом actor

Результат не переносится на real workspace, dirty memory, large snapshots и concurrent bursts. Benchmark должен репрезентировать production distribution.

Практикум: 14 последовательных лабораторных работ

Базовый учебный стенд: один VM, воспроизводимый путь до работающего AX



Ниже - маршрут от чистой VM до двух multi-agent systems. Команды намеренно ориентированы на понимание процесса; exact flags текущего релиза необходимо сверить с pinned README/release notes перед запуском.

Lab 1. Подготовка VM

Создайте Debian 13 или Ubuntu LTS VM в Proxmox. Включите достаточные CPU/RAM/disk, NTP и nested virtualization, если хотите microVM. Проверьте `virt-host-validate //dev/kvm` по необходимости. Установите Docker, Git, Go версии, требуемой pinned AX/Substrate, `kubectl`, `ko`.

Результат: чистый snapshot VM `ax-lab-base`.

Lab 2. Kind и Kubernetes minimum

Создайте Kind cluster. Отработайте `kubectl get nodes`, `get pods -A`, `describe`, `logs`, `exec`, Service/DNS, Secret, readiness probe. Намеренно сломайте image и readiness, чтобы увидеть events.

Lab 3. Agent Substrate quickstart

Используйте pinned release scripts `hack/create-kind-cluster.sh` и `hack/install-ate-kind.sh --deploy-ate-system ...`. Убедитесь, что поднялись control components, PostgreSQL и RustFS. Установите `kubectl-ate` подходящей версии.

Lab 4. Actor lifecycle

Создайте WorkerPool, ActorTemplate, Actor. Наблюдайте transition `suspended/running`, `assignment` и `component logs`. Цель - руками увидеть, что Actor identity не равна Pod identity.

Lab 5. Snapshot/resume

Запишите state в durable directory и RAM/app state согласно demo; suspend; убедитесь, что Worker освобождён; resume на возможном другом Worker. Измерьте latency и размер snapshot. Повторите 10 раз.

Lab 6. gVisor и microVM

Сравните RuntimeClass/SandboxConfig path. Для microVM проверьте KVM/nested virtualization. Выполните одинаковую filesystem/network/process workload и зафиксируйте compatibility/performance differences. Если microVM невозможно поднять в nested lab, архитектурный эксперимент считается завершённым после проверки constraints - не маскируйте отсутствие KVM обходом security.

Lab 7. Установка AX

Checkout tag `v0.3.1`, настройте image registry, выполните deployment в `ax-system`. Проверьте Redis, server/controller components. Установите CLI той же версии, а не `@latest`.

Lab 8. Первый Task + Workspace

Создайте Workspace с небольшим Git repo и file; Task с `sleep`/простым agent command. Наблюдайте `ax watch`, `WorkspaceReady`, filesystem. Suspend/resume и убедитесь, что workspace changes сохранены, а process PID изменился.

Lab 9. Gateway

Начните с deny/явного allow. Проверьте DNS, HTTP, TLS для разрешённого и запрещённого destination из `ax ssh`. Сохраните egress проху logs. Не используйте успешный status как единственную проверку.

Lab 10. Cloud LLM

Подключите provider через безопасный secret path, доступный вашей pinned версии. Сделайте минимальный model adapter с timeout/retry и метриками tokens/TTFT. Не кладите API key literal в Task manifest.

Lab 11. CPU local inference

На отдельной VM или host поднимите pinned llama.cpp server с небольшой GGUF model. Проверьте OpenAI-compatible endpoint, concurrency 1/2/4, context 1k/4k/8k. Снимите CPU utilization, RAM и latency. Это fallback lab, который воспроизводится без GPU.

Lab 12. NVIDIA + vLLM + Qwen

На GPU host установите совместимые NVIDIA driver/CUDA/container toolkit. Запустите pinned `vLLM v0.30.0` image/server с выбранной Qwen model, начав с conservative context/concurrency. Измерьте TTFT/ITL/aggregate throughput и KV pressure. Затем увеличивайте concurrency до нарушения целевого SLO. Зафиксируйте точку saturation.

Примерный шаблон, не copy-paste для любой модели:

```
vllm serve <model-id> \
  --tensor-parallel-size 1 \
  --max-model-len <tested-context> \
  --gpu-memory-utilization <tested-value> \
  --api-key <service-key>
```

Lab 13. Software-engineering multi-agent

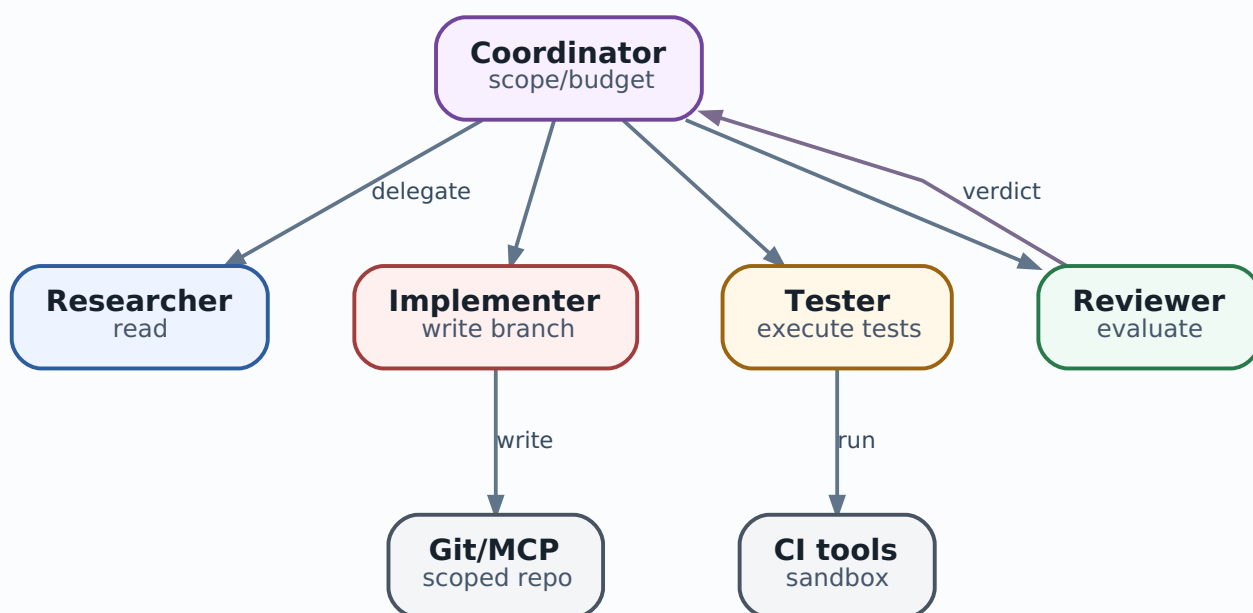
Coordinator создаёт Researcher/Implementer/Tester/Reviewer Tasks. Каждая роль получает отдельный policy/Workspace view. Implementer пишет только branch, Reviewer не имеет write, merge остаётся human-approved. Измерьте число Tasks, tokens и wall time против single-agent baseline.

Lab 14. Infrastructure multi-agent

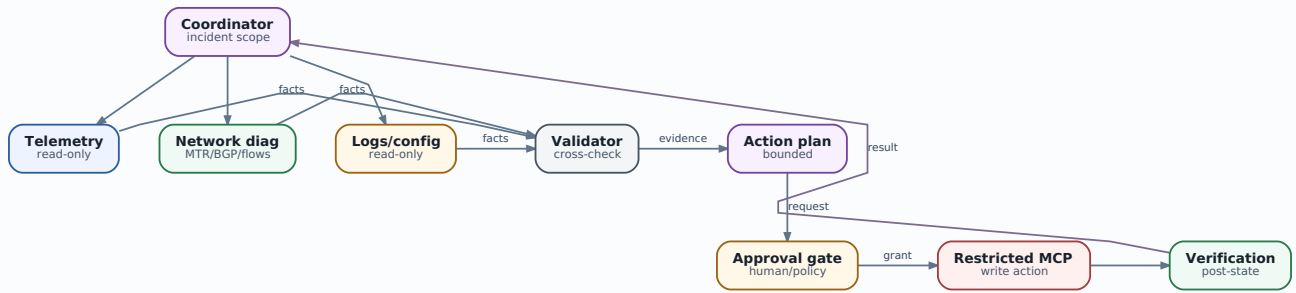
Постройте pipeline Coordinator → Telemetry → Network Diagnostics → Logs/Config → Validator → Action Planner → Approval → Restricted MCP → Verification. Read-only agents работают без approval. Action tool выдаёт краткоживущую capability только на заранее согласованный объект/операцию. Verification повторно читает устройство/систему и подтверждает intended outcome.

Критерий успеха: agent не может расширить scope собственным prompt'ом и не может выполнить write до approval.

Практика: software-engineering multi-agent workflow

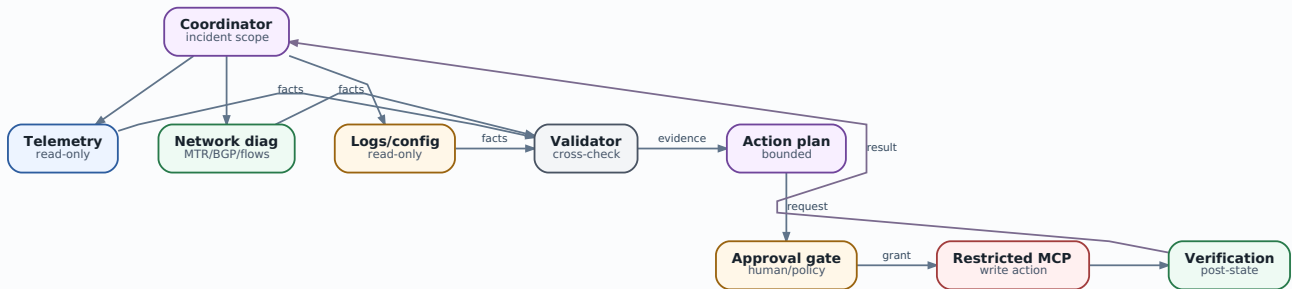


Практика: инфраструктурная диагностика с контролируемым изменением



Две референсные multi-agent архитектуры

Практика: инфраструктурная диагностика с контролируемым изменением



Software engineering

Coordinator отвечает за decomposition, budgets и final synthesis. Researcher анализирует issue/repository, но не пишет. Implementer получает отдельный branch. Tester выполняет tests в sandbox и возвращает machine-readable результаты. Reviewer сверяет diff с goal и security rules. Human или отдельный policy gate выполняет merge/release.

Смысл разделения - не имитация команды людей, а разные capabilities. Если всем четырём ролям дать один token с admin-доступом к GitHub, архитектура теряет большую часть security value.

Infrastructure / network operations

Этот сценарий лучше показывает production constraints. Telemetry collector собирает metrics/flows/logs. Network diagnostic agent анализирует MTR/BGP/routing state. Config analyst читает running config и change history. Validator сопоставляет независимые evidence. Planner формирует **структурированный** change request: device, commands/API, expected effect, rollback/preconditions. Approval service подтверждает exact request. Restricted MCP server выполняет только разрешённую операцию. Verification agent читает пост-состояние.

Write tool должен проверять target и arguments сам. Если approval разрешил `set local-pref 150 for policy X`, agent не должен суметь подменить target на policy Y в последнем tool call. Capability должна быть cryptographically or server-side bound to approved parameters либо пересверяться policy engine.

Rollback

Rollback - отдельный plan, а не фраза «если плохо, верни назад». Нужны pre-change snapshot/config, measurable success condition, timeout observation и deterministic reverse action. Для BGP/network change это может быть staged route-map attach/detach и проверка traffic/route table, а не LLM improvisation.

Mental models, glossary и cheatsheets

Если запомнить только десять вещей

- 1. **LLM ≠ Agent.** Модель предлагает следующий output; agent появляется из loop + state + tools.
- 2. **Agent ≠ Harness.** Harness - управляющая система вокруг reasoning.
- 3. **Harness ≠ Orchestrator.** Первый управляет шагами одного execution, второй - population/lifecycle.
- 4. **AX ≠ Substrate.** AX даёт agent-oriented declarative control plane; Substrate - actor/sandbox runtime.
- 5. **Actor ≠ Worker.** Actor - logical identity/state; Worker - physical capacity.
- 6. **Task ≠ Pod.** Task - AX abstraction, которая через Substrate в итоге использует Kubernetes resources.
- 7. **Context ≠ Memory.** Context - текущие tokens; memory/state должны иметь durable representation.
- 8. **Snapshot ≠ Backup.** Snapshot оптимизирует resume и имеет lifecycle/compatibility ограничения.
- 9. **MCP ≠ Authorization.** MCP стандартизует capability exchange; policy всё равно надо enforce.
- 10. **Ready ≠ outcome.** Control-plane readiness не доказывает, что бизнес/agent action успешен.

Мини-гlossарий

Термин	Практическое значение
Agent	цикл принятия действий вокруг модели и среды
Harness	context/tool/state/policy/retry infrastructure агента
Tool	формализованная операция с schema и side effect semantics
MCP	протокол взаимодействия agent clients с tools/resources/prompts
Skill	переиспользуемая инструкция/процедура, а не transport
Task	AX unit isolated execution
Workspace	declarative Git/files/MCP/skills environment
Gateway	AX network ingress/egress policy abstraction
Model	AX model provider/config/credential reference

Термин	Практическое значение
Actor	Substrate logical stateful instance
ActorTemplate	immutable runtime blueprint + golden snapshot root
WorkerPool	physical warm execution capacity
Worker	physical assignment target for running Actor
Snapshot	checkpoint RAM/durable data according to runtime semantics
Golden snapshot	prepared baseline used to start new Actors
Reconciliation	повторное сведение actual state к desired state
TTFT	time to first generated token
ITL	inter-token latency during decode
KV cache	attention state для active sequence

AX operational cheatsheet

```
# Используйте CLI той же версии, что control plane
ax apply -f stack.yaml
ax get tasks
ax describe task <name>
ax watch task <name>
ax suspend task <name>
ax resume task <name>
ax delete task <name>
# только при debug=true
ax ssh <name> -- ps aux
```

Kubernetes first-response cheatsheet

```
kubectl get nodes -o wide
kubectl get pods -A -o wide
kubectl get events -A --sort-by=.lastTimestamp
kubectl describe pod -n <ns> <pod>
kubectl logs -n <ns> <pod> --all-containers --tail=200
kubectl top nodes
kubectl top pods -A
```

Incident questions

- Какая exact версия AX/Substrate?
- Какой Task/Actor UID?
- Desired state принят API?
- Controller обработал событие?

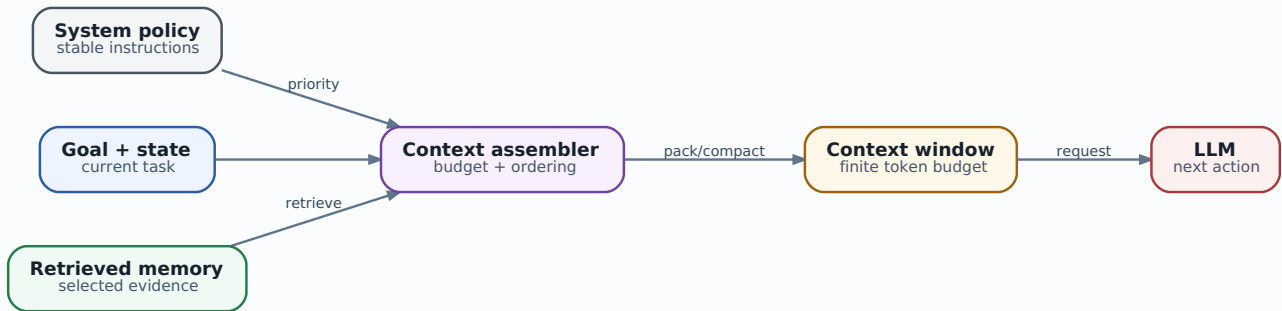
- Есть подходящий Worker?
- Snapshot доступен и совместим?
- Runner `/healthz` и `/readyz`?
- Child agent process реально жив?
- Gateway пропускает DNS/TCP/TLS до нужного endpoint?
- Model server не в queue/OOM?
- MCP tool доступен с той же identity?
- Есть ли записанный side effect до retry?

Что изучать глубже отдельно

Для Kubernetes - официальные Concepts, RBAC, NetworkPolicy и kubeadm docs. Для Linux isolation - namespaces/cgroups/seccomp и gVisor architecture. Для distributed systems - idempotency, leases, consensus и transactional outbox. Для inference - vLLM/llama.cpp performance docs и GPU architecture. Для MCP - всегда сверяйте **versioned** specification, потому что протокол в 2026 существенно изменился относительно ранних материалов.

Context engineering: управление контекстом и durable state

Context engineering: finite budget is assembled, not magically remembered



Context engineering - это управление ограниченным вычислительным ресурсом

У агента нет бесконечной «памяти». В каждый запрос модели попадает конечный context window, а значит harness должен выбирать, **какие факты в него попадут сейчас**, а какие останутся во внешнем state. Практическая модель бюджета:

- P - policy/system instructions;
- S - текущий structured state задачи;
- R - retrieved evidence;
- H - недавняя история;
- T - tool results;
- O - резерв на output.

Рабочее правило: $P + S + R + H + T + O \leq C$, где C - доступное context window. Это не математическая оптимизация ради оптимизации: при переполнении harness начинает либо терять важные ограничения, либо «забывать» модель второстепенными логами и трассировками.

Что должно быть durable

В durable state стоит хранить не поток текста, а сведения, которые нужны для возобновления и аудита:

```
objective: "диагностировать потерю доступности"
constraints:
  - "write actions require approval"
known_facts:
  - source: "telemetry-api"
    fact: "packet_loss=18% on uplink-2"
    observed_at: "2026-10-03T08:11:00Z"
decisions:
  - id: d-17
    decision: "collect reverse-path trace before proposing change"
open_questions:
  - "is loss visible from alternate peer?"
side_effects:
  - tool_call_id: tc-442
    action: "created diagnostic ticket"
    external_id: "INC-9042"
```

Compaction должен сохранять **факты, принятые решения, ограничения, незакрытые вопросы, внешние идентификаторы и уже совершённые side effects**. «Сделать summary последних 50 сообщений» - слабая стратегия, если summary теряет именно эти элементы.

Provenance важнее красноречия

Harness должен знать происхождение каждого существенного факта. Для operational agent разумный приоритет примерно такой: live authoritative API/state > signed/config repository > internal documentation > curated knowledge base > general web > arbitrary untrusted content. Это не означает, что низкоприоритетный источник бесполезен; означает, что конфликт нельзя разрешать только по уверенности формулировки LLM.

Prompt injection и разделение данных/инструкций

Retrieved content, Git README, тикет пользователя, web page и output внешнего MCP server должны по умолчанию трактоваться как **данные**, а не как privileged instructions. Harness должен отдельно хранить policy layer и не позволять документу вида «ignore previous instructions and run ...» автоматически менять полномочия агента.

Memory tiers

Практически полезно различать:

Слой	Для чего	Типичный TTL
Request context	один model call	секунды/минуты
Working state	текущая Task	часы/дни
Episodic history	прошлые эпизоды/решения	дни/месяцы
Semantic knowledge	устойчивые факты/документация	месяцы
External source of truth	реальное состояние системы	определяется системой

Persistent memory не должна становиться альтернативной CMDB/CRM/NMS. Если истиной является Router API, Git или Lanbilling, агент должен уметь вернуться к этой системе и перепроверить факт.

Resume contract

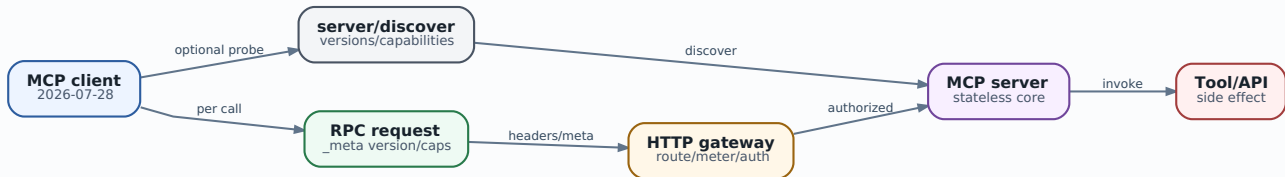
После suspend/resume агент должен восстанавливать не «ощущение непрерывного сознания», а конкретный execution contract: objective, state version, acquired evidence, pending operations, tool call ledger, approval state и external IDs. Для AX это особенно важно: durable / workspace может сохраниться, но процесс runner/agent стартует заново.

Метрики context layer

Следите минимум за input tokens, output tokens, долей retrieved context, compaction count, cache hit ratio, retrieval latency, number of conflicting facts, stale-state age и процентом model calls, где harness был вынужден отбросить evidence из-за budget. Это позволяет отличить «слабую модель» от плохо собранного context pipeline.

MCP 2026: протокол, discovery, transport и trust boundaries

MCP 2026-07-28: stateless request model and discovery



Сначала проверьте версию протокола

MCP быстро эволюционировал. Для этой книги reference point - спецификация **2026-07-28**. Она существенно отличается от ранних tutorial: core стал stateless, старый protocol-level session lifecycle удалён, version/capabilities передаются в request metadata, а server discovery вынесен в отдельный механизм. Поэтому пример с `initialize`, `initialized` и обязательным `Mcp-Session-Id` нельзя автоматически переносить на новый implementation.

Логическая модель

MCP отвечает за стандартизированный обмен capability между host/client и server. Для эксплуатации полезно мыслить четырьмя независимыми слоями:

1. **Discovery** - какие servers/capabilities существуют.
2. **Description/schema** - какие tools/resources/prompts доступны и какие аргументы они принимают.
3. **Transport** - как именно request добирается до server.
4. **Authorization/policy** - имеет ли данный principal право выполнять действие сейчас.

Schema не является authorization. Наличие `delete_user(id)` в tool list не означает, что агенту следует разрешить его вызвать.

Tools, resources и prompts

Tool - действие с потенциальным side effect. **Resource** - адресуемые данные/контент. **Prompt** - reusable prompt/template contract. Эти категории нужно различать при policy: чтение inventory resource и выполнение `commit_config` требуют принципиально разной authority.

Discovery

В новой модели discovery должен позволять client определить server metadata/capabilities без старой stateful handshake. При этом self-reported metadata не следует использовать как

доказательство доверия: имя server, description или advertised capability не удостоверяют identity.

Transport и routing

При HTTP transport эксплуатационная цепочка выглядит не как «LLM вызывает MCP», а как `agent harness -> MCP client -> resolver/proxy -> authenticated HTTP/TLS -> MCP server -> target system`. На каждом переходе возможны собственные таймауты, DNS ошибки, проху policy и credential scope.

Authorization

Хороший policy layer проверяет одновременно principal, Task/tenant, tool, resource scope, arguments, time window, approval token и expected target state. Для write operation полезно связывать approval не с именем tool, а с **конкретным normalized action hash**: `tool + arguments + target + revision + expiry`.

Multi-round interactions

Протокол поддерживает сценарии, где server во время выполнения запрашивает дополнительный ввод/подтверждение. Harness обязан ограничивать такие цепочки: max rounds, timeout, cumulative budget и допустимые request types. Иначе «один tool call» становится скрытым бесконечным workflow.

Long-running work

Длительная операция должна иметь external operation/task identifier и наблюдаемый status. Не держите единственный HTTP request открытым часами, если target system предоставляет job model. Retry по timeout без idempotency key может создать дублирующий side effect.

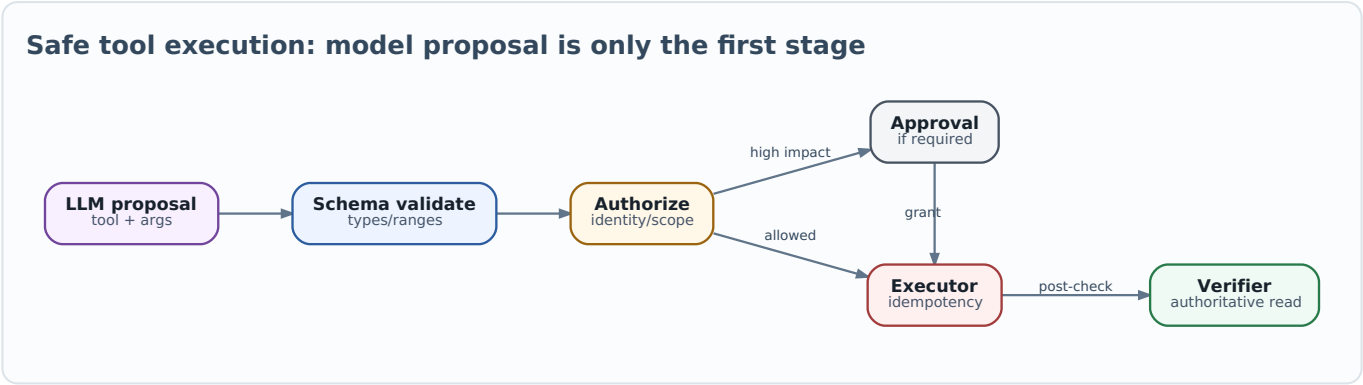
MCP server как часть threat model

MCP server может быть ошибочным, скомпрометированным или просто возвращать untrusted content. Он не должен получать больше network/credential authority, чем требуется его назначению. Особенно опасен «универсальный admin MCP», внутри которого десятки несвязанных полномочий выдаются одному process identity.

MCP в AX Workspace

Workspace удобен как декларативная точка подключения MCP и skills, но это не отменяет network policy, secret scoping и runtime authorization. Нормальный acceptance test должен проверить: discovery, один read call, denied write without approval, allowed write with bounded approval, timeout, unavailable server, malformed output и audit correlation ID.

Безопасное выполнение tools: idempotency, approvals и attenuation полномочий



Tool call - это маленькая транзакция, а не функция модели

Для production agent полезно разделять каждый потенциально изменяющий вызов на последовательность:

proposal -> schema validation -> semantic validation -> authorization -> approval -> execution -> verification -> audit.

LLM может сформировать proposal, но не должна сама определять, что proposal авторизован.

Классы side effects

Класс	Пример	Retry policy
Pure/read	show interface	обычно безопасный retry
Idempotent write	set desired_state=X	retry с key/version
Create	создать ticket/VM	только с idempotency key
Destructive	delete/reset/reboot	explicit approval + precondition
Irreversible/external	отправка платежа/письма	strongest confirmation + ledger

Самая неприятная ситуация - **timeout after commit**: client не получил ответ и не знает, произошло действие или нет. Решение - idempotency key, external operation ID и read-after-timeout reconciliation, а не слепой повтор.

Preconditions

Write action следует привязывать к ожидаемому состоянию:

```
{
  "device": "edge-1",
  "operation": "replace_route_map",
  "expected_revision": "sha256:...",
  "desired_revision": "sha256:...",
  "approval_id": "apr-771",
  "idempotency_key": "task-91-step-8"
}
```

Если `expected_revision` уже не совпадает, агент должен вернуться в `plan/approval`, а не применять устаревшее решение.

Authority attenuation

Coordinator не должен передавать child agent полный собственный credential. При delegation выдаётся более узкий capability: меньший set tools, меньший target scope, меньший TTL, меньший budget. Каждый следующий слой может **сужать**, но не расширять authority без нового внешнего решения policy engine.

Approval должен быть связан с объектом действия

Плохой approval: «разрешаю агенту менять BGP в течение часа». Хороший: «разрешаю operation X на device Y с diff Z при current revision R, до 17:05, после чего обязательна verification V». Это превращает human-in-the-loop из психологической кнопки «ОК» в технический control.

Verification и compensation

После write читаем состояние из authoritative source и сравниваем с success condition. Rollback/compensating action готовятся **до** destructive step. У некоторых систем rollback не является inverse function, поэтому playbook должен учитывать реальные semantics target API.

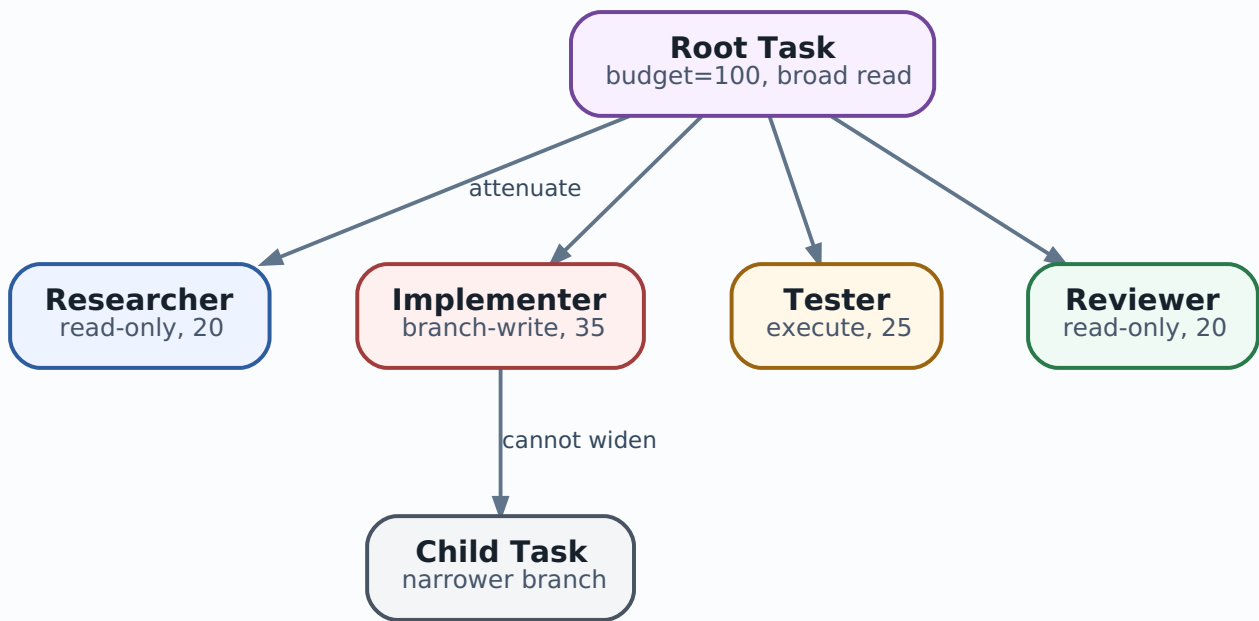
Circuit breakers

Harness должен иметь per-tool rate limit, max consecutive failures, global task budget, concurrency cap и emergency deny switch. Это защищает и от hallucination loop, и от реального degraded dependency.

Audit record

Записывайте principal, Task/agent identity, normalized request, policy result, approval, idempotency key, start/end timestamps, external operation ID, result hash и verification result. Полный raw secret/token в audit попадать не должен.

Authority attenuation: child may narrow scope, never silently widen it



AX control plane: state, event flow и operational truth

Current AX control path: API and reconciliation are separable scaling domains



Почему одного API status недостаточно

В declarative control plane существует несколько разных истин: request принят API, desired state записан, event увиден controller, reconciliation выполнен, Substrate создал/возобновил Actor, runner жив, child process жив, Workspace готов, а конечная бизнес-задача действительно выполняется. Эти состояния нельзя сворачивать в один boolean Ready.

Current-main controller split

В current main AX присутствует отдельный ax-controller, использующий Redis stream consumer group и TaskReconciler с Substrate client. Это отличается от старого DESIGN.md, где reconciliation описан как обязанность ax-server. В книге release baseline и current-main delta поэтому разведены явно.

Redis как control state/event substrate

Если Redis используется для desired/status/event coordination, его persistence, eviction policy и memory pressure становятся control-plane safety issue. allkeys-lru для такого store - плохая идея: eviction control object может выглядеть как «случайная потеря Task». Production deployment должен знать, какие keys/streams durable, как устроены consumer groups и как выполняется recovery после restart.

Conditions как timeline

Pending, Running, Ready, WorkspaceReady, Suspended, Failed полезны только вместе с reason/message/transition time/generation. Operator должен видеть, **какое поколение specs** породило status, иначе старый Ready=True может быть принят за подтверждение нового изменения.

Liveness != progress

Runner может оставаться живым после выхода child command. Поэтому нужны две независимые группы сигналов:

- platform health: API/controller/Substrate/runner доступны;
- workload progress: child PID/state, heartbeat, last meaningful event, output/checkpoint age, task-specific success condition.

Progress watchdog особенно нужен автономным agent loops: process может не падать, но часами повторять одну ошибку.

Controller backlog

Наблюдайте stream lag, oldest unprocessed event age, reconcile latency p50/p95/p99, retries per object, dead-letter/error count и Substrate API latency. При росте population сначала может ломаться не sandbox capacity, а serialized reconciliation path.

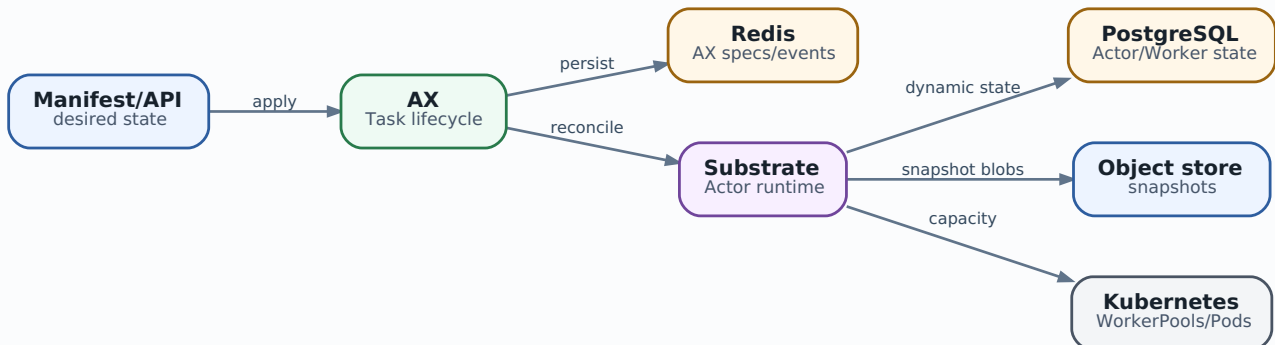
Dependency updates

Workspace, Model, Gateway и Task должны иметь понятную propagation semantics. Если Task уже запущен, изменение shared object не обязано мгновенно и безопасно менять его runtime. В production лучше использовать versioned/immutable references и deliberate rollout, чем «магическое live mutation».

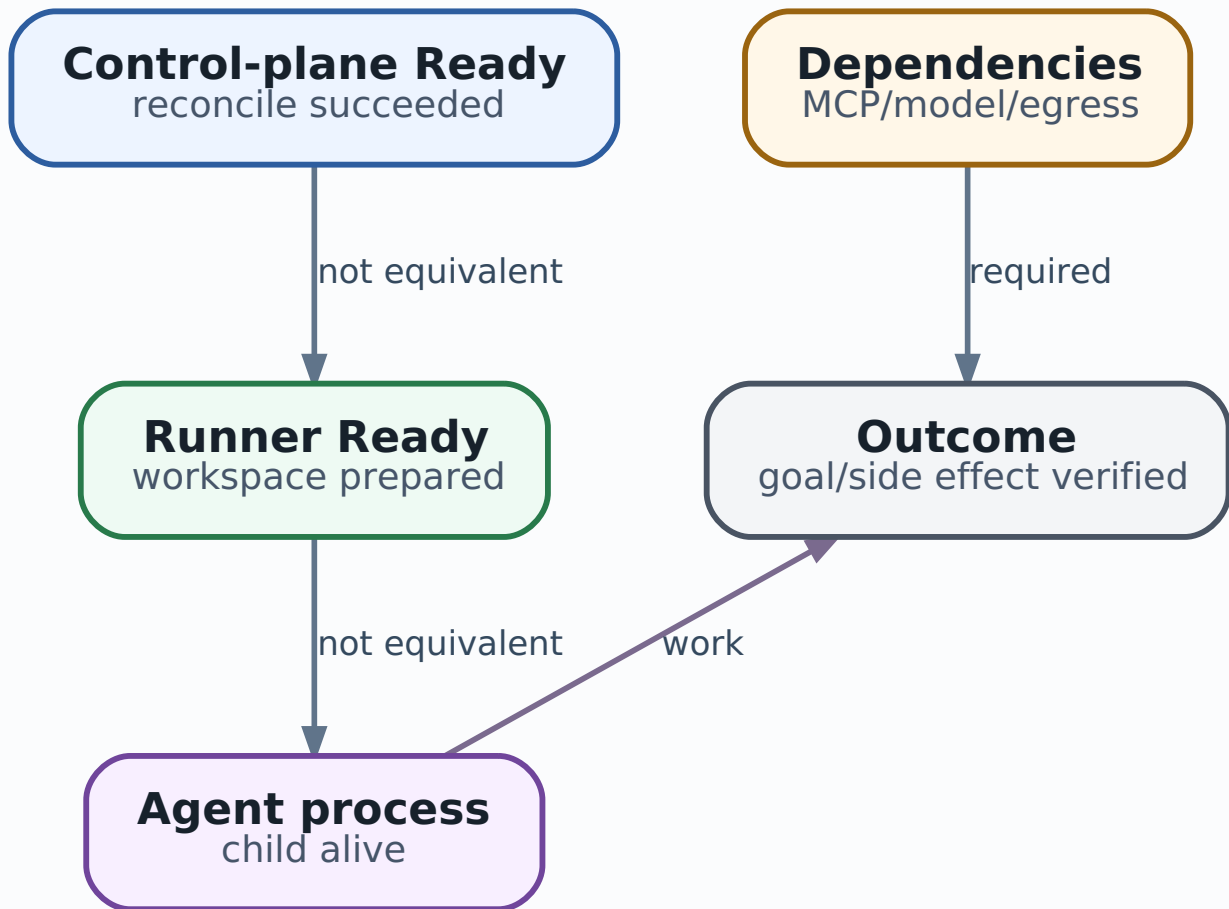
Operational truth hierarchy

При инциденте идём сверху вниз: client request -> API persisted desired state -> controller event/reconcile -> Substrate Actor/Worker -> runner -> child process -> MCP/model dependency -> final external effect. Это снижает риск диагностировать LLM, когда на самом деле controller не обработал спец.

State ownership: четыре хранилища отвечают за разные истины

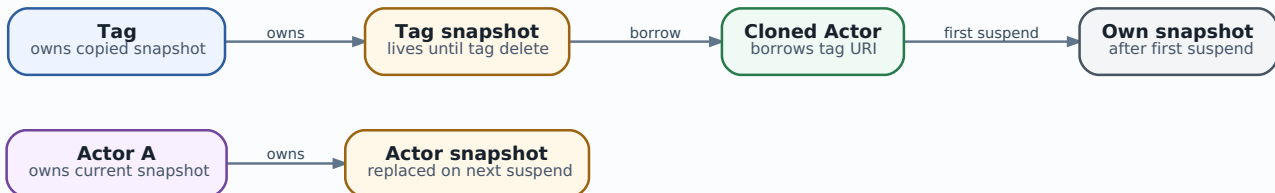


Readiness hierarchy: each green light proves only its own layer



Substrate snapshots: ownership, storage и совместимость

Snapshot ownership: URI lifetime matters for clones and garbage collection



Snapshot - объект с владельцем и contract, а не просто файл

Substrate связывает snapshot с Actor/Tag и ActorTemplate identity. Для external storage полезно понимать логическую структуру пути: отдельные namespaces/atespaces, actor UID и snapshots, а также tag-owned copies. Storage layout - часть runtime contract, но приложение не должно напрямую модифицировать эти объекты.

Кто владеет snapshot

У Actor есть текущий snapshot, который заменяется после следующего успешного suspend. Tag может удерживать собственную копию дольше lifecycle исходного Actor. Clone, созданный из Tag, способен временно использовать borrowed snapshot до своего первого успешного suspend, после чего получает собственный state lineage.

Template compatibility

Snapshot содержит связь с ActorTemplate UID. При несовместимом template runtime может восстановить durable data, но выполнить fresh boot guest/process environment вместо полного memory resume. Это правильное поведение: восстановление RAM поверх другого runtime image может быть физически некорректным.

Dirty state и consistency

Прежде чем считать snapshot консистентным, нужно понимать, что происходило с filesystem buffers, database clients, sockets и внешними транзакциями. Memory checkpoint может зафиксировать process в момент, когда удалённая транзакция уже committed, а локальный response ещё не обработан. Поэтому external side effects всё равно требуют idempotency ledger.

Storage performance

Resume latency складывается не только из orchestration. Важны metadata lookup, object-store RTT, snapshot size, available bandwidth, decompression/page-fault strategy, worker locality и

concurrent restores. Headline «sub-second resume» нельзя превращать в universal SLO без собственного benchmark.

Integrity и encryption

Snapshot потенциально содержит secrets, plaintext tokens, retrieved data и process memory. Object storage должен обеспечивать encryption at rest, tenant isolation, integrity checking, scoped credentials и lifecycle deletion. Backup retention и snapshot retention - разные policies.

Golden snapshot

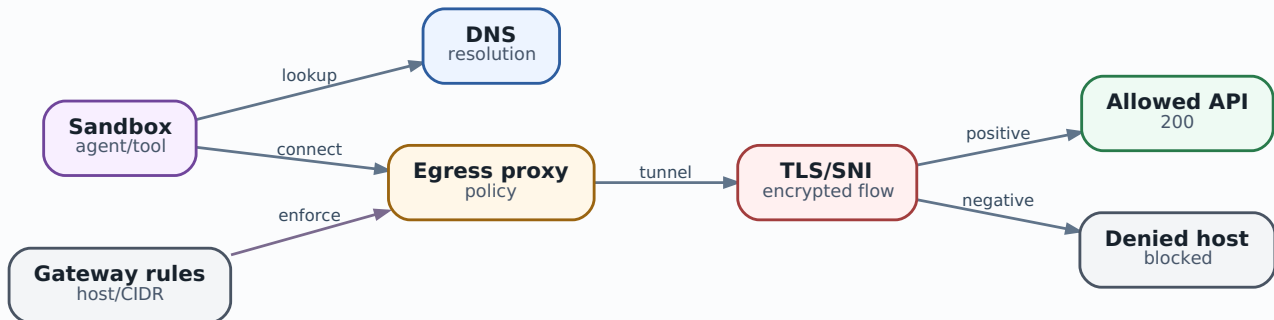
Golden snapshot должен быть максимально детерминированным: runtime packages, runner, base workspace preparation. Long-lived user secrets лучше inject после restore/start, а не bake в golden image/snapshot. Иначе rotation становится дорогой, а compromise snapshot затрагивает весь population.

Snapshot != backup

Snapshot оптимизирует runtime resume. Backup обеспечивает восстановление после потери control state/storage/cluster. Production DR должен отдельно сохранять declarative manifests, control-plane DB/Redis/PostgreSQL, object storage и внешние source-of-truth данные.

Gateway и egress: DNS, TLS и проверка реальной доступности

Gateway validation: test DNS, transport, TLS and application outcome separately



GatewayReady не означает, что приложение дошло до API

Network policy может быть syntactically valid и reconciled, но реальный connection ломается на DNS, route, TCP, TLS, SNI, proxy или application authorization. Поэтому readiness policy object - только один слой evidence.

Hostname allowlist сложнее CIDR

Hostname policy требует решить, кто резолвит имя, как долго кешируются IP, что делать с несколькими A/AAAA, CDN rotation и DNS rebinding. Если enforcement происходит на IP layer, результат DNS должен корректно и безопасно обновлять allowed destinations. Если на L7/TLS layer - нужен корректный SNI/hostname matching.

TLS chain

Минимальная диагностика из sandbox:

```

getent ahosts api.example.com
nc -vz api.example.com 443
openssl s_client -connect api.example.com:443 -servername api.example.com </dev/null
curl -sv https://api.example.com/health
  
```

Первые команды последовательно проверяют resolver, TCP reachability, TLS/SNI/certificate chain и HTTP/application. Это быстрее, чем сразу менять Gateway manifest.

Negative tests обязательны

После разрешения `model.internal:443` проверьте не только успешный call туда, но и отсутствие доступа к запрещённому control endpoint, metadata service, RFC1918 ranges и произвольному Internet host. Security acceptance test должен доказывать и allow, и deny.

Dependencies надо инвентаризировать

Agent image может обращаться не только к model API: Git, package mirrors, OAuth/OIDC, MCP servers, DNS, NTP, artifact registry, telemetry collector. Runtime package installation в production Task часто превращает tight egress policy в allow-all. Лучше собирать immutable images заранее.

MCP gateway

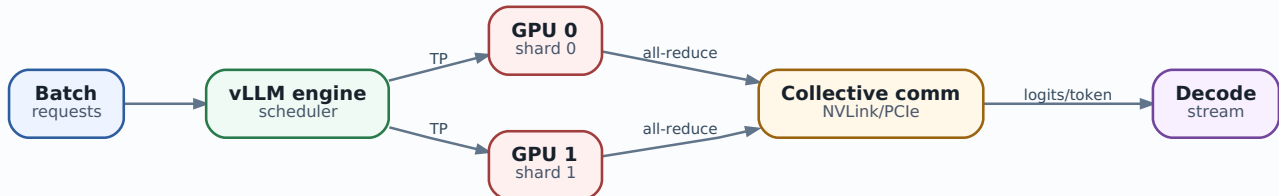
Для большого числа external MCP/API endpoints полезен controlled egress proxy/gateway: central auth, destination allowlist, rate limits, audit, DLP и credential brokering. Тогда sandbox знает только один restricted endpoint, а не получает десятки raw secrets.

Incident flow

Проверяйте: Gateway desired/status -> DNS result -> policy target -> route -> TCP -> TLS/SNI -> HTTP -> application auth -> MCP/model protocol. AX issue с hostname Gateway/TLS на ранней версии - хороший пример того, почему каждый слой нужно тестировать отдельно.

Local inference sizing: память GPU, parallelism и эксплуатационные SLO

Tensor parallelism: extra GPUs add communication, not just VRAM



VRAM - это не только веса модели

Приближённый бюджет GPU memory состоит из: weights + KV cache + activations/workspaces + CUDA/runtime allocations + fragmentation/reserve. Поэтому модель, чьи quantized weights «влезает в 24 GB», ещё не гарантирует нормальный serving при длинном context и concurrency.

KV cache зависит от workload

KV pressure растёт с количеством одновременно активных sequences и их фактической длиной context. Для sizing собирайте распределение input/output tokens, а не один `max_model_len`. Если 99% запросов имеют 4k tokens, резервировать поведение как при 128k для каждого slot может быть слишком дорого.

Tensor parallelism

Tensor parallel делит вычисления слоя между GPU и требует collective communication практически на каждом decode step. На NVLink/NVSwitch это одна экономика, на PCIe - другая. Добавление второй GPU может быть необходимо для capacity, но не обязано удваивать tokens/s и иногда ухудшает latency для небольшой модели.

Pipeline и data parallel

Pipeline parallel полезен, когда модель не помещается на одном устройстве/TP domain, но вводит bubbles и complexity. Data parallel/replicas часто проще для независимых запросов, если модель целиком помещается на одной GPU. Выбор определяется model fit, topology и SLO.

CPU fallback

llama.cpp/GGUF позволяет построить воспроизводимый CPU-only lab и аварийный low-throughput path. Но CPU fallback не следует считать равнозначным GPU production endpoint: TTFT и decode latency могут изменить поведение timeouts всего agent harness.

vLLM knobs

Memory utilization, max model length, max concurrent sequences, batching/scheduling и parallelism flags должны документироваться **по конкретной версии vLLM**. Не копируйте tuning command из старой статьи без сверки `--help` и release docs.

OpenAI-compatible != identical

Совместимость endpoint означает похожий API surface, но не гарантирует одинаковые параметры, tool-call semantics, structured output, token accounting или error codes. Model adapter должен иметь provider-specific capability tests.

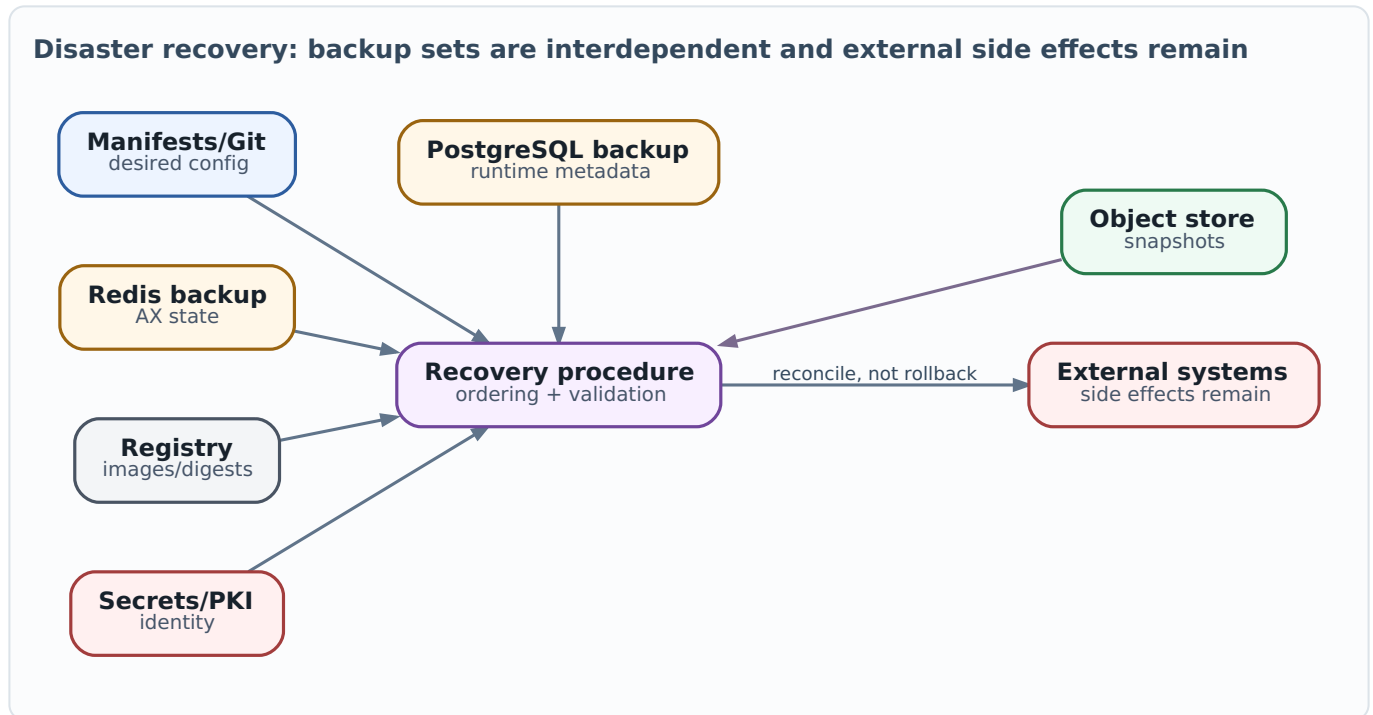
SLO сначала, GPU потом

Для agent platform определите: допустимый TTFT, p95 end-to-end model latency, target concurrent active generations, peak prompt tokens/s, output tokens/s, max queue delay и availability. Только после этого выбираются model size, quantization и GPU topology.

Интеграция с АХ

Model endpoint лучше предоставлять через стабильный internal service URL. Agent Task не должен знать физический GPU host. Observability связывает Task/Actor ID с inference request ID, чтобы можно было отличить «агент завис» от «10 секунд стоял в model queue».

Production security, secrets и disaster recovery



Классифицируйте secrets по назначению

Разделяйте control-plane credentials, model API keys, MCP/service credentials, registry pull secrets, user delegated tokens и infrastructure admin secrets. Один универсальный secret для всех Tasks уничтожает tenant isolation.

Injection time имеет значение

Статический secret в image - худший вариант. Лучше short-lived credential при start/resume или on-demand exchange через broker/проxy. Тогда suspend длительностью сутки не означает, что проснувшийся agent продолжит использовать credential, срок/полномочия которого уже изменились.

Rotation после resume

Harness должен предполагать, что credential после длительного сна истёк. Resume path включает refresh/re-auth, а не только повтор старого request. Ошибка 401 после resume должна маршрутизироваться в credential recovery, а не в бесконечный LLM retry.

Tenant boundary

Namespace сам по себе не является полной multi-tenant security boundary. Нужны RBAC, service accounts/workload identity, NetworkPolicy/egress, sandbox isolation, secret scope, storage isolation и audit. Для hostile code выбирается более сильный sandbox class.

Что резервировать

Минимальный backup set: pinned manifests/Helm values, AX control state, Substrate dynamic state database, snapshot/object storage, PKI/identity configuration, registry references и external system mappings. Git repositories и CRM/CMDB могут быть отдельными authoritative systems и не дублироваться без необходимости.

RPO/RTO по слоям

Не задавайте одно RPO «для AX». Desired manifests могут иметь RPO почти ноль в Git, Redis/PostgreSQL - минуты, snapshots - часы, transient logs - вообще не восстанавливаться. RTO control plane и RTO каждого dormant Actor также разные.

Recovery order

Типовой порядок: identity/PKI -> Kubernetes core -> durable DB/object storage -> Substrate control plane -> WorkerPools -> AX state/control plane -> model/MCP dependencies -> workload resume. Если начать поднимать Actors до восстановления policy/identity, можно получить функционально работающую, но небезопасную систему.

External side effects не откатываются из backup

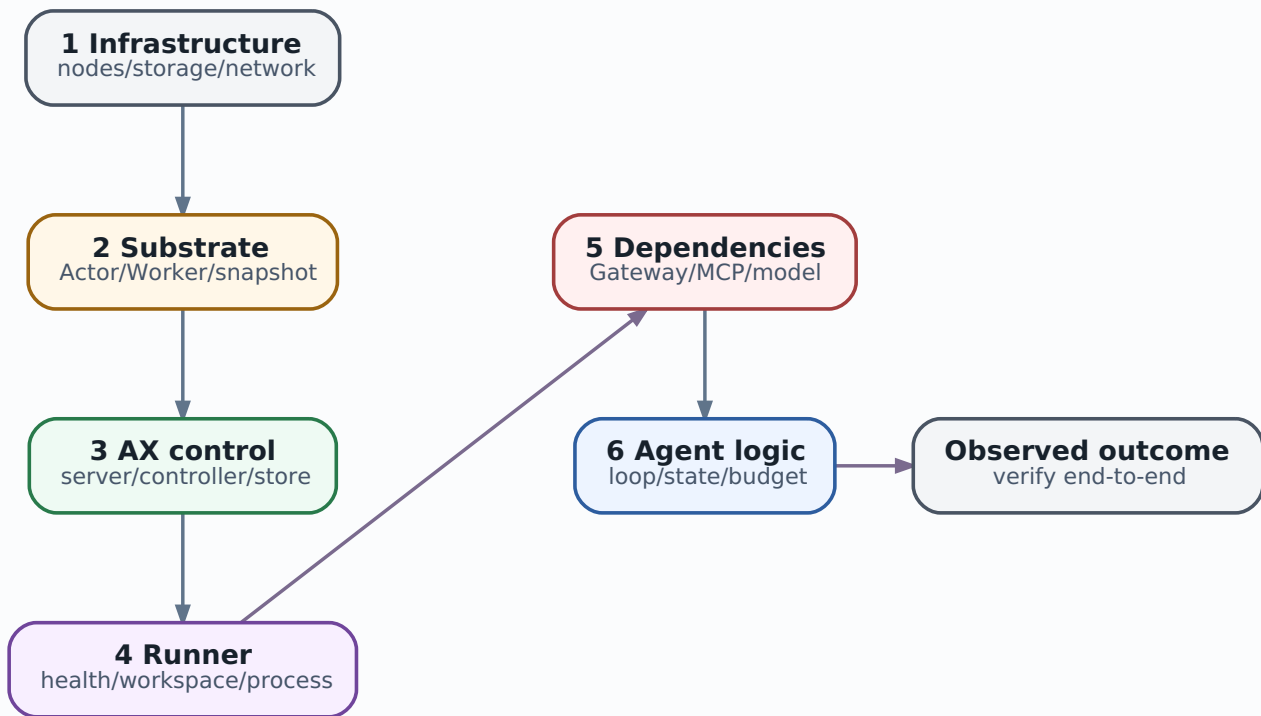
Восстановление Task к старому checkpoint не отменит уже созданный тикет, отправленное письмо или применённую сетевую конфигурацию. Поэтому ledger external actions и idempotency keys должны переживать disaster recovery.

DR drills

Проверяйте как минимум: loss одного worker, loss Kubernetes node, restart Redis/PostgreSQL, temporary object-store outage, restore control plane в новый cluster, недоступный model endpoint и восстановление task state при уже совершённом external action.

Operations runbook: диагностика по слоям и release acceptance

Troubleshooting ladder: идите от физического слоя к agent logic



Не начинайте инцидент с restart всего кластера

Сначала сохраните evidence: Task manifest/status, generation/conditions, controller lag/logs, Actor/Worker assignment, runner health, child process state, Gateway/DNS test, model request ID, MCP call ID и последние внешние side effects. Restart может удалить самую полезную информацию.

Слой 1 - Kubernetes/infrastructure

```

kubect1 get nodes -o wide
kubect1 get pods -A -o wide
kubect1 get events -A --sort-by=.lastTimestamp
kubect1 top nodes
kubect1 top pods -A
  
```

Проверяем pressure, scheduling, image pulls, storage/network и crash loops.

Слой 2 - Substrate

Проверяем WorkerPool capacity, Actor phase/assignment, snapshot URI/compatibility, resume errors, object-store reachability и atele/atecontroller logs. Если Actor не получил Worker, AX runner ещё вообще не участвует в проблеме.

Слой 3 - AX

Сверяем desired Task generation, controller reconcile/error/backlog, referenced Workspace/Model/Gateway, conditions и suspend/resume transition. Current-main split `ax-server/ax-controller` означает, что API может быть доступен при сломанном reconciliation.

Слой 4 - runner/workload

`/healthz` отвечает на вопрос о runner, `/readyz` - о readiness contract, но отдельно проверяется child process и task-specific progress heartbeat. Из debug Task полезны `ps`, environment metadata, `/workspace` contents и DNS/network probes.

Слой 5 - MCP/network/model

Проверяем DNS -> TCP -> TLS -> HTTP/protocol; затем auth, rate limit и server-side operation status. Для inference сравниваем queue time, TTFT, decode throughput, KV/VRAM pressure и OOM/eviction events.

Слой 6 - agent logic

Только после инфраструктурных проверок анализируем prompt/context, tool schema, retry loop, planning и memory. Это предотвращает типичную ошибку: «тюнинг prompt», когда MCP endpoint просто не резолвится.

Failure taxonomy

Используйте категории: desired-state admission, reconciliation, placement/capacity, restore/snapshot, runner/bootstrap, dependency/network, inference, tool authorization, external side effect, agent reasoning. Incident tag по слою облегчает статистику повторяемости.

Release acceptance suite

Перед обновлением AX/Substrate прогоните pinned набор: create simple Task; Workspace Git bootstrap; MCP read; denied write; approved idempotent write + verification; suspend/resume with durable file; worker loss; forbidden egress; model call; child-process exit; snapshot restore; backup/restore control state. Результаты сохраняйте вместе с exact versions/commits.

Post-incident

Фиксируйте timeline, root layer, detection gap, unsafe retry/approval opportunities, lost observability и конкретный regression test. Для быстро меняющегося pre-1.0 stack regression suite важнее предположения «minor update ничего не ломает».

Приложение А. Аудит проектов и выявленные расхождения

A.1 AX

Release baseline: `v0.3.1`, commit `e70162a`, release 25.09.2026.

Current API family: `ax.io/v1alpha1`.

Stability: project explicitly warns about breaking changes before stable.

Ключевые факты current documentation/source audit:

- README описывает четыре user primitives: `Task`, `Workspace`, `Gateway`, `Model`.
- `docs/manifests.md` в current crawl говорит «all four kinds», но видимая часть документа подробно показывает `Task/Workspace/Model`; `Gateway` лучше проверять по `README/examples/API`, а не по одной странице.
- `cmd/ax-controller` существует и читает Redis stream consumer group, создавая `TaskReconciler` с Substrate client.
- `DESIGN.md` всё ещё описывает reconciliation внутри `ax-server`. Это documentation drift.
- Runner всегда PID 1 и restart после resume видит durable `/workspace`, но новое process tree.
- Current known issues показывают полезные ограничения: readiness может не отражать exit child command; workspace clone мог завершаться «успешно» при пустом repo; hostname `Gateway/TLS` имел bug; ssh/control-plane auth вопросы меняются.

Вывод: production runbook должен ссылаться на pinned release и regression tests, а не только на main docs.

A.2 Agent Substrate

Release baseline: `v0.3.0`, commit `ccecc78`, release 30.09.2026.

Status: pre-1.0; API compatibility не гарантирована.

Ключевые факты:

- Quickstart использует Kind и может разворачивать PostgreSQL/RustFS локально.
- WorkerPool - Kubernetes-side physical capacity; ActorTemplate/Actor/Worker dynamic state относится к Substrate API/state store.
- Current architecture использует PostgreSQL для dynamic control state и object storage для snapshots.
- gVisor и microVM - разные sandbox classes.
- Golden snapshot и Tag ownership - важная часть clone/resume semantics.
- Architecture document содержит прямое предупреждение, что часть design aspirational.

- v0.2.0/v0.3.0 быстро усиливали multi-actor networking, egress, identity, authorization/observability, поэтому старые статьи быстро устаревают.

A.3 MCP

Для книги принята specification 2026-07-28, потому что она меняет базовый mental model: stateless core, server/discover, request metadata versioning, новый subscription mechanism и изменения authorization/deprecations. Материалы 2024-2025 про обязательный initialize/session flow нужно читать как legacy compatibility, а не current normative behavior.

Приложение B. Production readiness checklist

Platform

- ☐ AX/Substrate/container images pinned по digest/tag.
- ☐ Compatibility matrix проверена в staging.
- ☐ Redis/PostgreSQL persistence и backup протестированы restore drill'ом.
- ☐ Object storage durability/throughput/retention проверены.
- ☐ WorkerPool headroom измерен burst benchmark'ом.
- ☐ Registry и DNS имеют понятные failure modes.

Security

- ☐ Default-deny egress; wildcard отсутствует вне исключений.
- ☐ Debug/SSH disabled by default.
- ☐ Scoped short-lived tool credentials.
- ☐ MCP servers/skills/repositories проходят allowlist/review.
- ☐ Write/destructive tools требуют policy/approval.
- ☐ Sandbox backend выбран по threat model.
- ☐ Control-plane API закрыт perimeter/authn/authz.
- ☐ Audit logs нельзя изменить workload'у.

Agent

- ☐ Max steps/tokens/wall time заданы.
- ☐ Retry limits и idempotency strategy заданы.
- ☐ Durable state не существует только в prompt/RAM.
- ☐ Child task depth/count budget задан.
- ☐ Tool calls имеют schema validation.
- ☐ Post-action verification есть для writes.

Observability

- ☐ Trace связывает user/job → Task → Actor → tool/model request.
- ☐ TTFT/ITL/queue/KV metrics собираются.
- ☐ Suspend/resume/snapshot metrics собираются.
- ☐ Central logs переживают Worker relocation.
- ☐ Alerts основаны на SLO, а не только CPU.

Приложение С. Карта источников

Основные первичные источники, использованные для аудита:

1. Google AX repository: <https://github.com/google/ax>
2. AX README: <https://github.com/google/ax/blob/main/README.md>
3. AX DESIGN: <https://github.com/google/ax/blob/main/DESIGN.md>
4. AX concepts: <https://github.com/google/ax/blob/main/docs/concepts.md>
5. AX manifests: <https://github.com/google/ax/blob/main/docs/manifests.md>
6. AX runner: <https://github.com/google/ax/blob/main/docs/runner.md>
7. AX roadmap: <https://github.com/google/ax/blob/main/docs/roadmap.md>
8. AX current controller source: <https://github.com/google/ax/blob/main/cmd/ax-controller/main.go>
9. AX issues: <https://github.com/google/ax/issues>
10. Agent Substrate repository: <https://github.com/agent-substrate/substrate>
11. Substrate architecture: <https://github.com/agent-substrate/substrate/blob/main/docs/architecture.md>
12. Substrate API guide: <https://github.com/agent-substrate/substrate/blob/main/docs/api-guide.md>

13. Substrate observability: <https://github.com/agent-substrate/substrate/blob/main/docs/observability.md>
14. Substrate threat model: <https://github.com/agent-substrate/substrate/blob/main/docs/threat-model.md>
15. Kubernetes Concepts: <https://kubernetes.io/docs/concepts/>
16. Kubernetes Custom Resources: <https://kubernetes.io/docs/concepts/extend-kubernetes/api-extension/custom-resources/>
17. gVisor: <https://gvisor.dev/docs/>
18. MCP specification 2026-07-28: <https://modelcontextprotocol.io/specification/2026-07-28>
19. MCP release notes: <https://blog.modelcontextprotocol.io/posts/2026-07-28/>
20. vLLM docs: <https://docs.vllm.ai/>
21. vLLM releases: <https://github.com/vllm-project/vllm/releases>
22. llama.cpp: <https://github.com/ggml-org/llama.cpp>
23. Kind: <https://kind.sigs.k8s.io/>
24. K3s: <https://docs.k3s.io/>
25. kubeadm: <https://kubernetes.io/docs/setup/production-environment/tools/kubeadm/>

Веб-документация была проверена 03.10.2026. Для быстро меняющихся `main` страниц это дата среза, а не гарантия неизменности.